Dipartimento di Ingegneria e Architettura

CORSO DI LAUREA IN INGEGNERIA DELL'ENERGIA ELETTRICA E DEI SISTEMI

PROVA FINALE

AN IMPLEMENTATION OF OPEN-RMF IN AN OFFICE ENVIRONMENT

Laureando: Michele Belletti

Relatore: Stefano Seriani
Correlatori: Roberto Pugliese
Georgios Kourousias

ANNO ACCADEMICO 2024-2025

Vorrei ringraziare il professore Stefano Siriani, relatore di questa tesi, per l'aiuto fornitomi. I miei correlatori Roberto Pugliese e Georgios Kourousias, correlatori di questa tesi di laurea, oltre all'aiuto fornitomi, per la grande conoscenza che mi hanno donato, per la precisione e la disponibilità dimostatami durante tutto il periodo di stesura.
Inoltre vorrei ringraziare l'Elettra Sincotrone Trieste e tutti i miei colleghi del TeambuildingSSI (Roberto, Andrey, Francesco, Valentina, Alessandro, Adriano, Martin, Milan, Michele, Iztok, Marco, Emiliano, Matteo).
Vorrei ringraziare i miei genitori e la mia sorellina Alessandra per i loro saggi consigli, la loro capacità d ascoltarmi, per avermi spronato e spinto a dare sempre il meglio di me stesso.
Per ultimi ma non meno importanti, i miei amici più cari: Marco, Luca, Lorenzo, Matteo, Giorgia, Fabio, Patrizio e Giulia che mi sono stati vicini e mi hanno supportato durante tutto il mio percorso.
Un sentito grazie a tutti!

Contents

1. Introduction

1.1 Background and motivation

Elettra Sincrotrone Trieste[1] is a major research facility that primarily provides users access to two light sources: the synchrotron Elettra and the free electron laser FERMI. In addition to operating these sources, the facility actively researches new technologies and techniques to enhance its capabilities and operations. Many recent advancements are based on robotics, which, especially after the COVID-19 pandemic and with the rapid rise of large language models (LLMs), have enabled new ways for users to interact with the infrastructure, including robot-assisted information retrieval and package delivery services. Previously, the facility developed a global planner, D-Star (D-star based optimized trajectory planner)[41], within the ROS 1 framework to enable robot navigation across large-scale environments. However, as ROS 1 has reached end-of-life, the codebase must now be migrated to ROS 2.
Additionally, the facility has multiple robots, some of which are yet to be built, while others currently operating on ROS 1 that need to be ported also to ROS 2. Furthermore, the facility utilizes robots from various vendors, each specialized for different tasks and not designed for seamless coordination with one another and all is in an environment that has narrow passages and lifts, which can create significant coordination challenges during navigation. To address this issue, Open-RMF[2] has been developed in recent years as a multiplatform system for controlling and managing heterogeneous fleets of robots.

1.2 Research questions and objectives

The objectives of this thesis are: to port the D-Star global planner from ROS 1 to ROS 2, to integrate a Rover Mini unmanned ground vehicle (UGV) (see 1.1a) from the company RoverRobotics[3] into the ROS 2 Nav2 ecosystem making it a Autonomous Mobile Robot (AMR), to port the D-Star global planner from ROS 1 to ROS 2, upgrade our Jobot AMR (see 1.1b) from ROS 1 to ROS 2, to incorporate the robots into the Open-RMF framework for centralized fleet management and evaluate the suitability of the Open-RMF ecosystem for deployment at the Elettra Sincrotrone Trieste.

[1] https://www.elettra.eu/
[2] https://www.open-rmf.org/
[3] https://roverrobotics.com/

(a) RoverRobotics Rover Mini

(b) Jobot Rover

Figure 1.1: Robots used

1.3 Scope and limitations

The principal limitation is the current lack of access to lift control systems, which prevents their direct integration and operation by Open-RMF. Additionally, there is no automated system for opening and closing doors, another feature that can be managed by Open-RMF. While Open-RMF offers a high degree of customizability, and human-in-the-loop interactions can be configured—such as triggering a light or sending a notification when a robot approaches a door—this method requires a human to manually open the door or operate the lift. Such an approach, however, is not ideal for achieving full autonomy.

2. State of the Art

The integration of autonomous robots into our daily environments is increasing significantly to reduce costs associated with package logistics[28][30] and their use in various environments such as hospitals[18][11], mining [13], warehouses[31][16] and agriculture[6]. For a long period, the solution for moving objects in various industries has been the use of automated guided vehicles (AGVs). These vehicles rely on markers or laser systems to navigate their surroundings. However, in environments such as office buildings, where interaction with humans is constant, the use of automated guided vehicles (AGVs) poses challenges. AGVs often cannot bypass obstacles or cannot enter some areas due to safety and space concerns, so it necessitates the use of autonomous mobile robots (AMRs)[37] with their more autonomy capabilities can overcome these challenges. There are several types of AMRs that can be divided by the type of locomotion: wheeled robots, tracked robots and legged robots. Since wheeled robots offer several advantages, such as energy efficiency, higher payload capacity[38], increased stability [33], lower mechanical complexity, and ease of control in structured environments[4] they are the optimal solution for tasks that are performed in an office. Wheeled robots can adopt various locomotion models: differential drive systems (which utilize differential steering or skid steering techniques), omni-wheels and Ackermann steering. Given the advantages of the differential drive, such as greater payload and increased maneuverability without the need for a mechanical steering subsystem, the differential drive structure is the most used for robotics application [45][38] in small spaces.
These robots leverage various proprietary software solutions alongside open-source platforms such as Ardupilot[19] and Robot Operating System ROS[32] to navigate in the environment. Originally developed as an academic project, ROS has greatly expanded its application scope[23]. Transitioning to ROS 2 introduced new features for data transfer[29], executing nodes[22],lifecycle node management, and security[40], which facilitate its adoption in production environments (see 2.1 and 2.3).
Among the projects built on ROS[15][24] for navigation, Nav2 stands out for its ability to manage autonomous navigation[20][3]. Given the robot's position estimation within a map, which can be obtained through state estimation utilizing sensor data [26] in combination with a Simultaneous Localization And Mapping (SLAM)[14][21][25][5] or through localization alone [12][8] if a map is already available, a robot can effectively navigate its environment (see 2.2). In our current work, we aim to control a two differential drive Unmanned Ground Vehicle (UGV) using Nav2. Its modularity allows for the evaluation of multiple global [34][41] and local [10][44] planners.
Given the necessity to manage multiple robots, a solution for overseeing a heterogeneous fleet of robots is required. Additionally, while AGVs effectively manage choke points with fleet management software to prevent congestion, AMRs often struggle without such coordination, resulting in delays and traffic congestion[37]. Among the available options (see 2.4), we will use the Open Robotics Middleware Framework (Open-RMF)(see 2.4.1), a middleware for robots that has obtained substantial interest[35][17] for its capabilities in traffic management and parallel

task execution.

2.1 Overview of ROS1 and ROS2

The Robotics Operating System (ROS)[1] is an open-source collection of tools and libraries designed to support the development of robotics applications[32][23]. Despite its name, ROS is not an operating system (OS) in the traditional sense, but rather a software development kit (SDK) that provides developers with a comprehensive set of tools for building robotic systems. It offers several features typically associated with an OS, including hardware abstraction, low-level device control, implementation of commonly used functionalities, inter-process communication and package management. The ROS ecosystem is built upon the following core components:

- **Middleware communication layer (RMW)**: Built on the Data Distribution Service (DDS) standard[29], this layer enables nodes to exchange information using an anonymous publish/subscribe pattern. This design promotes fault isolation, separation of concerns and modular interfaces. ROS also supports synchronous, Remote Procedure Call (RPC)-style communication between services.
  While the default DDS implementation is eProsima Fast DDS (`rmw_fastrtps_cpp`), this work uses Eclipse Cyclone DDS (`rmw_cyclonedds_cpp`) due to its better compatibility with the Nav2 navigation stack in ROS Humble and it is the one used in Free Fleet (see 3.4.3).

- **Multiple Client Libraries**: Allow programming with ROS using multiple languages and access to the core communication APIs.

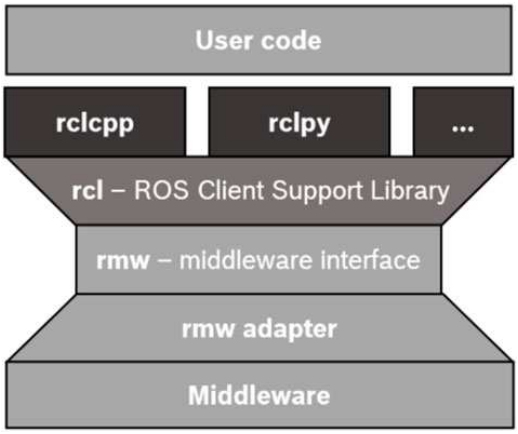


Figure 2.1: ROS 2 Client Library API Stack

- **Development tools**: ROS provides a suite of tools for managing and enhancing software development, including utilities for launching, introspection, debugging, visualization, plotting, logging, and playback.

- **Multi-machine support** ROS includes tools and libraries for retrieving, building, writing and executing code across multiple computers and OS.

- **Community and governance**: ROS is supported by an active and extensive global community, coordinated primarily by Open Robotics[2], which serves as the central organization for its development and maintenance.

[1] https://www.ros.org/
[2] https://www.openrobotics.org/

2.1.1 Structure of ROS

The official ROS documentation[3] provides comprehensive information about the system. All communication in ROS is handled through the **ROS Graph**, a peer-to-peer network of processes (called nodes) that execute concurrently and exchange information with one another. Some components of this graph are described below. Note that some of the functionalities described are specific to ROS 2 and are not available in ROS 1.

- **Nodes**: A node is a modular process that should perform a specific task. Nodes communicate with each other through topics, services, actions and parameters. In ROS 2, nodes can be implemented as **LifecycleNode**, which provide a managed state machine with defined transitions for startup and shutdown, enabling more deterministic behavior.

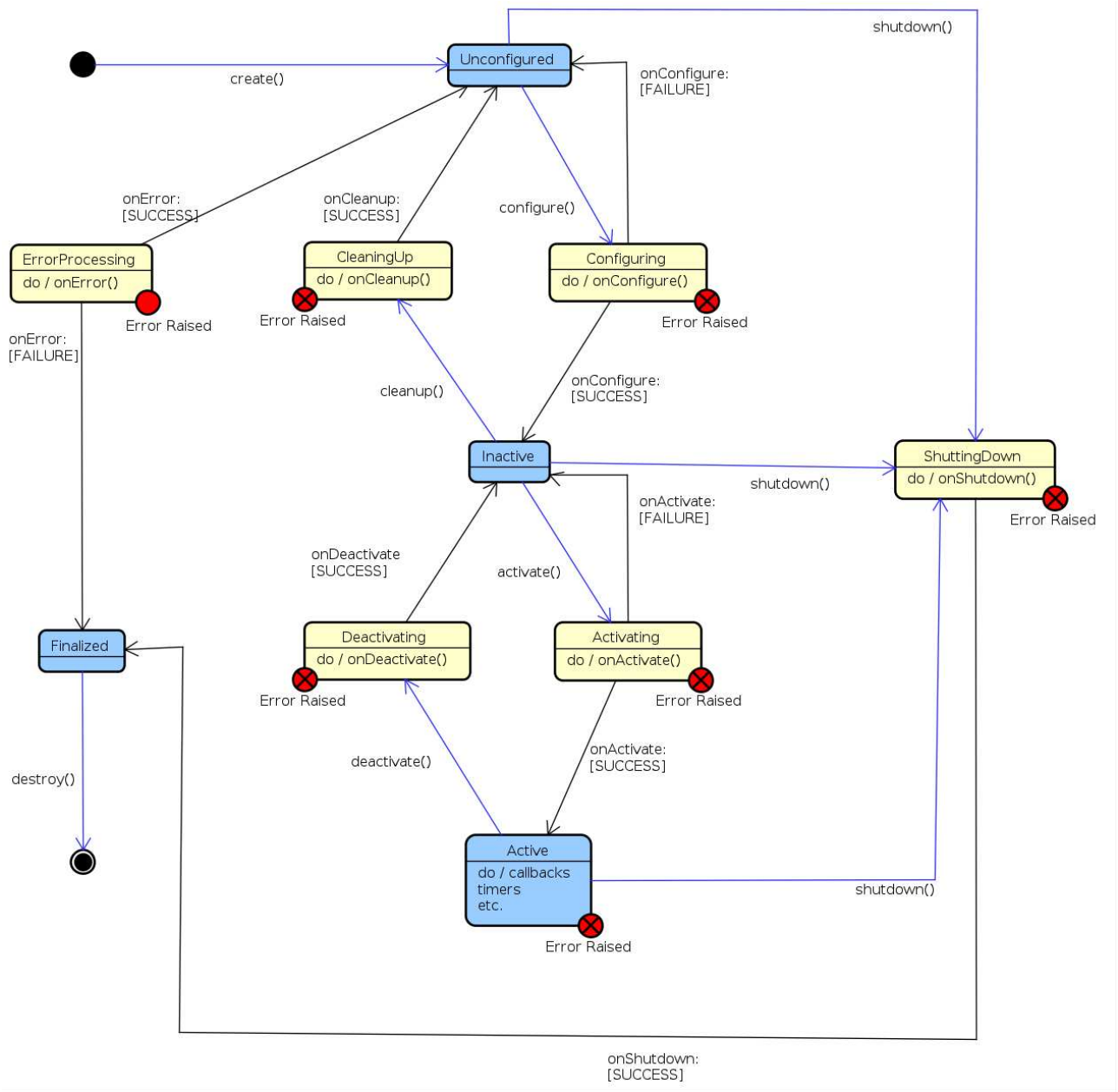


Figure 2.2: State Machine of LifecycleNode

[3]https://docs.ros.org/en/rolling/index.html

Nodes can also be instantiated into a process using the **Composition** feature, which also allows bypassing middleware overhead through intra-process communication, reducing RAM and CPU usage while improving goodput and latency[22].

- **Topics**: Topics are used for communication between nodes to share streams of messages in a publisher-subscriber structure.

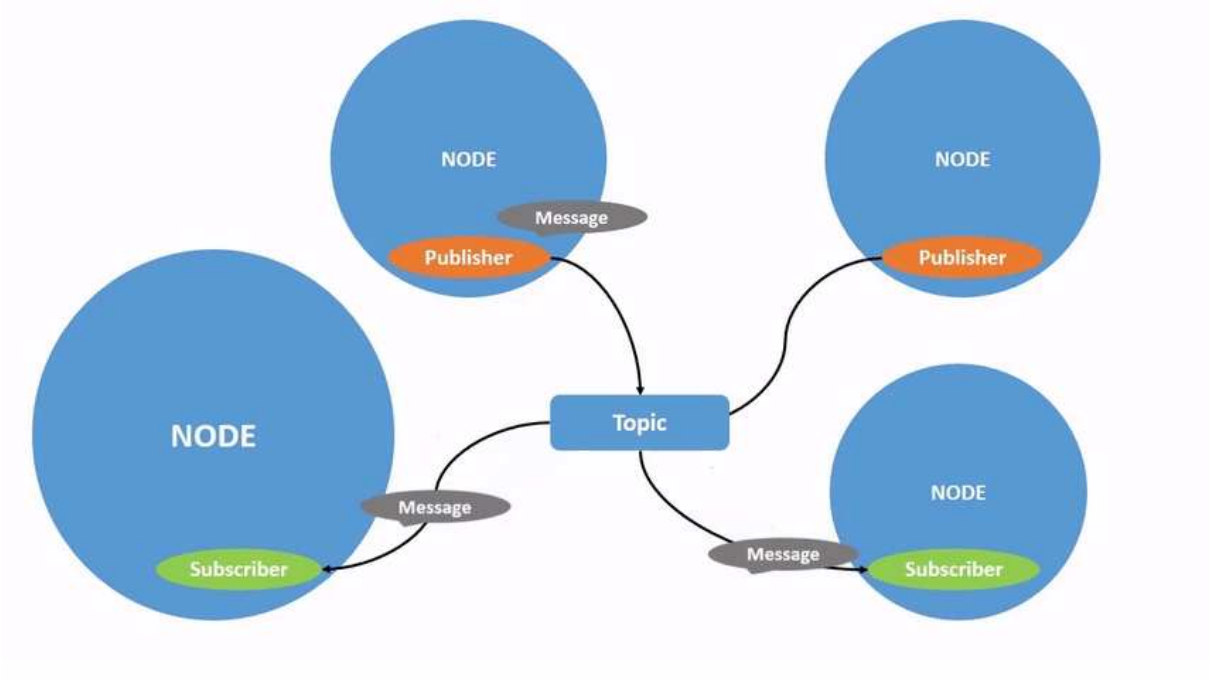


Figure 2.3: Topic process

- **Service**: Services are based on the request-response model. When a node with a service client sends a Request Message to a node with a service server, it receives a Response Message from that specific server.

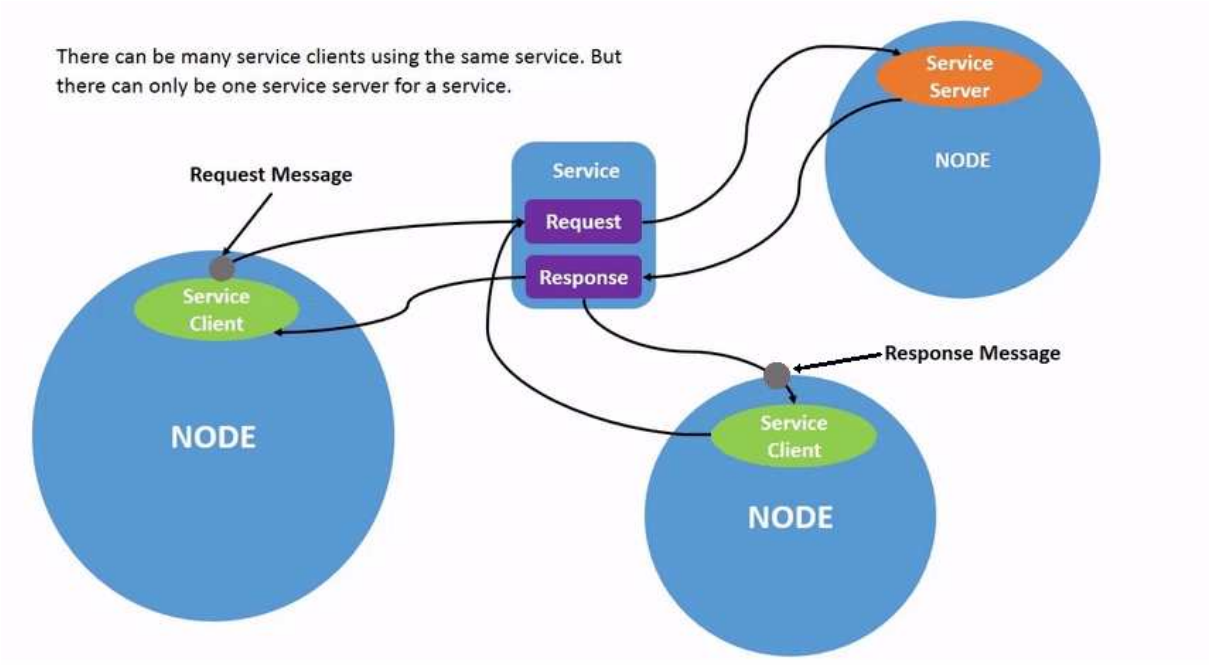


Figure 2.4: Service process

- **Parameters**: Parameters are used to configure nodes at startup (and during runtime), without changing the code.
- **Actions**: Actions are designed to support long-running tasks that require feedback and the capability to interrupt when needed. They combine two services (goal and result) and a publisher-subscriber mechanism (feedback). A node sends a goal request and receives an acknowledgment, then starts publishing feedback messages while executing the task, and finally the action server responds with the result.

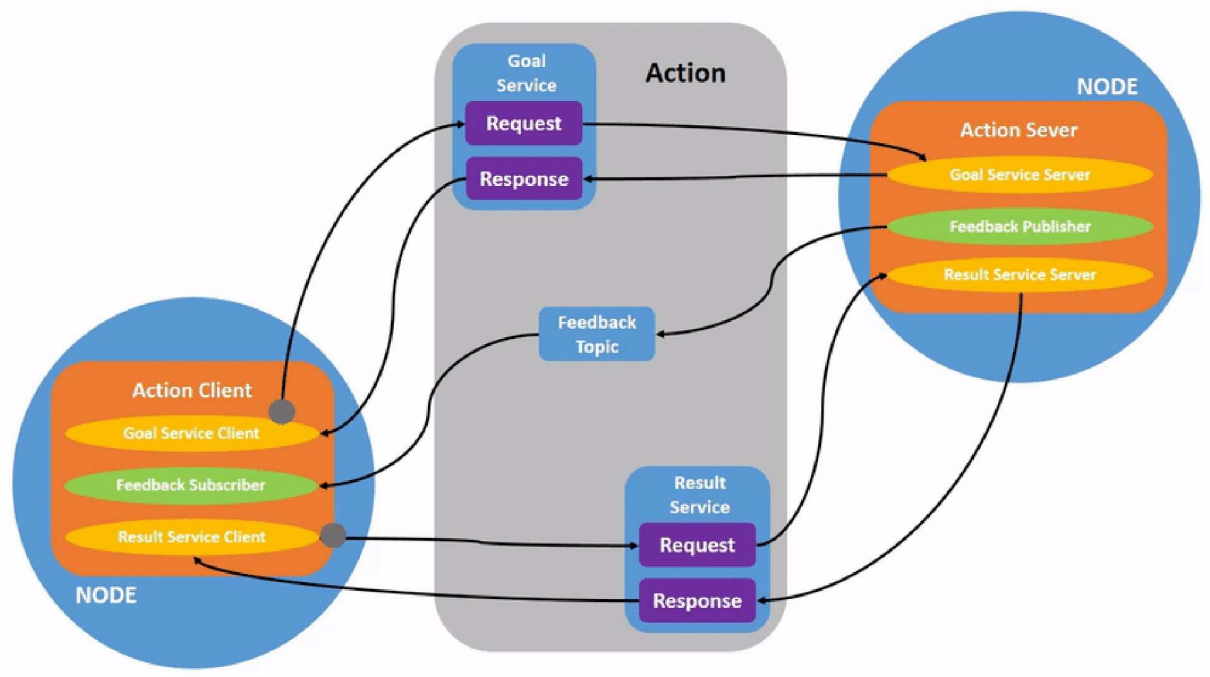


Figure 2.5: Action process

2.1.2 Tools of ROS

This section describes the tools available in ROS for developing robotic systems:

- **Launch**: A file structure that allows the simultaneous start of multiple nodes with a set of parameters.
- **rqt**: A graphical user interface (GUI) tool for ROS. It includes a series of plugins that facilitate the development, monitoring, and diagnostics of the ROS system.
- **Rviz**: A 3D visualization tool for the Robot Operating System. It allows the visualization of sensor data and state information from ROS.

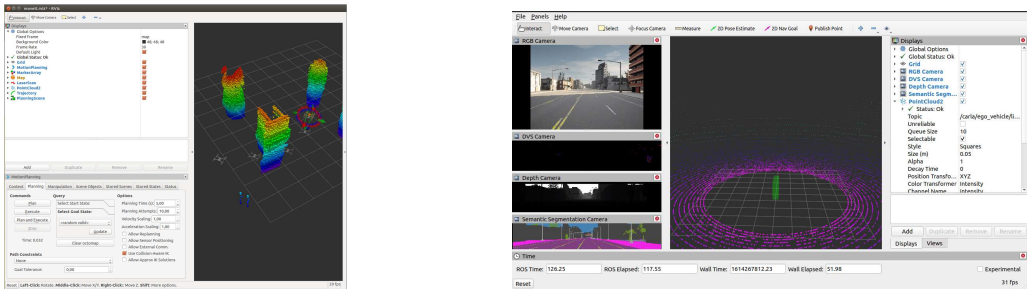


Figure 2.7: Examples of visualization of ROS data with Rviz

- **rosdep**: A command-line utility for installing system dependencies required by ROS packages.

- **sros2**: A security extension for ROS 2 that provides tools and instructions to apply security policies to DDS (Data Distribution Service)[40].

Another important category of tools that are integrated with ROS is simulation environments. These environments allow developers to test their code on simulated robots under various conditions, which helps identify bugs that could potentially harm a real robot or its environment. Simulation tools provide realistic physical behavior for all components of a robotic system. According to the ROS documentation, two main simulation tools are commonly used: Gazebo and Webots. Gazebo is also maintained by Open Robotics and is widely adopted in many projects, including the one discussed in this thesis.
Gazebo offers:

- **ROS Integration**: A bridge that converts Protobuf messages to ROS messages, enabling communication between Gazebo and ROS.

- **Performance tools**: Features for distributed simulation across servers, dynamic asset loading, and adjustable time steps to optimize simulation performance.

- **Realistic simulation**: Support for a variety of sensors, including their noise models, within a 3D world rendered using OGRE 2.1[4] and powered by multiple physics engines; the default one is DART[5].

- **Extensibility**: A modular design that supports plugins, allowing developers to run custom code that interacts directly with the simulation environment.

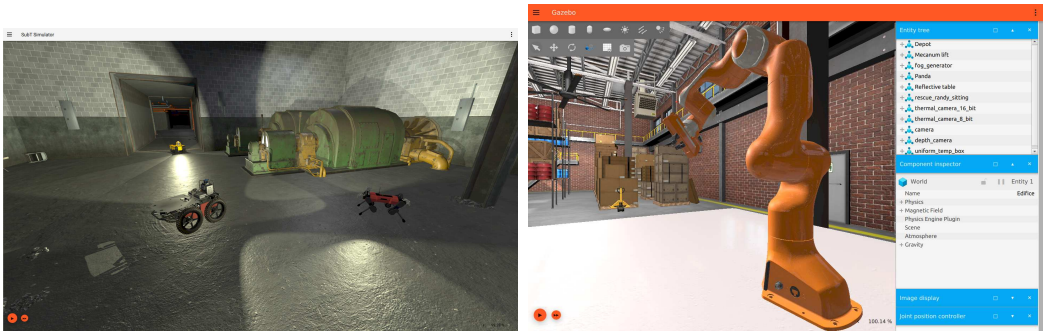


Figure 2.9: Examples of Gazebo

2.2 Introduction to NAV2 and Google Cartographer

Multiple tools are available to enable a robot to move autonomously within an environment. One of the most complete systems is Nav2 [6] [24], the successor to the ROS 1 Navigation Stack.

[4]https://wiki.ogre3d.org/Home
[5]https://dartsim.github.io/
[6]https://nav2.org/

2.2.1 Description of Nav2

Nav2 provides a comprehensive set of tools for perception, planning, control, localization, and visualization, allowing any robot to navigate autonomously and reliably, even in complex environments[34]. It enables the construction of an environmental model from sensor data, the dynamically calculating of the optimal path to avoid obstacles, the setting of motor velocities, and the executing of navigation tasks using Behavior Trees.

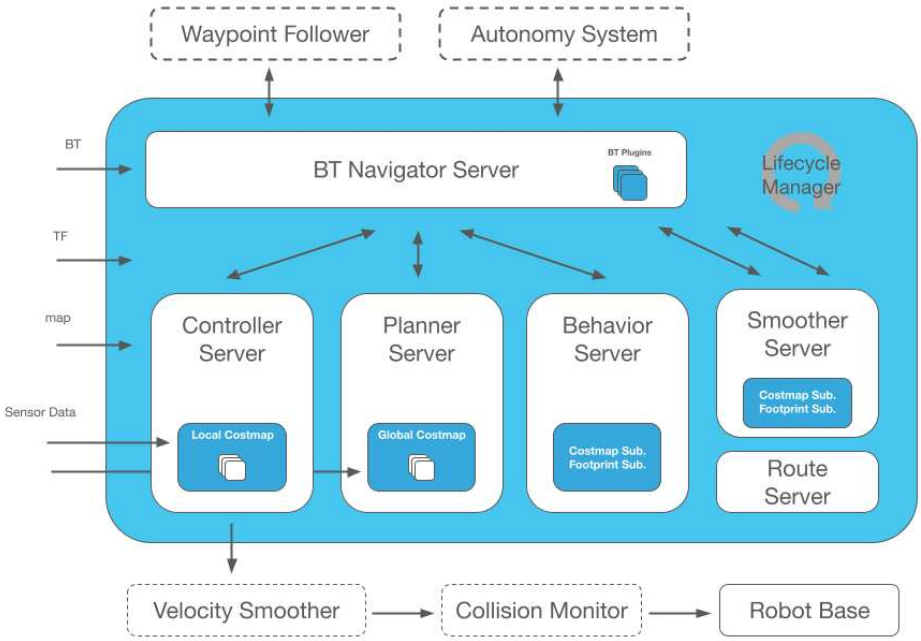


Figure 2.10: Nav2 architecture

Nav2 introduces several improvements over ROS 1, including:

- **Behavior trees**[7]: Nav2 uses Behavior Trees to structure and execute tasks in a human-readable and modular way. These trees enable the creation of complex systems and can be tested with various tools. The BehaviorTree.CPP[7] library is used to implement them, and it allows linking different trees for flexible behavior management.
- **nav2_util LifecycleNode**: This is a wrapper around the standard `rclcpp_lifecycle::LifecycleNode` described earlier in 2.1.1. It adds a bond connection to the Lifecycle Manager node, which monitors the state of each node and can safely transition nodes through their lifecycle states in the event of a failure.
- **Action Server**: Described in 2.1.1, Nav2 uses action servers for Behavior Tree interactions, such as NavigationToPose (navigate to a specific location) and NavigationThroughPoses (navigate through multiple waypoints).

Nav2 is composed of several key nodes that work together to control the robot:

- **Controller Server**: Handles movement requests. It uses plugin-based modules such as a progress checker (to verify if the robot is making progress), a goal checker (to determine if the robot has reached its destination) and a controller (to generate velocity commands using the **local costmap** while avoiding collisions).

[7]https://www.behaviortree.dev/

- **Planner Server**: Computes the global path to the destination using sensor data and a plugin-based global planner loaded with the **global costmap**.
- **BT Navigator Server**: Implements the NavigateToPose, NavigateThroughPoses, and other task interfaces. It is a Behavior Tree-based implementation of navigation that is intended to allow for flexibility in the navigation task and provide a way to easily specify complex robot behaviors.

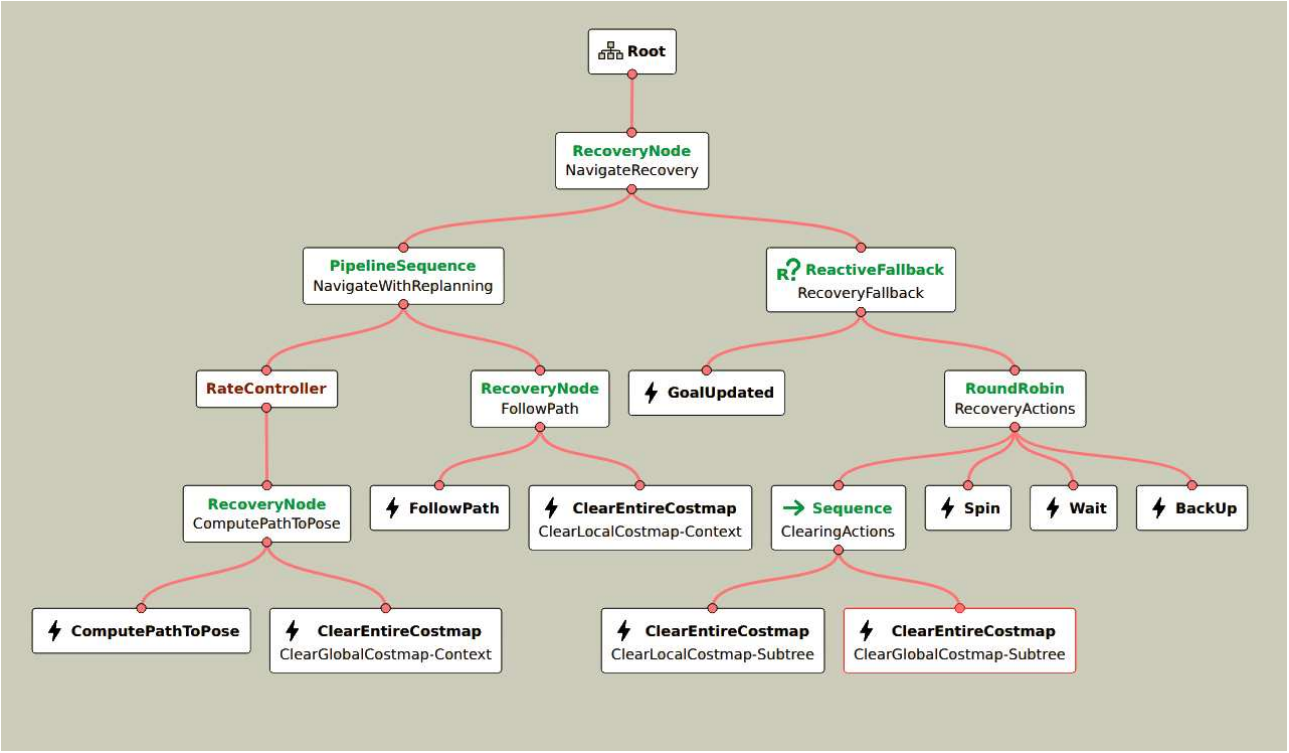


Figure 2.11: Humble Nav2 BT navigation `navigate_to_pose_w_replanning_and_recovery.xml` loaded with Groot2

- **Smoother Server**: Smooths the path, minimizing sharp turns and rapid rotations.
- **Behavior Server**: Handles specific behavior actions like spinning, backing up, and waiting.
- **Waypoint Follower**: An example of orchestrated control (an **Autonomy System**), this node takes a list of waypoints and uses the NavigateToPose action server to move between them, optionally performing custom tasks (e.g., taking a photo) at each waypoint through plugins.
- **Route Server**: Calculates routes through a predefined navigation graph instead of using free-space planning.
- **Velocity Smoother**: Smooths velocity, acceleration, and deadband signals sent to the robot to ensure safe and smooth motion.
- **Costmap** is a representation of the environment using a regular 2D grid of cells with a cost (unknown, free, occupied or inflated) and is used for generate the global plan or to compute local control efforts.

And require the following information for navigation:

- A **state estimation** of the robot through a suitable TF tree, a tree structure of coordinate frames generated using the transform library[9], based on the REP 105, that should provide the following frame structure: `map` → `odom` → `base_link`. According to the REP 105, they are defined as:
  - `map` : A world-fixed frame that may change in discrete jumps. It's useful for long-term global reference but not suitable for short-term localization.
  - `odom` : A continuous world-fixed frame that may drift over time. It provides short-term local reference.
  - `base_link`: A robot fixed frame.

  The `odom` → `base_link` link can be estimated using various sensors. For improved accuracy, we use the `robot_localization` package[26], which provides nonlinear state estimation using a Kalman Filter that fuses the robot's odometry and the IMU data.
  The `map` → `odom` link can be generated with a global positioning system like Simultaneous Localization And Mapping (SLAM) method[5] or using the one provided by Nav2, Adaptive Monte-Carlo Localization (AMCL).
- **map** can either be preloaded or built dynamically using a SLAM method. One SLAM method system is Google Cartographer, which will be described in the next section. It is also the SLAM method that was used to generate the map we use for navigation.

2.2.2 Cartographer

Cartographer[14] is a SLAM method that operates using two main subsystems: a **Local SLAM** (called **Frontend**) that builds a series of locally consistent **submaps** from successive scans and from the pose extrapolator with the possibility that it can drift during acquisition and a **Global SLAM** (called **Backend**) that reduces drift by performing loop closure—detecting when the robot has returned to a previously visited location. It uses scan matching against submaps and applies global optimization to align all submaps consistently and build a globally accurate map. Cartographer is capable of achieving mapping accuracy of 5 cm, but it requires extensive parameter tuning to reach optimal performance.

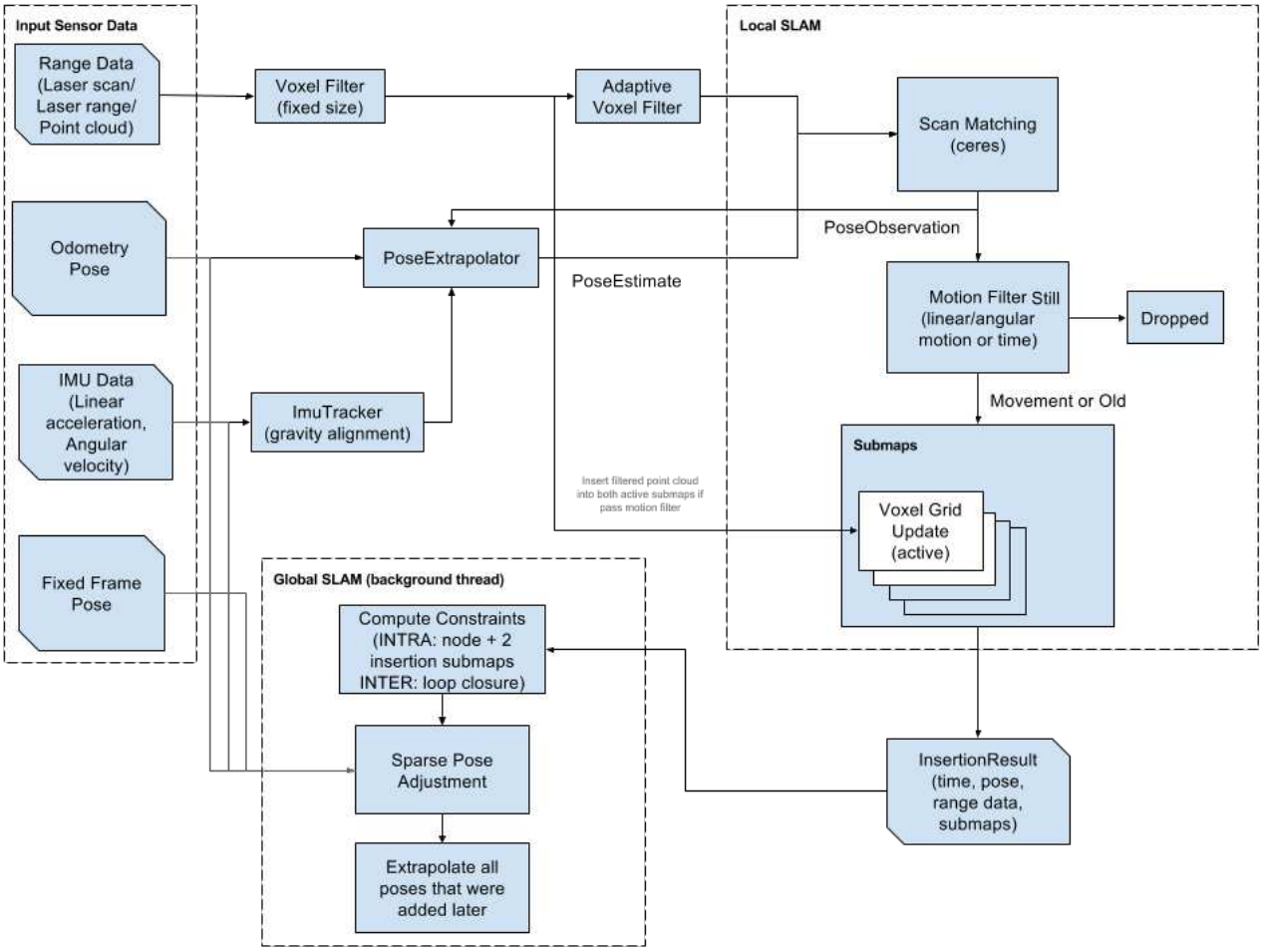


Figure 2.12: Cartographer Algorithm

2.3 Review of related work on porting navigation stack from ROS 1 to ROS 2

The widespread adoption of the Robot Operating System (ROS) has introduced new use cases that were not originally considered during its development. Initially created as an academic project, ROS encountered limitations as users began applying it to more complex scenarios such as fleet management, deployment on embedded boards and microcontrollers, real-time systems, unreliable networks and industrial applications. These emerging requirements highlighted architectural constraints in ROS 1, ultimately necessitating a complete redesign, which

led to the development of ROS 2. Comprehensive documentation for ROS 2 is available at https://design.ros2.org/ . Today, many companies have adopted the ROS 2 ecosystem, noting improvements in development workflows [23] and in specific subsystems such as Nav2 [34]. New initiatives, such as micro-ROS[8], aim to bring ROS 2 capabilities to microcontrollers. Additionally, features like Node Composition offer improved performance optimization [22] while sros2 allows for a better security [40]. As outlined in these references, ROS 2 offers substantial advantages over ROS 1, making it the preferred framework for modern robotic applications. With this in mind, we will migrate the global planner using the official ROS 2 documentation[9] and various secondary sources [10].

2.4 Overview of open source fleet management systems, particularly Open-RMF

Currently, several fleet management systems are available:

- **Isaac Mission Dispatch**[11]: A microservice-enabled fleet management system developed by NVIDIA that allows users to submit missions to multiple robots and monitor both robot and mission statuses. It provides connectivity between robots and the fleet management system but does not handle logistics such as task allocation or conflict resolution (e.g., intersecting robot paths). The implementation relies on the VDA5050 protocol, an industry standard for communication between cloud control services and mobile robots, and uses MQTT, a lightweight, publish-subscribe network protocol designed for devices with limited resources and bandwidth.

- **ROOSTER Fleet Management**[12]:An open-source ROS 1-based project that supports heterogeneous fleet management with task allocation, scheduling, and routing functionalities.

- **Open Robotics Middleware Framework (Open-RMF)**[13]: A free, open-source, and modular system that facilitates interoperability between multiple robot fleets and physical infrastructure such as doors, elevators and building management systems.

- **Robofleet**[14][36]: An open-source system based on ROS 1 that supports inter-robot communication, remote monitoring, and remote tasking for heterogeneous fleets of ROS-enabled service robots. It is specifically designed to handle network variability and to incorporate security control features.

- **openTCS**[15]: (open Transportation Control System) is a free control system software for coordinating fleets of automated guided vehicles (AGVs) and mobile robots. It can control any automatic vehicle,independent of their specific characteristics, with communication

[8]https://micro.ros.org/
[9]https://docs.ros.org/en/rolling/How-To-Guides/Migrating-from-ROS1.html
[10]https://industrial-training-master.readthedocs.io/en/melodic/_source/session7/ROS1-to-ROS2-porting.html
[11]https://github.com/NVIDIA-ISAAC/isaac_mission_dispatch
[12]https://github.com/ROOSTER-fleet-management
[13]https://www.open-rmf.org/
[14]https://github.com/ut-amrl/robofleet
[15]https://opentcs.org

capabilities. Also, It can manage vehicles of different types and performing different tasks at the same time.

Other frameworks provide the necessary infrastructure to build a fleet control system:

- **Transitive Robotics**[16]: An open-source framework for full-stack robotics, designed to simplify the development of capabilities that connect robots, cloud services, and web front-ends through a dynamically updating shared data model. It addresses common challenges in building full-stack robotic applications, such as unreliable networks, intermittently offline robots, and cross-device versioning.

- **Robot Web Tools**[17]: A collection of tools that enable communication with remote robots via web technologies, facilitating remote monitoring, and interaction.

Given that Open-RMF offers control over multiple fleets, enables structured interaction between them to achieve optimal outcomes, enable interaction with the building infrastructure, provides a comprehensive toolset for system development, is based on ROS 2, and is open source, it will be used as our fleet management system.

2.4.1 Open-RMF description

Open-RMF is a flexible and modular toolkit built on ROS 2 that facilitates interaction among heterogeneous robot fleets. It employs standardized protocols to enable communication with infrastructure, environments and automation systems, optimizing the utilization of critical shared resources such as elevators, doors, narrow corridors, and other robots. By coordinating access to these resources, Open-RMF introduces system-level intelligence that helps prevent conflicts and ensures proper task and resource allocation. Figure 2.13 illustrates the architecture and operational workflow of Open-RMF.

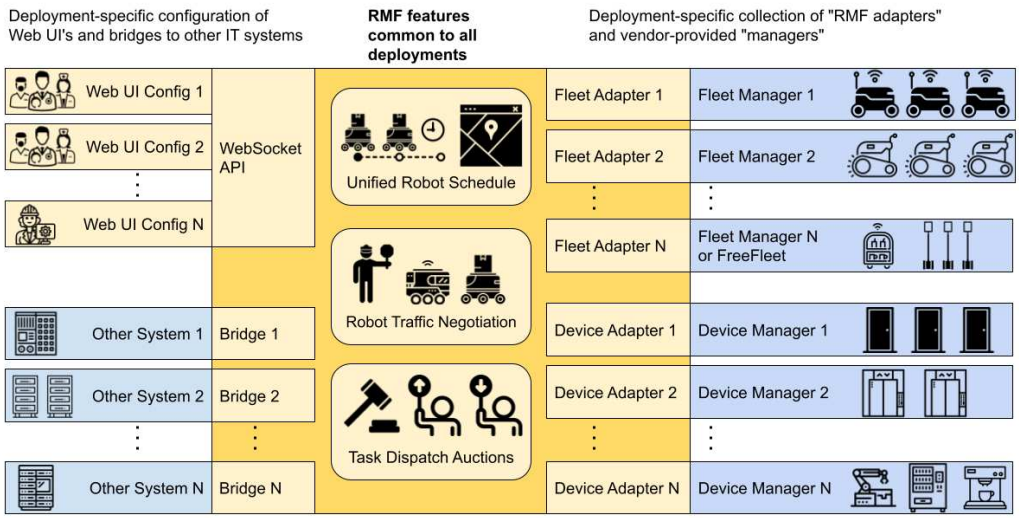


Figure 2.13: Open-RMF Structure

The fleets that can be controlled through Open-RMF can be categorized into three types:

[16]https://transitiverobotics.com/
[17]https://robotwebtools.github.io/

- *Full Control*: Fleets of this type provide full status updates and allow complete control over path planning and execution.
- *Traffic Light*: These fleets also provide full status updates, but only allow limited control, such as pausing and resuming operations. Open-RMF has control over certain behaviors, but not over the specific paths.
- *Read Only*: These fleets only provide status updates and cannot be controlled. Only one read-only fleet can exist in the system at any given time, as the presence of multiple read-only fleets would make it nearly impossible to avoid deadlocks or resource conflicts.

If a fleet does not belong to one of these categories, it falls under the classification of *No Interface* and is incompatible with Open-RMF, as this can lead to deadlocks in resource allocation.
The core of Open-RMF, known as `rmf_core`, is composed of several modular packages, each serving a distinct role within the system:

- `rmf_traffic`: Provides core scheduling and traffic management capabilities. It is based on a platform-agnostic **Traffic Schedule Database** and includes two main mechanisms for traffic deconfliction:

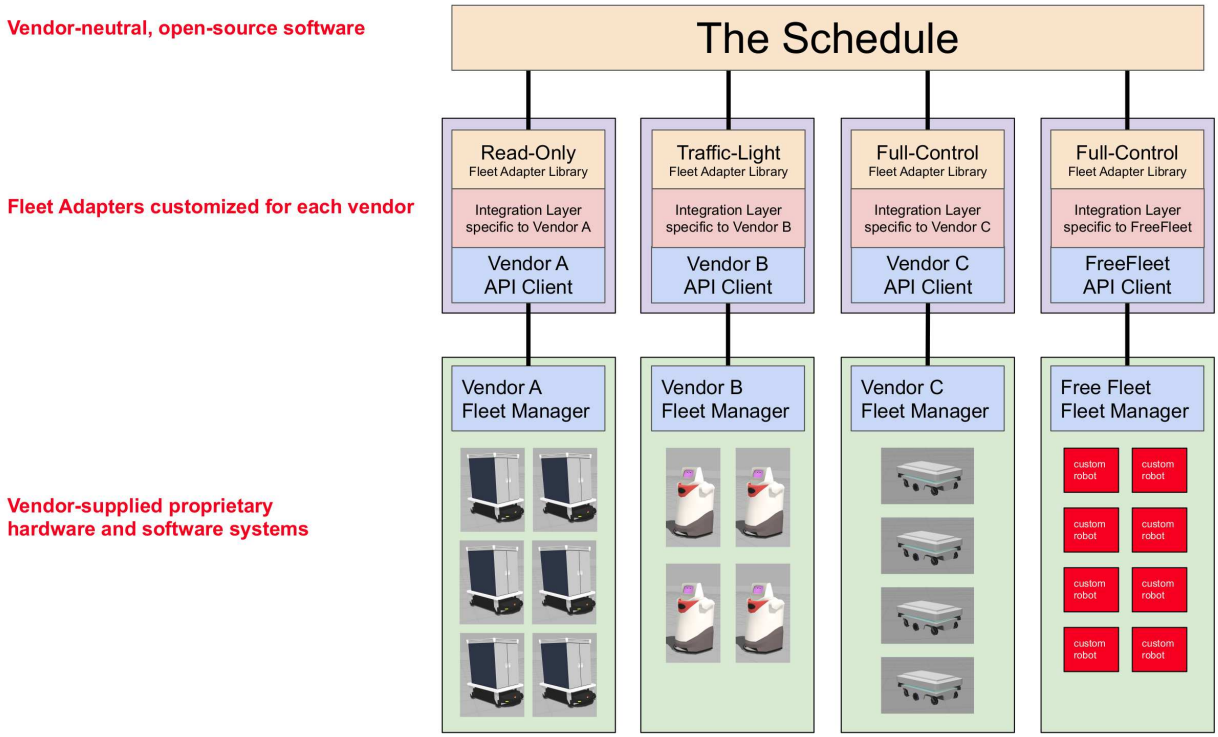


Figure 2.14: Open-RMF Schedule structure

  - *Prevention*: When changes occur in the database (e.g., route cancellations), fleet managers can proactively replan routes to avoid conflicts.
  - *Negotiation*: When a potential conflict is detected between two or more schedule participants, a conflict notice is issued to the relevant fleet managers. Each manager responds with a preferred path that accommodates other agents and an external arbitration module selects the optimal path.

Conflict avoidance for Automated Guided Vehicles (AGVs) is implemented using a time-dependent extension to A* search. This algorithm considers both spatial and temporal dimensions, allowing it to generate conflict-free paths by accounting for the future movements of other agents. Support for Autonomous Mobile Robots (AMRs) is under active development.

- `rmf_task`: Contains APIs and base classes for defining and managing tasks in RMF. The framework natively supports three types of task requests:
  - *Clean*: For robots capable of cleaning floor spaces.
  - *Delivery*: For robots tasked with transporting items between locations.
  - *Loop*: For robots required to repeatedly navigate between two or more points.
- `rmf_dispatcher_node`: Contains the logic responsible for task allocation and management across multi-fleet systems. When a user submits a new task request, the dispatcher intelligently assigns it to the most suitable robot by querying each fleet adapter capable of performing the task. These adapters respond with bids indicating which robot is best suited, and RMF uses this bidding mechanism to determine the optimal assignment, as illustrated in Figure 2.15.

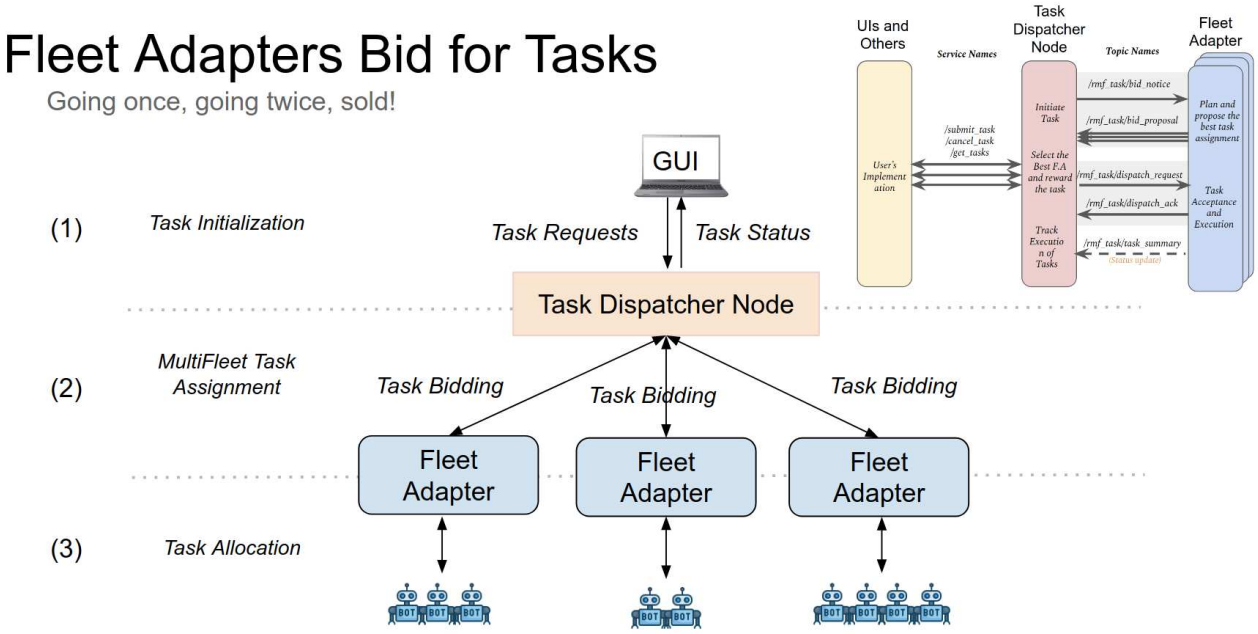


Figure 2.15: Open-RMF Bid Task

- `rmf_battery`: Provides battery consumption estimation models for different robot activities, aiding in energy-aware task planning.
- `rmf_obstacle`: Multiple packages that allow an interaction between the presence of obstacles or humans detected via external sensors and the lanes of the fleet navigation graph.

- `rmf_ros2`: Supplies ROS 2 adapters, nodes, and Python bindings to integrate `rmf_core` components into a ROS 2 distributed system, as shown in the following picture.

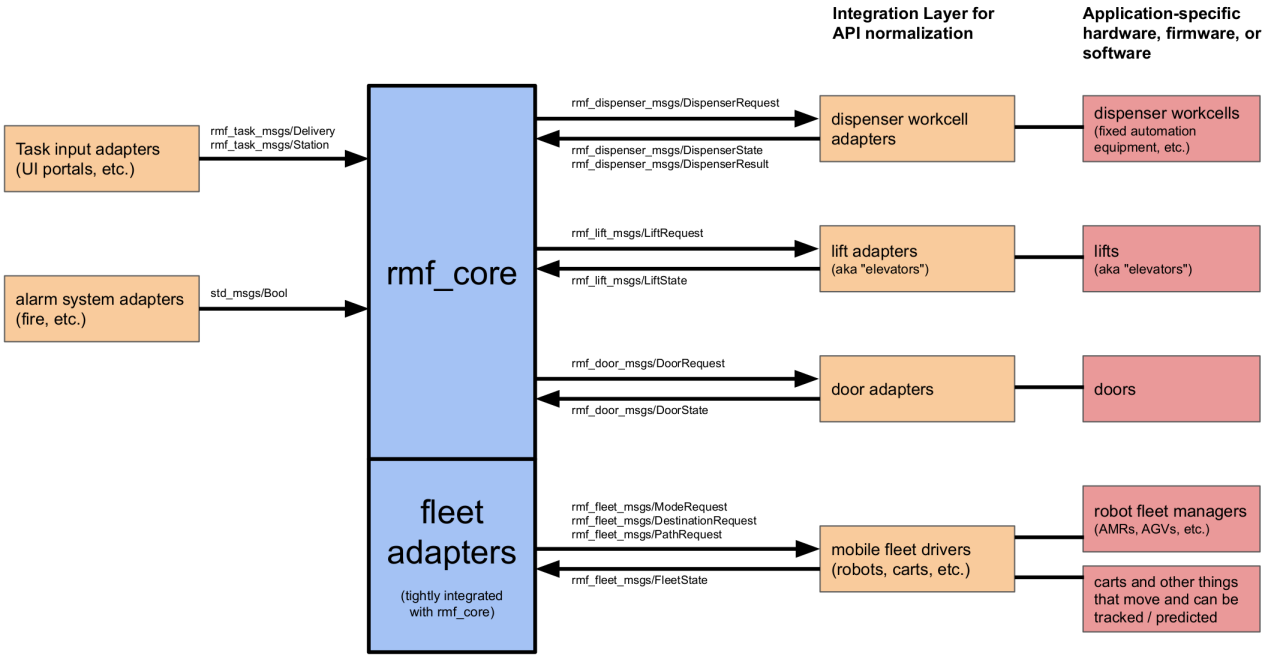


Figure 2.16: The communication needed to be handled by `rmf_ros2`

- `rmf_utils`: A collection of C++ utility functions and tools used by various RMF packages.

The following tools are available to support development with Open-RMF:

- `rmf_demos`: A collection of demonstration packages showcasing the capabilities of Open-RMF. These serve as a useful starting point for development and experimentation.

- `rmf_traffic_editor`: A graphical map editor used to define traffic patterns and interaction points (e.g., doors, lifts) that RMF uses for traffic management. It also supports the generation of Gazebo simulation worlds based on the designed maps.

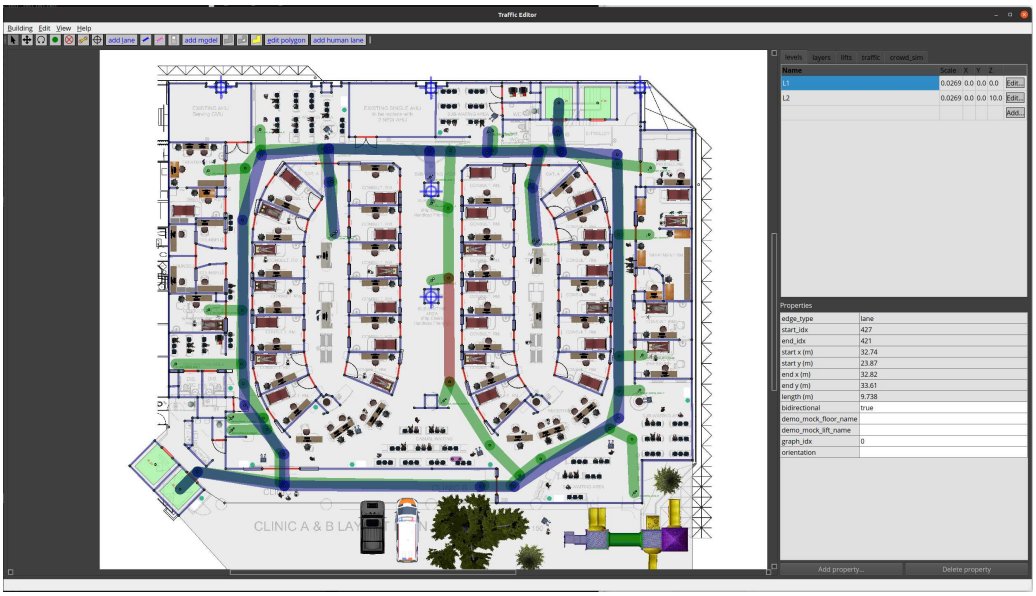


Figure 2.17: Traffic Editor

Another version of this software, `rmf_site`[18], is currently undergoing active deployment.

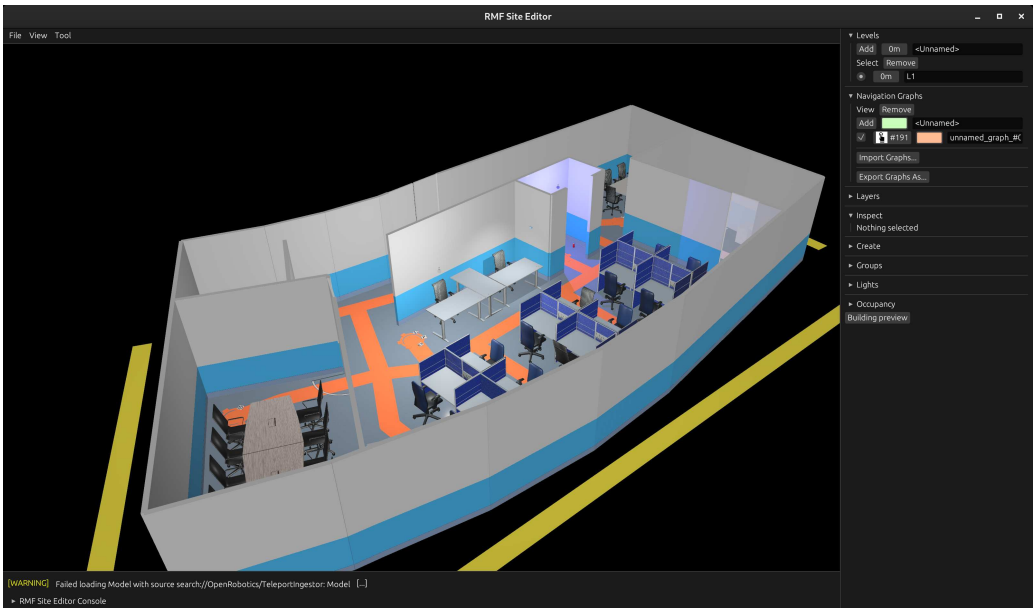


Figure 2.18: RMF Site Editor

[18]https://github.com/open-rmf/rmf_site

- `fleet_adapter_template`: An open-source fleet adapter template that enables the integration of standalone mobile robots into Open-RMF, allowing the creation of heterogeneous fleets.

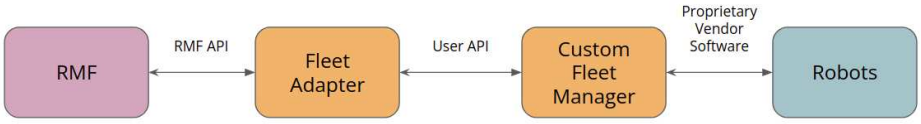


Figure 2.19: Fleet Adapter

From this package was developed `free_fleet` for controlling robots developed in ROS 1 and ROS 2.

- `rmf_visualization`: An RViz plugin that provides visualization and monitoring of the RMF schedule, enabling users to observe and interact with traffic and task execution in real time.

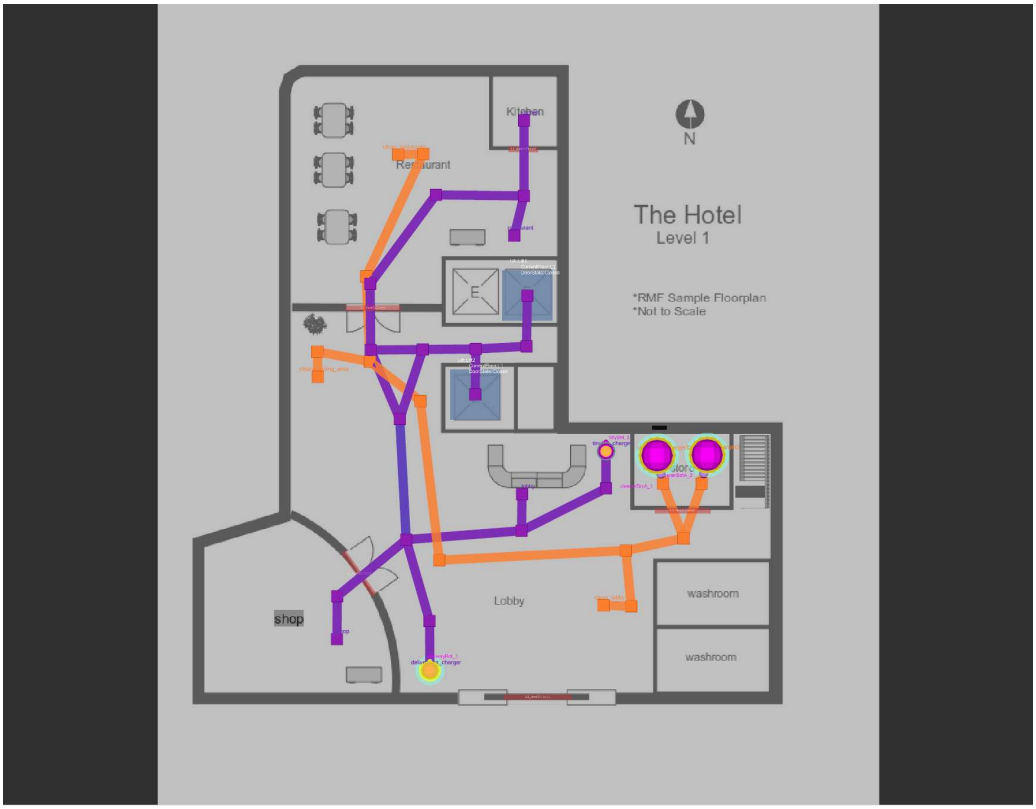


Figure 2.20: Visualization of RMF via RViz given by `rmf_visualization`

- `rmf-web` : A web-based application for visualization and control of the Robotic Middleware Framework (`rmf_core`). It provides a user-friendly interface for monitoring system state and interacting with fleet operations.

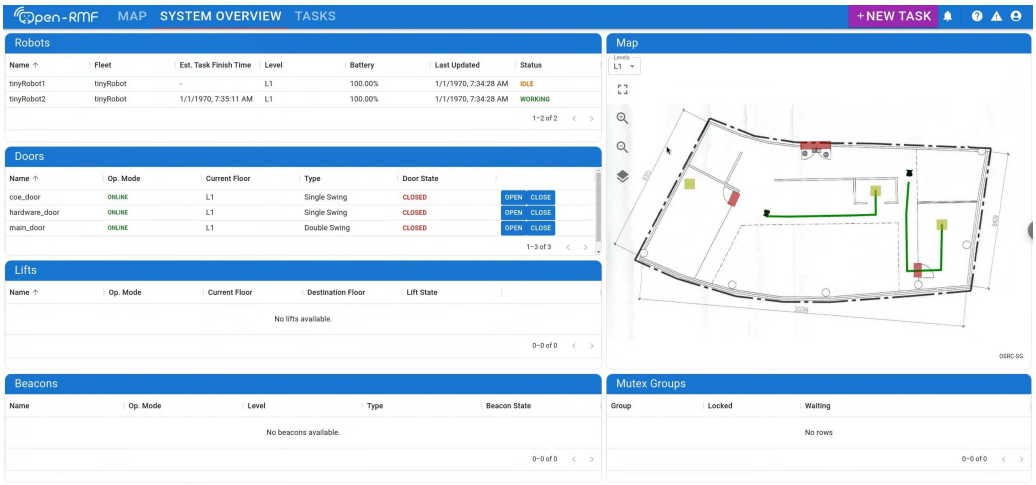


Figure 2.21: RMF Web

- `rmf_simulation`: A set of Gazebo plugins designed to simulate various aspects of building infrastructure—such as doors, lifts, and crowds—as well as robot behavior, including navigation and interactions with workcells.
- `door_adapter_template` and `lift_adapter_template`: Templates for writing a python based Open-RMF door and lift adapter.
- **Simulation assets**: Freely available 3D models and environment assets used to support simulation development and testing in Gazebo.

3. Methodology

In this part, we describe the entire procedure followed in the development of this project:

- The setup of our testing platform, based on the Rover Mini and a suite of onboard sensors.
- The configuration of the Nav2 navigation stack.
- The procedure used to port the global planner from ROS 1 to ROS 2, and its integration into the Nav2 framework.
- The changes made when porting the old autonomous mobile robot Jobot from ROS 1 to ROS 2.
- The tools and methods used to integrate the system into Open-RMF.

All source code developed for this project is publicly available in the following GitHub repository: `michbelle/tesi_mag`[1]. The repository also includes the necessary components for simulating the system.
The simulation environment is containerized using Docker, which provides a consistent and OS-independent base system. The Docker image is built on top of `osrf/ros:jazzy-desktop-full` and includes all the packages developed for this thesis, as well as the required Open-RMF tools and its demo simulation.

3.1 Description of research platform and hardware used

The testing platform is an unmanned ground vehicle (UGV) called Mini that is a four-wheel drive (4WD) skid-steer vehicle (see Figure 1.1a). It has a maximum speed of 12.87 km/h and a payload of 45.36 kg. Its operational time ranges from 60 to 90 minutes under active usage, and up to 6 hours in idle mode. The manufacturer provides control and simulation code on GitHub[2], compatible with the Gazebo simulator. However, this code is officially tested only for the ROS 2 Humble distribution, which has therefore been selected as the version of ROS used to control it.
The UGV features a USB interface that allows connection to a PC or single-board computer for control purposes. In this work, a mini PC is used to control the UGV and also to manage the entire navigation stack based on Nav2. The Mini PC used is a **Beelink SER3-E-16512EJ0W64PRO** with a Ryzen 7 3750H processor which is a quad-core 8-thread 2.3 GHz mobile processor that boosts to 4.0 GHz with a Radeon RX Vega 10 Graphics. The mini PC draws power directly from the UGV, which supplies a 19V output, a factor that reduces the overall operational time.

[1] https://github.com/michbelle/tesi_mag
[2] https://github.com/RoverRobotics/roverrobotics_ros2

Figure 3.1: Mini PC used

To improve odometry estimation, an additional sensor is integrated: the **IMU Witmotion (WIT) JY901B**. This device is a 9-Axis Combined IMU/Magnetometer/Altimeter (measure linear accelerations, angular velocities, Euler angles, magnetic field, barometry, altitude) that communicates using the TTL/UART protocol. Sensor data is acquired using the Witmotion Sensors UART Connection Library[42], using a USB-to-serial converter and are published in the ROS 2 environment through a dedicated node, `witmotion_ros`, available on GitHub at `ElettraSciComp/witmotion_IMU_ros`[3].

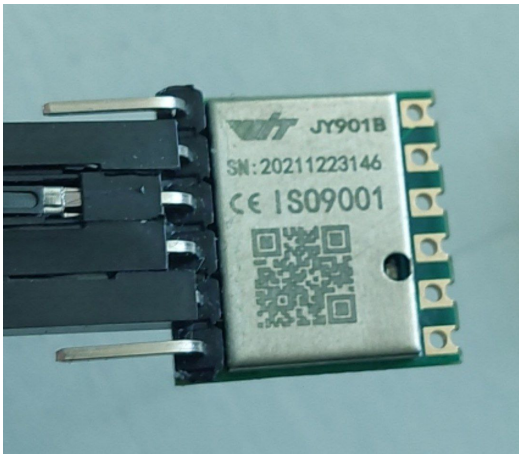


Figure 3.2: IMU used

A **2D laser scanner** is employed to provide environment perception and is used to compensate for odometry drift through the localization (see 2.2.1), which provides a transformation link between the `map` and `odom` TF frames. The 2D laser scanner provides a cost-effective solution for environmental perception, providing dense spatial information that enables obstacle detection and basic environment recognition. Compared to alternative technologies [2][25], it requires significantly less computational power, making it a practical choice for lightweight and resource-constrained robotic platforms. The Laser scan used is **RPLIDAR S1** a 360 degree 2D laser scanner (LIDAR) with a distance range of 40 m on white objects and 10 m on black ones. It features a sample rate of 9.2kHz and a scan rate ranging from 8 to 15 Hz, resulting in an angular resolution from 0.313° to 0.587°, with an accuracy of ±5cm and resolution of 3cm.

[3]https://github.com/ElettraSciComp/witmotion_IMU_ros/tree/ros2

The ROS 2 driver for the RPLIDAR S1 is available on Github at: `Slamtec/sllidar_ros2`[4]. All components are physically mounted on the rover using custom 3D-printed parts, as shown

Figure 3.3: LIDAR used

in the following image.

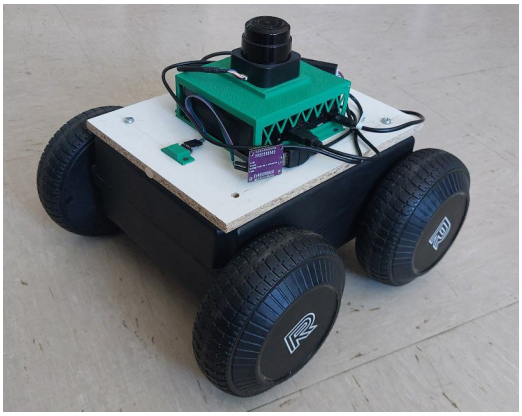


Figure 3.4: Robot used

To reflect the updated hardware configuration, the original URDF (Unified Robot Description Format) file provided by the manufacturer was modified. This ensures accurate sensor placement relative to the `base_link` frame, which is essential for proper operation of sensor-dependent ROS 2 nodes.
A 3D model of the robot, including all coordinate frames (TF), is shown in this picture:
With the system fully assembled, the following ROS 2 launch files, from the package `mini_launchpad`, are used to retrieve data from the sensors connected to the robot's onboard mini PC:

- `0.1r_sensorLaunch.launch.py`: Launches the nodes responsible for publishing data from the IMU and LIDAR sensors.
- `0.2r_mini.launch.py`: Publishes odometry data based on the built-in encoders of the robot and subscribes to the `/cmd_vel` to move the robot.

While, for simulation, the following launch files, from the package `mini_simulation`, can be used for starting the simulation:

[4]https://github.com/Slamtec/sllidar_ros2

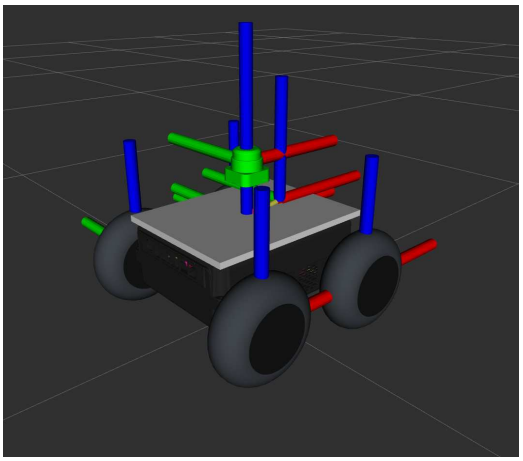


Figure 3.5: 3D model with TF

- `0s_ign_spawn_world_gazebo.launch.py`: This launches Gazebo with the world.

- `1s_ign_spawn_mini_gazebo.launch.py`: Spawn the model of the robot with all sensors in gazebo and launch the `ros_gz_bridge/parameter_bridge` node that is used to share simulated data between Gazebo and the ROS 2 environment.

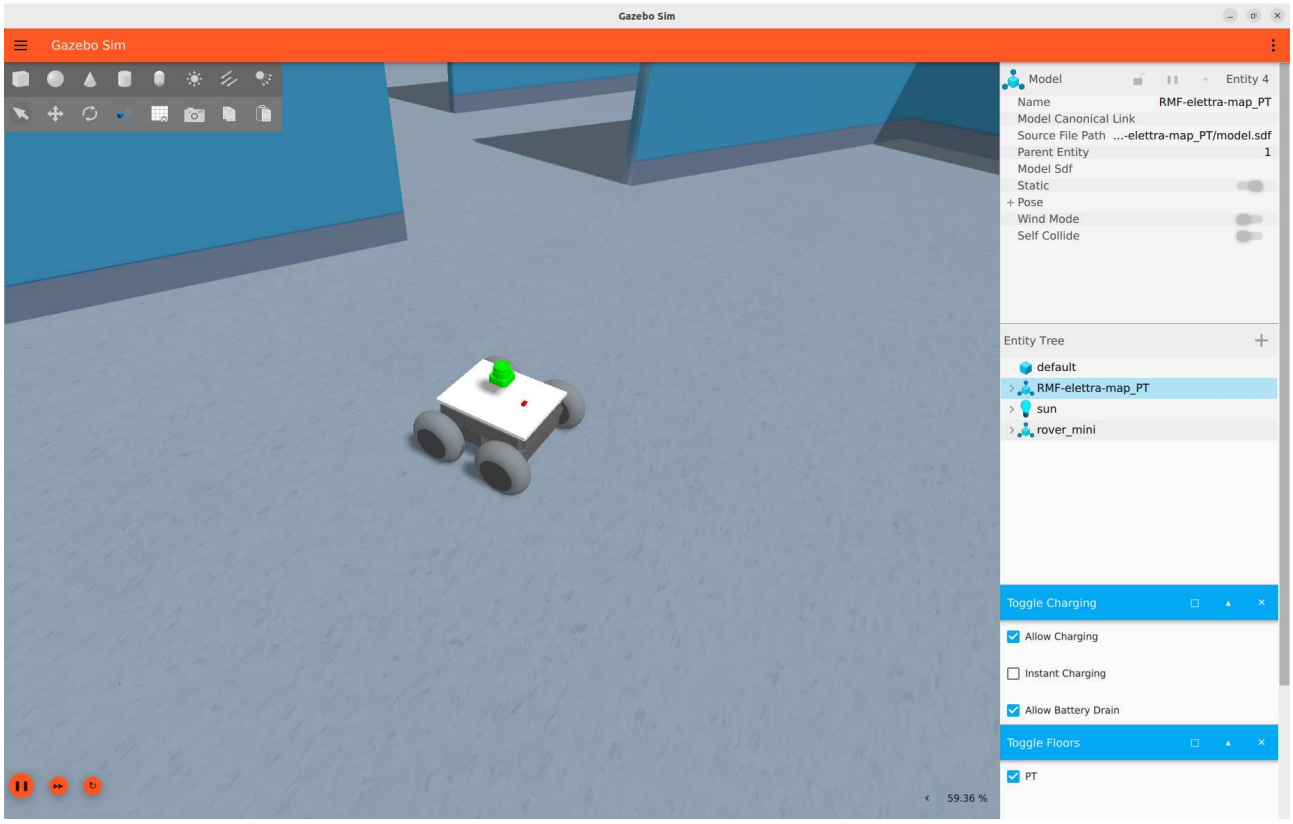


Figure 3.6: Model of the robot in Gazebo Ignition

3.2 Steps taken to port the navigation stack from ROS 1 to ROS 2

In this chapter, we outline the requirements for the operation of Nav2 and provide a comprehensive overview of the packages utilized by Nav2. Particular emphasis is placed on the process of porting the Global Planner module from ROS 1 to ROS 2, with a detailed discussion of the required adaptations and the challenges encountered during the transition.

3.2.1 Odometry Estimation `odom` → `base_link`

As previously discussed, the raw odometry data provided by the rover is not sufficiently accurate, particularly in rotational movements. This is a well-known issue in odometry estimation for skid-steering vehicles, which tend to produce significant drift and errors when turning using only the data from the encoders.
To address this problem, the `robot_localization` package[26] was employed, specifically its `ekf_node` Node. This package enables the fusion of multiple sensors (even multiple instances of the same sensor type), and supports output in various ROS message formats, such as `nav_msgs/Odometry`. It also allows preprocessing of sensor messages prior to inserting them into the estimation process. The filter performs continuous state estimation and is robust to intermittent sensor data loss, maintaining a coherent position estimate over time.
To launch the EKF node, the following launch file from the package `mini_launchpad` was written:

`1<r/s>_robot_localizationEKF.launch.py`

Configuration Odometry Estimation in `ekf_node`

As previously described and detailed in the node's documentation[5], we added the information about the sensors used to the configuration file. The principal information to include is the type of sensor and on which topic to subscribe, in our case `odom0:odometry/wheels` and `imu0:imu/data`. For each sensor, we need to choose which components of the state measurement data we want to fuse using the following array by setting the corresponding values to `true` for those we wish to include and `false` for those we do not:
`sensorX_config`:$[X, Y, Z, roll, pitch, yaw, \dot{X}, \dot{Y}, \dot{Z}, \dot{roll}, \dot{pitch}, \dot{yaw}, \ddot{X}, \ddot{Y}, \ddot{Z}]$
Additionally, the process noise covariance matrix Q can be tuned. This matrix represents the noise we add to the total error after each prediction step. Generally, the larger the value of Q relative to the variance of a given variable in an input message, the faster the filter will converge to the value indicated by the measurement.
More information on the testing and the parameters analysis is described in Section 4.2, Testing and Evaluation.

3.2.2 Localization `map` → `odom`

In this section, we describe the process of acquiring a map and establishing the transformation between the `map` frame and the odometry (`odom`) frame using a localization method. This is necessary because, in long-term operation, the robot's position in the world, when determined solely using odometry data, tends to accumulate errors and is thus not sufficiently accurate. Therefore, a localization technique is required to compensate for these errors.

[5] https://docs.ros.org/en/noetic/api/robot_localization/html/index.html

Mapping

Although a map is already available, we will describe the process of constructing a map using the Simultaneous Localization And Mapping (**SLAM**) technique[5]. Below, we provide instructions for running the following SLAM packages: `slam_toolbox`[21] (GitHub: SteveMacenski/slam_toolbox[6]) and `cartographer_ros`[14]
(GitHub: ros2/cartographer_ros[7]).
These packages can be launched from the `mini_launchpad` using the following launch files:

```
X<r/s>_mappingW<slam/cartographer>.launch.py
```

To save the generated map, use the following command, which works for both `slam_toolbox` and `cartographer_ros`. This will save the map as two files: a `.pgm` image and a `.yaml` metadata file:

```
ros2 run nav2_map_server map_saver_cli -f map
```

When using Cartographer, it is also possible to save the map as a `.pdstream` format with the following command:

```
ros2 service call /write_state cartographer_ros_msgs/srv/WriteState
  "{filename:  "map.pdstream", include_unfinished_submaps :false}"
```

Localization Only

Since we already have a map, we only need to focus on localization. The map in use was created by the Jobot robot using Cartographer while navigating through Building T of Elettra, which serves as an office building. The map is:

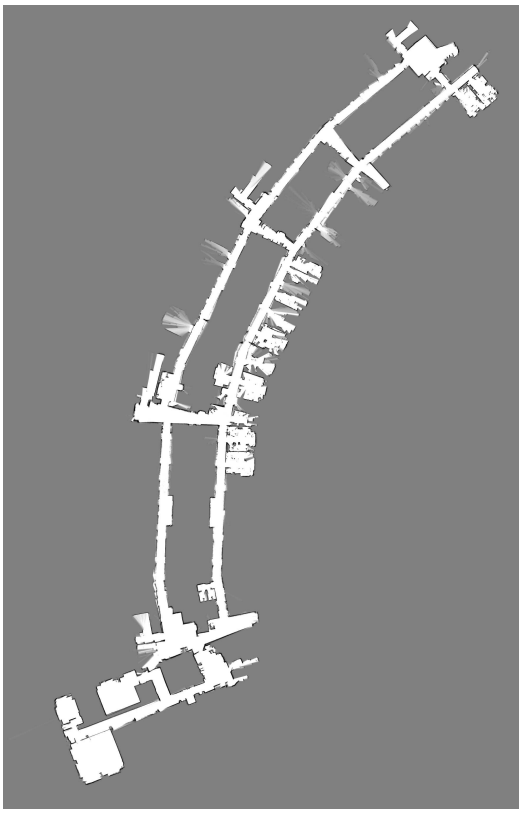


Figure 3.7: Map acquired with Cartographer

[6]https://github.com/SteveMacenski/slam_toolbox/tree/ros2
[7]https://github.com/cartographer-project/cartographer_ros

We initially attempted to use Cartographer solely for localization; however, due to the project's limited maintenance, we encountered difficulties in using it effectively. As a solution, we opted for the method provided by the Nav2 package, specifically the `amcl` node from the `nav2_amcl` package that contains a pure and fully-parameterized implementation of the grid particle filter localizer[12][39].
One limitation of this approach is the need to convert the map acquired with Cartographer from the `.pbstream` format into a `.pgm` image and `.yaml` metadata pair, which are required by the `amcl` node. This conversion can be performed using the following command:

```
ros2 run cartographer_ros cartographer_pbstream_to_ros_map \
        -pbstream_filename map.pbstream \
        -map_filename map.pgm \
        -resolution 0.05
```

Since certain walls in the Cartographer generated map were not solid black (due to the uncertainty of their position during mapping), the conversion process introduced errors that led to these areas to being misinterpreted as free space. As a result, the planner generated paths that passed through them. To address this issue, the walls were manually corrected using GIMP, an image editing tool, by modifying the `.pgm` map file.
Additionally, our planner is designed with the assumption that unknown spaces are not usable for planning. Therefore, we set the parameter `track_unknown_space` in the global costmap node to `true` (`false` by default) to achieve optimal results from our planner. The corrected map is shown below:

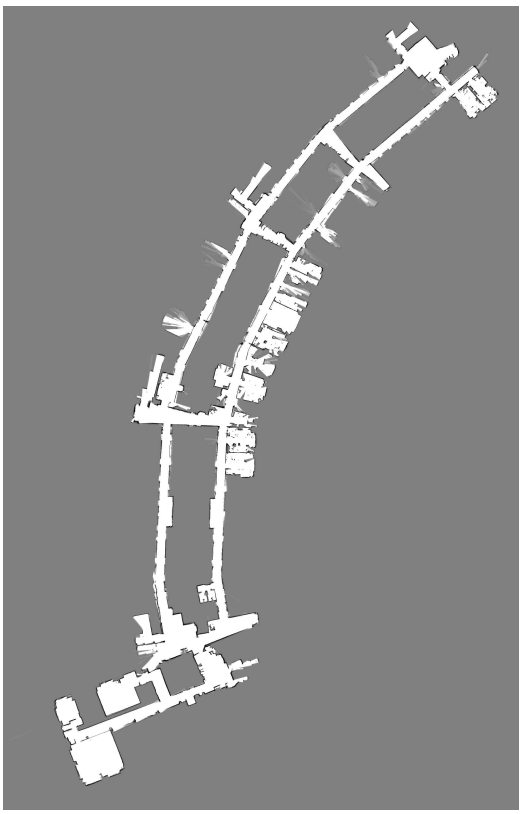


Figure 3.8: Map edited with Gimp and used

With the map properly adjusted, all necessary tools for localization are now available. We created the following launch file in the package `mini_launchpad` to initiate localization:

`3<r/s>_L<amcl/Cartographer>.launch.py`

At this stage, all requirements are met to utilize the Nav2 navigation stack.

3.2.3 Structure Navigation Stack Nav2

In this section, we will discuss the components used in the Nav2 navigation stack and describe the process of porting the Global Planner from ROS 1 to ROS 2.

Navigation Launch file

The launch file contains all the necessary processes required for the Navigation stack Nav2 to operate. It also implements the composition structure (2.1.1) and the lifecycle manager (2.1.1). Below is a list of the core components used, along with a brief description of the plugins associated with each. Additional information can be found in this documentation[8] , and a migration guide is available in this documentation[9] :

- `nav2_controller::ControllerServer` that uses the following plugins:
  - As `controller_plugins` use `nav2_mppi_controller::MPPIController`: a Model Predictive Path Integral (MPPI)[44] algorithm to track a path with adaptive collision avoidance or use the `dwb_core::DWBLocalPlanner` : that use the Dynamic Window Approach [10]
  - As `progress_checker_plugins` use `nav2_controller::SimpleProgressChecker` that checks whether the robot has made positional progress.
  - As `goal_checker_plugins` use `nav2_controller::SimpleGoalChecker` that checks whether the robot has reached the goal pose.
- `nav2_smoother::SmootherServer` that uses the following plugins:
  - As `smoother_plugins` use `nav2_smoother::SimpleSmoother` that takes an input path and applies a lightweight smoothing algorithm. It weights the initial path points and the smoothed path points to create a balanced result where the path retains its high level characteristics but reduces oscillations or jagged features.
- `nav2_planner::PlannerServer` that uses the following plugins:
  - As `planner_plugins` use `dstar_global_planner::DStarGlobalPlanner` that is our global planner plugin and is an implementation of the D* informed incremental search algorithm with external optimizers[41].
- `nav2_behaviors::BehaviorServer` that uses the following plugins:
  - As `behavior_plugins` use: `nav2_behaviors/Spin` performs an in-place rotation by a specified angle, `nav2_behaviors/BackUp` moves the robot backward by a set distance, `nav2_behaviors/Wait` is used to pause navigation for a specified duration, allowing time for temporary occlusions to clear before resuming path planning.

[8] https://docs.nav2.org/setup_guides/algorithm/select_algorithm.html
[9] https://docs.nav2.org/migration/index.html

- `nav2_bt_navigator::BtNavigator` that uses the following plugins:
  - As `navigators` use `nav2_bt_navigator::NavigateToPoseNavigator` and `nav2_bt_navigator::NavigateThroughPosesNavigator` that utilize the default behavior trees (BT) for navigation.
- `nav2_lifecycle_manager::LifecycleManager` that handles the lifecycle transition states for the stack in a deterministic way.

The simulation configuration file contains minor differences due to the use of different versions of ROS 2: Jazzy in the simulated environment and Humble on the physical robot. In the next section, we will describe the key steps taken to port the global planner `DStarGlobalPlanner` from ROS 1 to ROS 2.

3.2.4 Porting ROS 1 DStarGlobalPlanner to ROS 2

As previously mentioned, the `DStarGlobalPlanner`[10] is an implementation of the D* informed incremental search algorithm[1] with external optimizers[41] for the ROS 1 `move_base` package[11].
Since the code is written using ROS 1, which is not compatible with ROS 2, it will need to be ported to the newer version. This process was divided into two main phases. In the first phase, we migrated the base structure of the package from ROS 1 to ROS 2, following these ROS 2 porting guidelines [12] [13]. In the second phase, we adapted the code to comply with the structure required by Nav2 planner plugins, based on the Nav2 documentation titled: "Writing a New Planner Plugin"[14]. The following subsections describe these two phases in detail. The first focuses on the structural migration to ROS 2 and the integration into the Nav2 framework. The second details the modifications needed to adapt the planner logic and interfaces to match ROS 2 conventions and the plugin architecture.

3.2.5 Porting code from ROS 1 to ROS 2

In the `package.xml` file, which describes the package as an organizational unit for your ROS 2 code, the following changes were made:

- `buildtool_depend` : `catking` → `ament_cmake`: `ament_cmake` is the new build system for CMake based packages. Compared to `catkin`, it introduces many utilities for managing packages. More information is available in the documentation[15].
- `roscpp` → `rclcpp`: `rclcpp` is the new ROS client library for C++, built on `rcl`, which provides the foundation for language-specific client libraries. `rcl` communicates with `rmw`(ROS middleware interface), which handles the communication with `DDS` (Data Distribution Service).

[10] https://github.com/ElettraSciComp/DStar-Trajectory-Planner/tree/ros2_port
[11] https://wiki.ros.org/move_base
[12] https://docs.ros.org/en/rolling/How-To-Guides/Migrating-from-ROS1.html
[13] https://industrial-training-master.readthedocs.io/en/melodic/_source/session7/ROS1-to-ROS2-porting.html
[14] https://docs.nav2.org/plugin_tutorials/docs/writing_new_nav2planner_plugin.html
[15] https://docs.ros.org/en/rolling/Concepts/Advanced/About-Build-System.html

- Updated package naming to follow the ROS 2 conventions.
- To inform `colcon` that the package uses `ament_cmake`, the build type must be explicitly specified in the `build_type`.

Subsequently, the `CMakeLists.txt` file was updated with the following major changes:

- Package names were updated to reflect ROS 2 naming conventions.
- C++ standard was set to 17.
- `find_package()` is required to be called for each package.
- `ament_export_dependencies()` is used to declare runtime dependencies.
- The package must be declared in ROS with `ament_package()`.

Other minor changes are:

- Log calls were updated from `ROS_` to `RCLCPP_`
- Message type structures changed (e.g., `geometry_msgs::PoseStamped` → `geometry_msgs::msg::PoseStamped`).
- The structure of the C++ program now requires the use of smart pointers.

Additional changes specific to the ROS 2 Navigation stack Nav2 are discussed in the following section.

3.2.6 Porting code from ROS 1 Navigation stack to Nav2

The following changes were made to port the planner to the Nav2 framework:

- Since the planner is a **global planner**, it now inherits from the new abstract interface `nav2_core::GlobalPlanner`. As a result, we updated the `global_planner_plugin.xml` file to change the `base_class_type` accordingly.
- As a plugin based on `nav2_core::GlobalPlanner`, the planner must implement five pure virtual methods required for proper operation: `configure()`, `activate()`, `deactivate()` and `cleanup()`, `createPlan()`. Below is a description of each method:
  - `configure()`: It is called when the node enters `on_configure` state. This method is responsible for declaring ROS parameters and initializing the planner's member variables. It takes the following input parameters:
    * `rclcpp_lifecycle::LifecycleNode::WeakPtr` A weak pointer to the base lifecycle node.
    * The planner's name as a `std::string`.
    * A shared pointer to the `tf` buffer `std::shared_ptr<tf2_ros::Buffer>`.
    * A shared pointer to the costmap `std::shared_ptr<nav2_costmap_2d::Costmap2DROS>`.

    * For jazzy also: `std::function<bool()> cancel_checker` that allows for the cancellation of the current planning task.

  This replaces the `initialize` method and the class initialization used in ROS 1.

- `activate()`: Called when the node enters `on_activate` state, it is used to implement operations which are necessary before planner goes to an active state. In our case, we allocate memory and fill our internal costmap `state_map`, which is used by the D* (`dstar`) algorithm.
- `deactivate()`: Called when the node enters `on_deactivate` state. This method is used for implementing operations which are necessary before planner goes to an inactive state. In our case, this method is not used and is left empty.
- `cleanup()`: Called when the node enters `on_cleanup` state. This method is used for cleaning up resources which are created for the planner. In our case we destroy the allocated memory used by `state_map` and `dstar`.
- `createPlan()`: Invoked when the planner server requests a global plan between a start and goal pose. It requires the start and goal position as `geometry_msgs::msg::PoseStamped` and returns a path `nav_msgs::msg::Path`. It replaces the `makePlan` used in ROS 1.

- At the end of the C++ code, we added the following lines to export the planner as a plugin:

```
#include "pluginlib/class_list_macros.hpp"
PLUGINLIB_EXPORT_CLASS(<type of the plugin class>, <type of the base class>);
```

- Because it is a plugin, it is necessary to provide a description using a plugin declaration file. This is done by creating a file named `global_planner_plugin.xml` in the same directory as the `package.xml`. The `global_planner_plugin.xml` file specifies the plugin's class type and the base class it inherits from, following the format required by the pluginlib system. Then we edit the CMake file as described in the documentation[16].

Building the project

The system was built using the build tool `colcon`[17]. It is a command line tool to improve the workflow of building, testing and using multiple software packages. It automates the process, handles the ordering and sets up the environment to use the packages.
After successful compilation, the global planner is ready for use within the Nav2 framework.

3.3 Jobot porting procedure to ROS 2

Elettra also has a Jobot, a 2WD rover still based on ROS 1. It was decided to port it to ROS 2 in order to effectively evaluate the use of Open-RMF.

[16] https://docs.ros.org/en/rolling/Tutorials/Beginner-Client-Libraries/Pluginlib.html
[17] https://github.com/colcon

3.3.1 Jobot Description

Jobot is a 6-wheeled AMR with a 2WD configuration. It is equipped with a mini PC that collects data from an RPLIDAR A2 laser sensor and acceleration data from a serial bus connected to an Arduino microprocessor, which is linked to an IMU. An image of the robot is available in 1.1b. The steps taken to port the code in ROS 2 are described in the following section. When porting the system, we used the same design for navigation and control that was used when it was built the first time[43].

3.3.2 What changes were made in the code

The code uses the serial ROS 1 library to communicate with the Arduino and retrieve the data from the encoders and the IMU. This library is no longer available in ROS 2, so to avoid rewriting the structure of the Jobot code, we utilized the same library ported to ROS 2, which is available on GitHub: iwelch82/serial-ros2 . The retrieved data is used to publish the odometry pose estimation of the robot, while a secondary node handles the IMU data.
The node that publishes the IMU data was updated to ROS 2 by simply using the base node logic, whereas the main node was rewritten using the lifecycle structure. This logic is employed to configure the node during the on_configure method, and if it fails or a request for deactivation or cleanup is made, it stops and cleans everything. This structure allows to remotely controlling the passage through the states, for now and in our case it will be connected to an automatic Node manager like the one used in Nav2, the `lifecycle_manager` in `nav2_lifecycle_manager`. This node checks all the states of the nodes that are connected to it and sequentially configures and activates them. If a crash occurs or if we stop a specific node's execution, the lifecycle communicates the change of state and activates a procedure to stop the connected nodes. This node is only called in the launch file.
In addiction, we updated the custom messages to ROS 2 that the Jobot uses internally.
Once everything is configured, the node starts to publish the robot's state odometry but does not publish static transforms between the sensors. These transforms are manually published using the `static_transform_publisher` provided by `tf2_ros`.

To minimize changes, the software is built and launched inside a Docker container that shares the device of the control PC of the Jobot.

This, along with the same software used for navigation, enables this platform to function as an autonomous mobile robot (AMR).

3.3.3 Create a model for the simulation

For simulation purposes, we need to create a model of the Jobot in the URDF format, which will be used by Gazebo to build a 3D representation and generate data from the sensors equipped on it.
Since Jobot has a 2WD configuration, we will add two cylinders as wheels, which will be controlled by the Gazebo plugin `ignition::gazebo::systems::DiffDrive`. This plugin allows the robot to receive velocity commands and move accordingly.
Next, we add the sensors gazebo types `IMU` and `gpu_lidar` to simulate the IMU and the lidar on the robot. This procedure is similar to the one used for the Mini Rover from the Rover Robotics. At the end we obtain the following model:

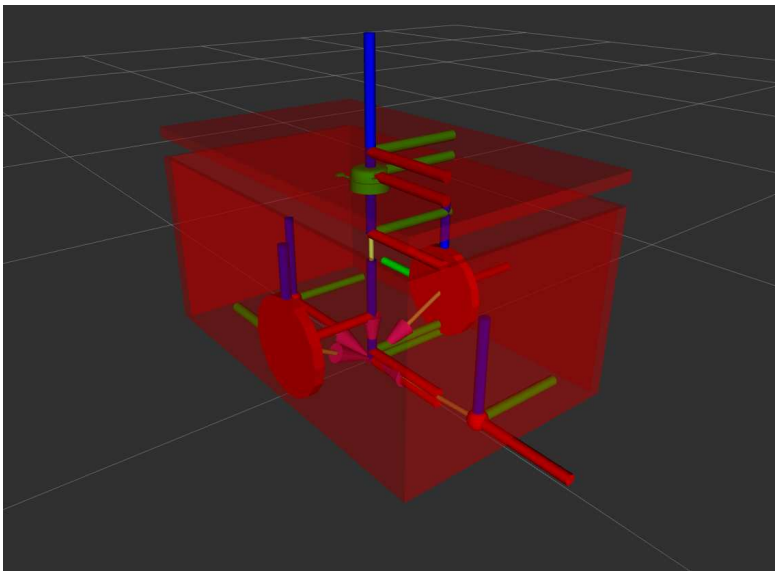


Figure 3.9: Jobot robot URDF model

To launch the sensors and the robot, a naming structure similar to that used for the Rover Mini is implemented for the Jobot. The following packages have been created for the Jobot:

- `jobot_driver_ros2`: This package contains the code for controlling the Jobot.
- `jobot_launchpad`: This package is responsible for launching Nav2.
- `jobot_msgs`: This package includes the custom messages used by Jobot.
- `jobot_simulation`: This package contains all the code necessary to simulate the Jobot in Gazebo.

3.4 Integration of the navigation stack with Open-RMF

To integrate a fleet in Open-RMF, we first need information about the building's infrastructure, such as automatic doors or lifts that can be controlled and the **route maps** that robots can use. **Route maps** are essential for predicting the paths robots will take, enabling Open-RMF to perform effective **traffic management** and avoid conflicts. All this information has to be stored in a file that can be generated using the **traffic editor tool**[18] [19], a tool provided by Open-RMF. The following section describes how the Traffic Editor works and how we used it in our implementation.

3.4.1 Traffic Management - Route Maps Generation

To begin describing the environment for Open-RMF, we created a new project in the Traffic Editor by generating a `.building.yaml` file. We then imported both the map produced by Cartographer and the floor plan of the facility. After importing the map,we defined the positions of key elements and planned the traffic flow for the floor, following best practices as outlined in the Open-RMF documentation[20]. The following components of the Traffic Editor were used to encode the necessary environment data:

- **vertex**: They are waypoints or a part of the description of walls, doors or else. Each vertex can have several properties:
  - `is_holding_point` : Indicates that the robot is allowed to wait at this waypoint for an indefinite period of time.
  - `is_parking_spot`: Designates a parking location for a robot.
  - `is_passthrough_point`: A waypoint where the robot should not stop.
  - `is_charger`: Identifies a charging station.
  - `is_cleaning_zone`: Used to indicate that this waypoint belongs to a cleaning zone for the `Clean` task.
  - `dock_name`: Used for triggering docking behavior via RMF.
  - `pickup_dispenser`: Specifies a dispenser workcell for `Delivery` tasks.
  - `dropoff_ingestor`: Specifies an ingestor workcell for `Delivery` tasks.
  - `spawn_robot_type`: Used in simulation to spawn a robot model.
  - `spawn_robot_name`: Used in simulation to identify a spawned robot.
- `measurement`: Connects two vertex and is used to set the scale of the drawing. In our case, we used the width of a door.
- `door`: Connects two vertex and is used for interacting with the real world and simulate doors. If a robot encounters a door on its route, RMF sends a command to open it. When generating the simulation world model, a controllable door model is automatically added.

[18] https://github.com/open-rmf/rmf_traffic_editor
[19] https://osrf.github.io/ros2multirobotbook/intro.html
[20] https://osrf.github.io/ros2multirobotbook/integration_nav-maps-strategies.html

- `traffic lane`: Connects two waypoints and define the paths that fleets will follow. Lanes can be grouped, assigned to specific fleets, and configured as unidirectional or bidirectional. An optional "orientation" setting can be specified to require robots to move forward or backward along the lane. During planning, Open-RMF assumes that all traffic lanes are straight lines.
- `fiducials`: Used for multi-floor alignment and as anchors for the ROS 2 map. They provide spatial reference points between floors.
- `lift`: Is used for interacting with the real world and simulate elevators. If a robot goes to a lift, RMF sends a command to call it at the floor the robot is and, when the robot reach the waypoint inside it, then RMF send the lift to the designed floor. When generating the simulation world model, a controllable lift model is automatically added.
- `walls`: Connect two waypoints to define the location of a wall in the environment. When generating the simulation world, a wall is created between these points.
- `floor`: Add the ground plane for robots in the simulation world. This component can be modified to include holes (e.g., for elevator shafts).
- `environment assets`: Adds Gazebo models of various furniture and objects to the simulation, helping to create a realistic environment for testing and validation.

In the process of designing route maps, several strategic choices were made:

- Only one robot is allowed in a hallway at a time.
- Holding points (leaf nodes) were added along corridors or near their entrances and exits to allow one robot to yield to another already in the hallway. For certain task-specific waypoints, it is recommended to assign a very low speed limit to discourage RMF from using them as generic holding points.
- Mutex groups were defined on lanes and waypoints to ensure exclusive access, avoiding deadlocks by allowing only one robot to occupy a mutex-assigned group at any time.

At the end we obtained the following configuration map:

The route map `.yaml` files for each graph can be generated using the following command:

```
ros2 run rmf_building_map_tools building_map_generator nav \
     ${building_map_path} ${output_nav_graphs_dir}
```

For the simulation, the world can be generated using the following command:

```
ros2 run rmf_building_map_tools building_map_generator gazebo \
     ${building_map_path} ${output_world_path} ${output_model_dir}
```

And the following command can be used to download the models required for the simulation world:

```
ros2 run rmf_building_map_tools building_map_model_downloader \
     ${building_map_path} -f -e ~/.gazebo/models
```

The simulation environment generated from the configuration resulted in the following world:

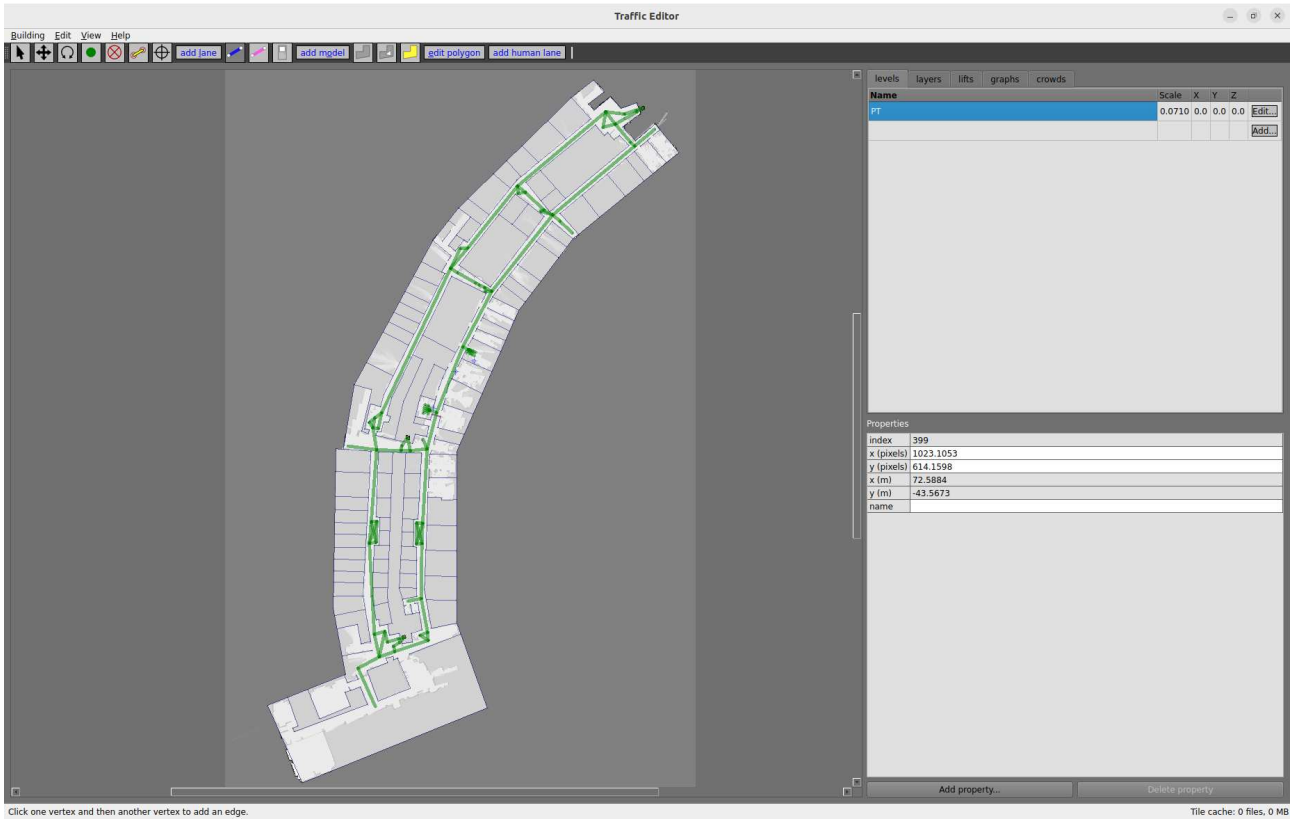


Figure 3.10: Traffic editor with Elettra's route map

We can now launch the core components of RMF.

3.4.2 RMF core

This is the launch file inside the `rmf_server_elettra` used to start the core components of RMF:
`0_rmf_core.launch.xml`

This launch file starts the following nodes, which form the core structure of RMF:

- `rmf_traffic_schedule` : Manages traffic scheduling by controlling path structures, publishing itinerary topics and handling conflict resolution.
- `rmf_traffic_blockade` : Acts as the Traffic Blockade Moderator, managing checkpoint information.
- `building_map_server` : Publishes map data, including points of interest such as parking spots.
- `visualization.launch.xml`: Optional nodes that launch visualization tools for monitoring and controlling RMF via rviz2.
- `door_supervisor`: Publishes control requests for doors using ROS communication structures.

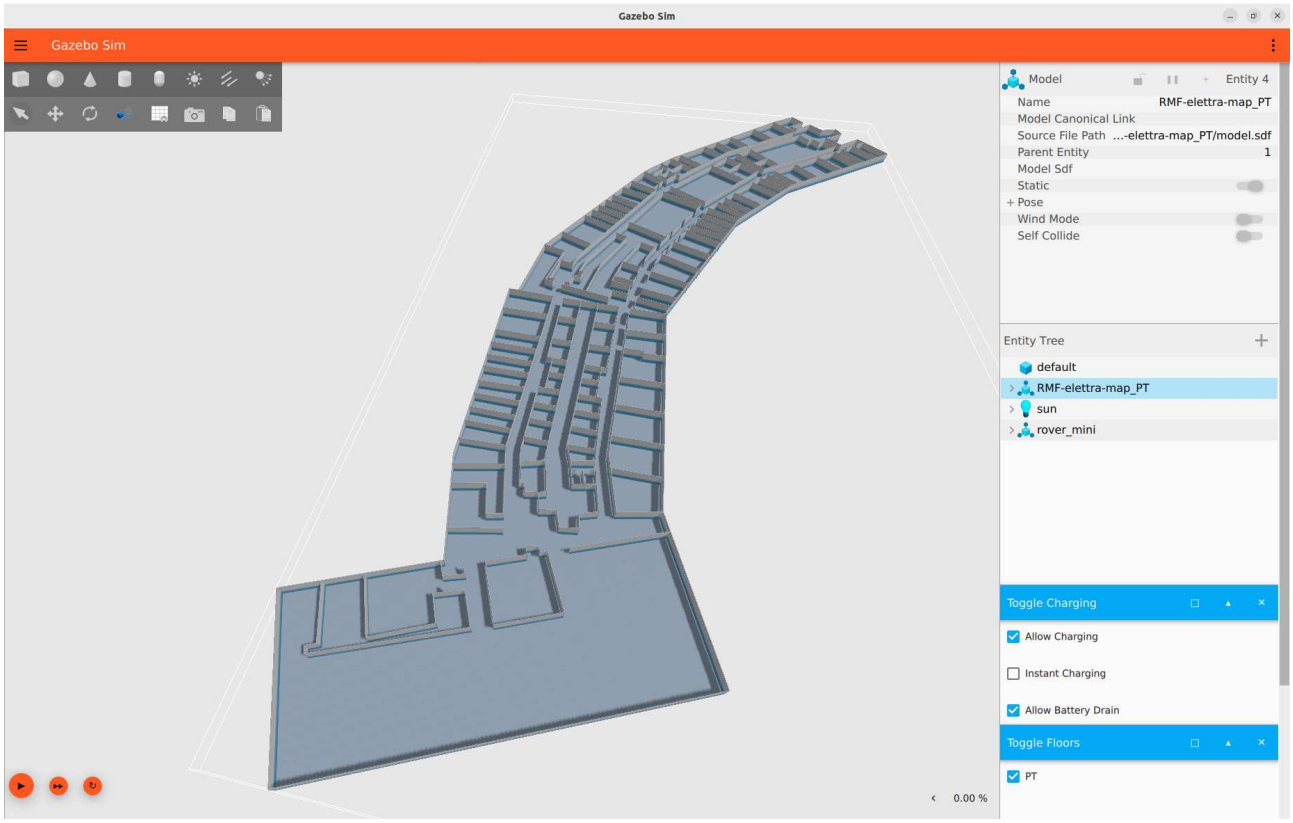


Figure 3.11: World generated using traffic editor

- `lift_supervisor`: Publishes control requests for lifts using ROS communication structures.

- `rmf_task_dispatcher`: Allows to dispatch submitted task to the best fleet/robot within RMF.

- `queue_manager`: Manages reservations for parking spots and helps prevent conflicts.

Once the core system is launched, the next step is to connect our fleets using **fleet adapters**. Open-RMF provides the Free Fleet adapter, which is compatible with ROS 2 and Nav2. The following section will detail how this adapter works and how we integrated it into our system.

3.4.3 Free Fleet

Free fleet[21] is a python implementation of the `full_control` Open-RMF Fleet Adapter[22], which serves as a general template for communicating with a fleet manager. Its primary goal is to control robots running under both ROS 1 and ROS 2. It uses `zenoh` as a communication layer between each robot and the fleet adapter. For now it supports the `cyclonedds` RMW, other RMW implementations will be set up in the future. Below is an overview of how communication is managed using the Free Fleet adapter:

[21] https://github.com/open-rmf/free_fleet
[22] https://github.com/open-rmf/fleet_adapter_template

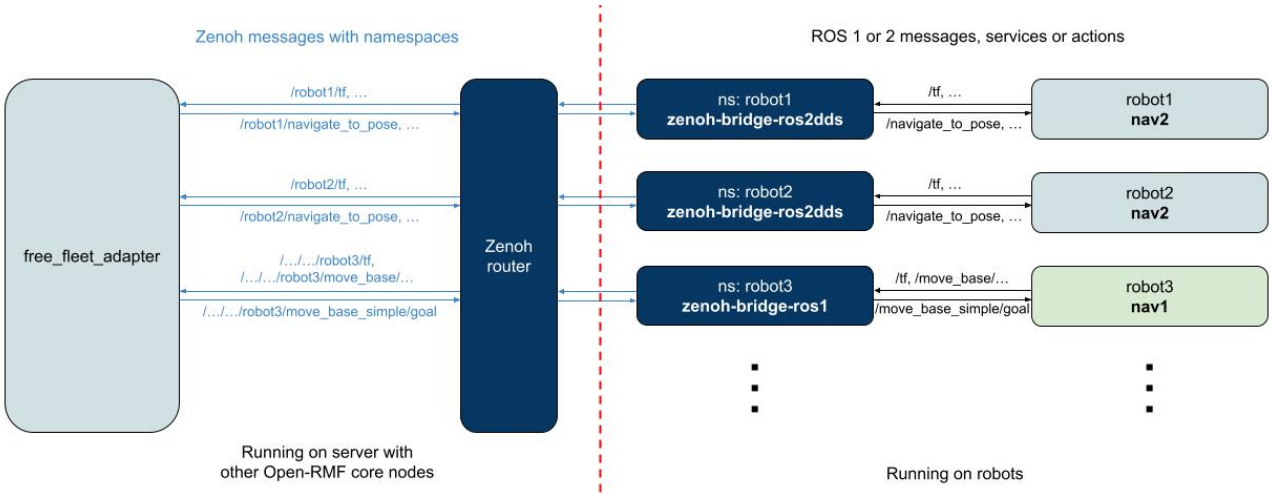


Figure 3.12: Free Fleet Adapter Communication

Setup fleet

To use the Free Fleet, it is necessary to create a `.yaml` configuration file for the fleet. This file describes the robots within the fleet that share the same characteristics. The configuration includes essential information such as the fleet name, velocity limit of the fleet, robot dimensions and mechanical details, power management parameters, the tasks the fleet is capable of performing, a list of all robots in the fleet with their individual names and their assigned chargers. It also needs the positional correlation between the robot's map and the RMF map.
After setting up the fleet configuration, the next step is to configure Zenoh for communication.

Setup Zenoh

Zenoh[23] is a pub/sub/query protocol that unifies data in motion, data at rest and computations. It combines traditional pub/sub with geo distributed storage, queries and computations, while maintaining high efficiency in both time and space. Zenoh allows selective topic transmission over the network, reducing bandwidth usage to specific servers.
To set up Zenoh for use with Free Fleet, the following steps are required:

- Zenoh router must be running on a server. This router can be launched using the command: (`$ zenohd`). Multiple instances of the Zenoh router can be run for redundancy.
- On each robot and on the RMF server, the standalone executable `zenoh-bridge-ros2dds` must be launched with the associated configuration file. This executable bridges all ROS 2 communications using DDS over Zenoh. The configuration of this file depends on the machine. On the server it's configured to accept all the traffic from each robot, while on the robot we need to setup:
  - The `namespace`, which must match the fleet configuration file namespace.
  - Enabling the publication of the robot's frame status topics (`.*/tf` and `.*/tf_static`) and battery information (`.*/battery_state`)
  - Enabling communication with action servers controlling navigation (`.*/navigate_to_pose`)

[23] https://zenoh.io/

– Zenoh-specific configuration details, such as the communication mode, the server address, the protocol and the port for connection endpoints (e.g., `tcp/<ip address server>:7447`)

Launching Free Fleet

After completing the setup, we can use the following launch file to start the Free Fleet adapter: `1rmf_mini_fleet_adapter.launch.xml`
This launch file takes as parameters the fleet configuration file and the route maps that the fleet will use. Once launched, it is possible to monitor the robot topics on the RMF server.
This is also replicated for the Jobot fleet, launching the following file
`1rmf_jobot_fleet_adapter.launch.xml`

3.4.4 Web interfaces

Open-RMF also offers a web-based interface for local testing, `rmf-web`[24], to visualize and control the system on a browser. It consists of two main components:

- api server : Based on the OpenAPI specification and implemented with the Python FastAPI library, it provides REST API methods to communicate with Open-RMF.
- dashboard : In our case, a demo dashboard that connects to the API server and allows control of the fleet.

Both components are available as Docker images and can be launched with the command:
`docker compose -f /path/to/compose.yml up`
Then we can connect to the web dashboard:

3.4.5 Deployment template

The project Open-RMF also offers a deployment template[25] for managing all the systems and all the tools to use the system on the web. It's based on Kubernetes[26], an open source system for automating deployment, scaling, and management of containerized applications, and the git repository contains the demo for simulation that is displayed in the interface.
It's built on Keycloak[27], to add authentication to the services, and Nginx[28], to manage the reverse proxy. We then modify the configuration files to adjust to our needs, obtaining a web interface that can be controlled in

[24]https://github.com/open-rmf/rmf-web
[25]https://github.com/open-rmf/rmf_deployment_template
[26]https://kubernetes.io/
[27]https://www.keycloak.org/
[28]https://nginx.org/

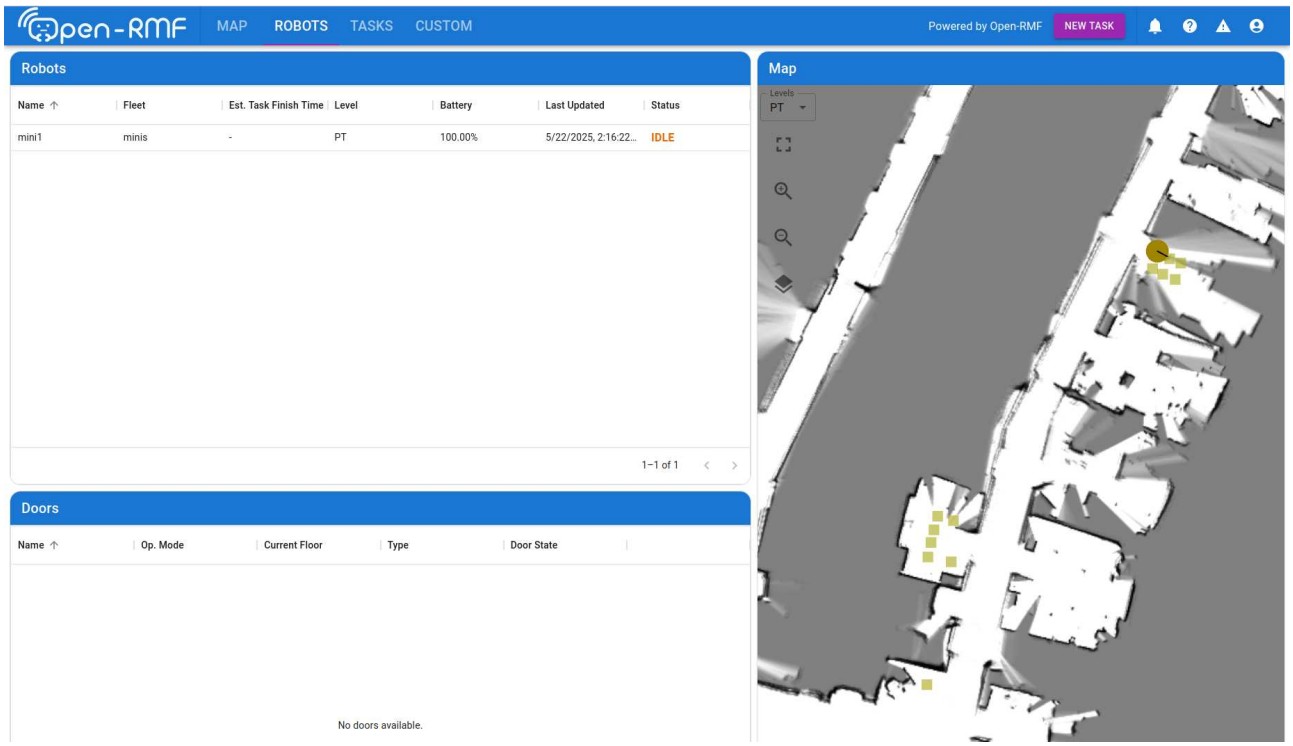


Figure 3.13: Open-RMF Web Dashboard

4. Testing and evaluation of the system

During the testing, we saw that the performance of odometry and localization needed to be improved through some parameter tuning. In the following paragraphs, I will describe the method used and the effects of changing parameters on the output.
We divided the study into two parts. The first part focuses on improving the estimation of the odometry by changing the parameters of the `robot_localization` package without utilizing the correction of the AMCL node.
Then, in the second part, we will set the parameters for the AMCL node to estimate the real position of the robot with respect to the map using the newly obtained odometry data. This process was conducted using recorded sensor data collected through the rosbag package, which is described in the next section.

4.1 Testing environment using Rosbag

To allow a better comparison of the output results obtained by changing the parameters, we need a method to record the data and replay it. This solution within the ROS ecosystem is the tool rosbag2, which allows recording and playback of communications in a ROS 2 system efficiently.

During the recording, we controlled the robots by publishing on the `/cmd_vel` topic using a joystick, following some paths that can be interesting to analyze. The paths are the following:

For the Mini, we chose to use two distinct paths for the different types of configurations to be performed. For the parameters associated with the `robot_localization` package, we followed a long path with many turns (see 4.1):

Figure 4.1: Path followed in record 001 for the mini

and the following one (Figure 4.2) for setting the parameters for the AMCL node. This path was chosen because,on the right side of the door, along the corridor, the doors are closed and can reduce the robot's ability to accurately determine its position, making it crucial to fine-tune the AMCL parameters accordingly:

Figure 4.2: Path followed in record 007 for the mini

For the Jobot, the path is the following one (Figure 4.3) and it has similarities with the second paths used for the Mini. After passing through the hallway, the doors were closed to assess how much this would affect the AMCL.

Figure 4.3: Path followed during record 001 for the jobot

Other records were made for validating the parameter choices.
To record data, we use the command `ros2 bag record`. For playback, it is essential to publish to the `/clock` topic in order to avoid errors related to discrepancies in time between the recorded data and the system time: "the timestamp on the message is earlier than all the data in the transform cache." This can be achieved with rosbag2 by using the `-clock` flag, which publishes to `/clock`. Additionally, the `use_sim_time` flag must be set to `true` for all nodes, just as it is required in simulation.

To reduce the testing time, we played the data at a rate five times higher using the flag `-r 5` for tuning the `robot_localization` , while for tuning the AMCL we used `-r 2`. This flag increases the output rate of each topic by n times. Tests were performed with rates from 1 to 5 for multiple recordings, and the output showed that the results given by the `robot_localization` package were similar. However, the result for the AMCL was similar only with a rate of 2.

Now that we have set the testing environment, the process of setting the parameters for the `robot_localization` package is as follows.

4.2 Parameters setting for the `robot_localization`

For the analysis of the data from `robot_localization` we wanted to visualize the results of the `robot_localization` node without the influence of the AMCL node. To achieve this, we set all the α parameters that define the behavior of the AMCL node to 0.0. This allowed us

to visualize in Rviz the position of the walls during the movements of the robot, allowing us to better understand its position and the type of error that the odometry exhibits.

As described before, the `robot_localization` package contains an implementation of the Extended Kalman Filter (EKF) that supports multiple sensors and allows us to select which data fields are fused with the current state estimation. It also provides its process noise covariance matrix, Q, for tuning to obtain better performances.
To better understand the dynamics of our changes in the parameters, we describe the theory behind it in the following paragraph.
We want to know the state x, given a non liner system f with process noise w:

$$x_k = f(x_{k-1}) + w_{k-1} \tag{4.1}$$

and with a measurement z given by the sensor model h with noise v :

$$z_k = h(x_k) + v_k \tag{4.2}$$

Using a prediction step:

$$\hat{x}_k = f(x_{k-1}) \tag{4.3}$$
$$\hat{P}_k = F P_{k-1} F^T + Q \tag{4.4}$$

where:
- P is the estimated error covariance.
- F is the Jacobian of f.
- Q is the process noise covariance.

And the correction step:

$$K = \hat{P}_k H^T (H \hat{P}_k H^T + R)^{-1} \tag{4.5}$$
$$x_k = \hat{x}_k + K(z - H\hat{x}_k) \tag{4.6}$$
$$P_k = (I - KH)\hat{P}_k (I - KH)^T + KRK^T \tag{4.7}$$

where:
- H is the observation matrix.
- R is the measurement noise covariance.
- K is the Kalman gain.

In short, for the process noise covariance matrix Q, the better the **omnidirectional motion model** of the `robot_localization` package matches the robot, the smaller these values can be. However, if we find that a given variable is slow to converge, one approach is to increase the process noise covariance diagonal value for the variable in question. This will cause the filter's predicted error to be larger, leading the filter to trust the incoming measurement more during correction.

In the next sections, we describe our procedure to tune these parameters for the Mini and for the Jobot.

4.2.1 Setting up the EKF of the mini

The data available for the Mini are given by the mounted IMU and its estimated odometry.

Using this data, and without any tuning, we obtain:

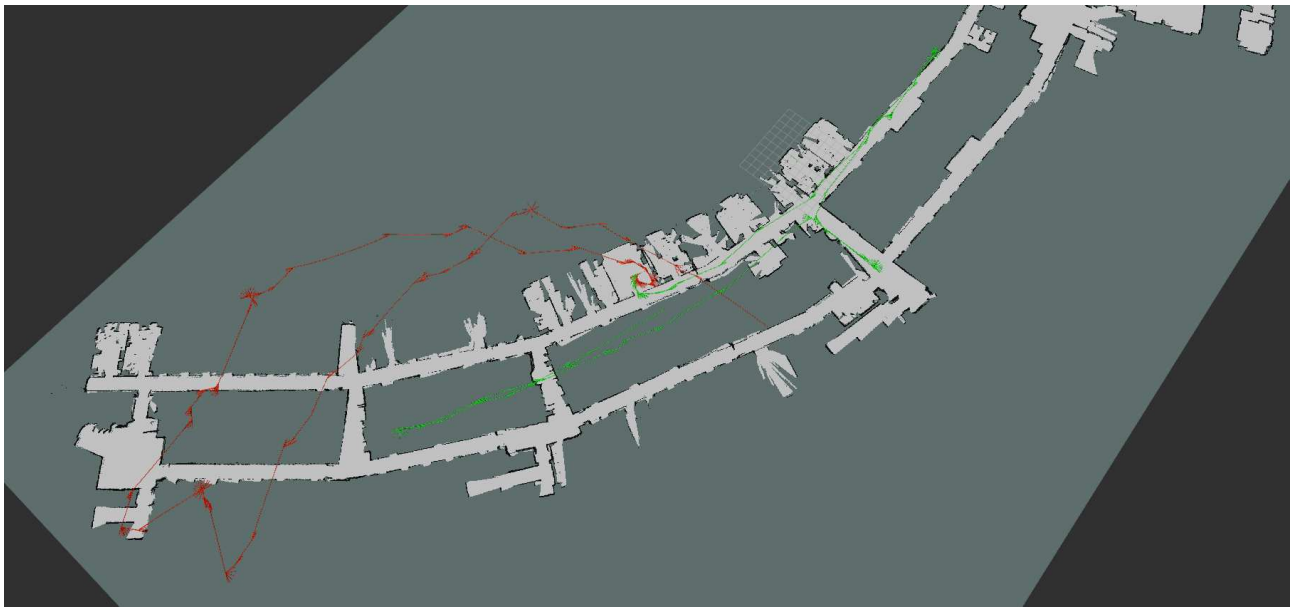


Figure 4.4: Mini odometry given by the robot (red) and odometry given by the `robot_localization` package (green)

The figure shows the magnitude of the angular error for the Mini. To compensate for this error, we analyze which data our sensor are able to provide:
IMU (topic `/imu/data` with an output frequency of 100 Hz)

- orientation : cannot be used due to random errors from the sensor, as visible in this picture.

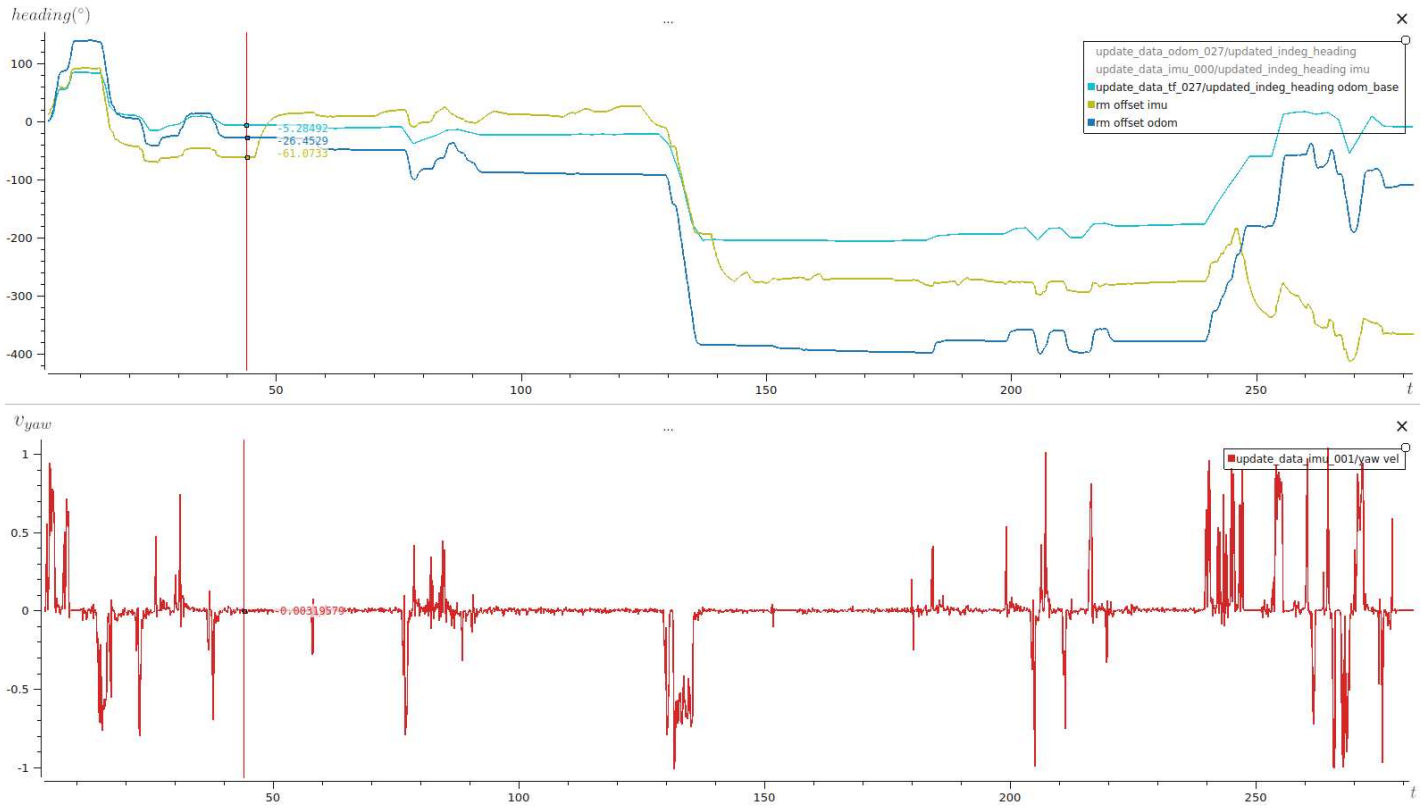


Figure 4.5: Heading data from the `robot_localization` (light blue), IMU orientation (green) and odometry from the robot (blue) versus the v_{yaw} measured by the gyroscope of the IMU

- gyroscopic : we use its data about v_{yaw}
- acceleration : added a constant increment of the position and incremented the error, so it was not used.

odometry (topic `/odometry/wheels` with a output frequency of 15 Hz)

- pose : it is not correct and adds a lot of error to the estimation.
- velocity : it is also the input given to the robot, providing direct information about velocity, so we take v_x and v_y from the data. We tried to use v_{yaw}, but it introduced a lot of error in the estimation.

After choosing the input data to use, we tried changing the values of the process noise covariance Q, obtaining these results 4.1. This test was performed using record 001, where the start position is inside the office and the end is outside the door of the office (a distance of about 4 m) with a total length of about 187 m:

v_x (-)	v_y (-)	v_{yaw} (-)	data name	diff start end (m)
0.001	0.001	0.001	000	12.36
0.01	0.01	0.01	001	5.88
0.1	0.1	0.1	002	5.10
10.0	10.0	10.0	003	4.37
50.0	50.0	50.0	004	4.31
0.001	0.001	0.1	005	4.83
0.001	0.001	10.0	006	4.33
0.001	0.001	50.0	007	4.27
0.001	0.1	0.001	008	5.88
0.001	10.0	0.001	009	5.88
0.001	50.0	0.001	010	5.88
0.1	0.001	0.001	011	5.94
10.0	0.001	0.001	012	5.94
50.0	0.001	0.001	013	5.97

Table 4.1: `robot_localization` results for the Mini

Some of the results:

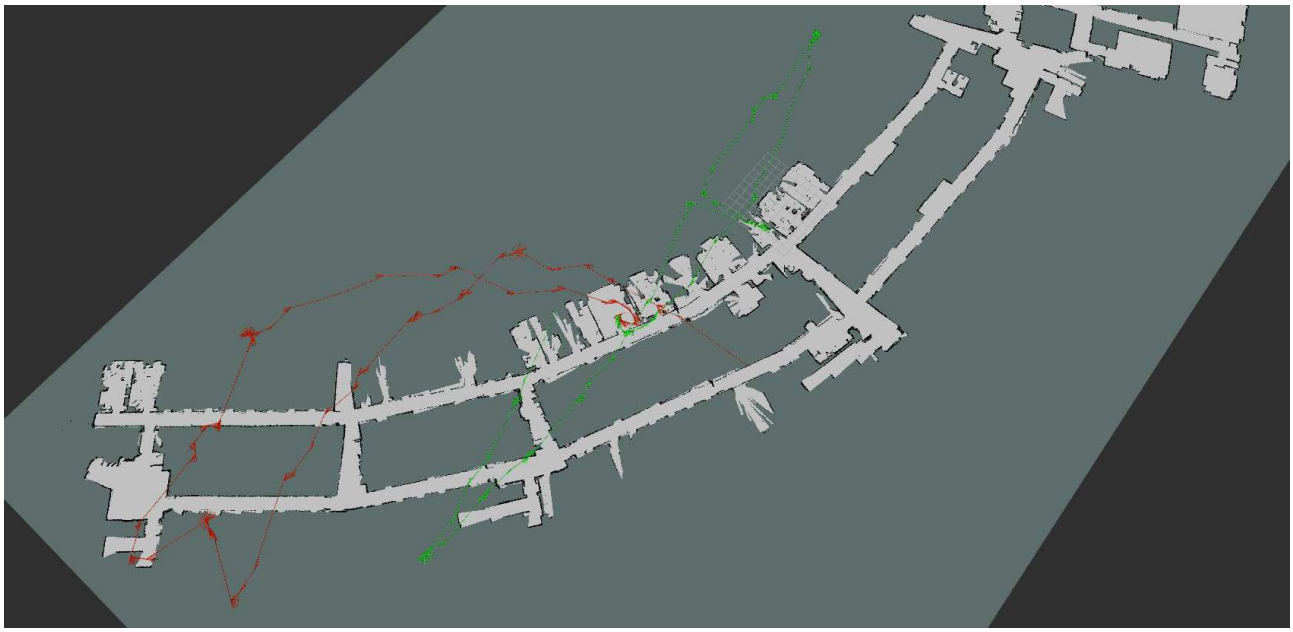


Figure 4.6: Results from data 005

All the tests are displayed in the next graph:

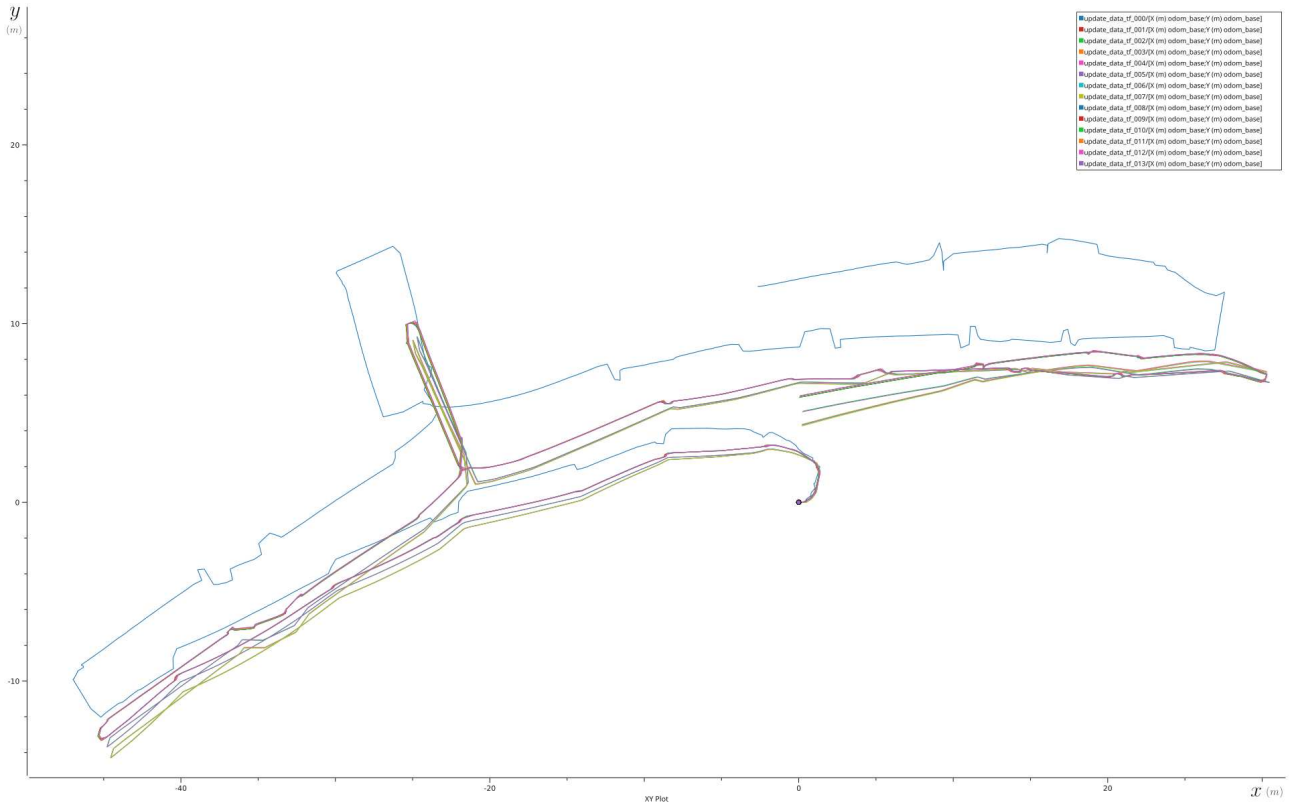


Figure 4.7: Results changing parameters of the process noise covariance using the record 001

We can see that the best results were achieved by incrementing the value of the process covariance matrix for v_{yaw}, while changing the other inputs did not affect the results. More tests were performed with larger and smaller values of those variables, but they did not change the outcome. Therefore, the best result was obtained with a value for v_{yaw} of 50.0 while for the other two it was kept at 0.1, and this gives us the following result:

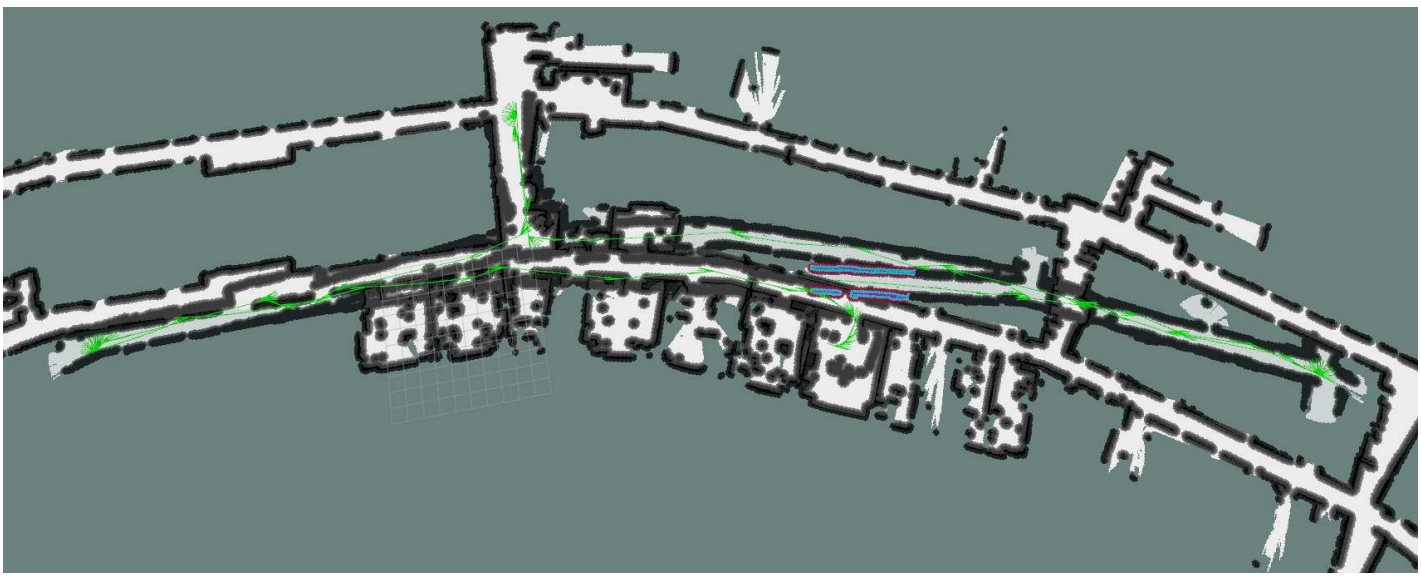


Figure 4.8: Best result for the Mini with the information of walls position

As shown in Figure 4.8, examining the correspondence between the walls of the map and the walls recorded during navigation indicates that the estimated position is incorrect. However,

adjusting the values in the process noise matrix did not improve this estimation. This is a correction that the AMCL node is intended to address.

4.2.2 Setting up the EKF of the Jobot

A similar procedure to that used for the Mini was performed for the Jobot. We discovered that in both recordings, the robot, while moving through a fire door, makes a "jump" visible in the odometry. This may be caused by a small step needed to pass through the fire door.
From some initial tests, it is evident that the estimation of the length of the path followed by the robot is correct, so we focused our attention on the observable error in rotation. The data of measure 000 was performed with only the *yaw* of the odometry of the robot:

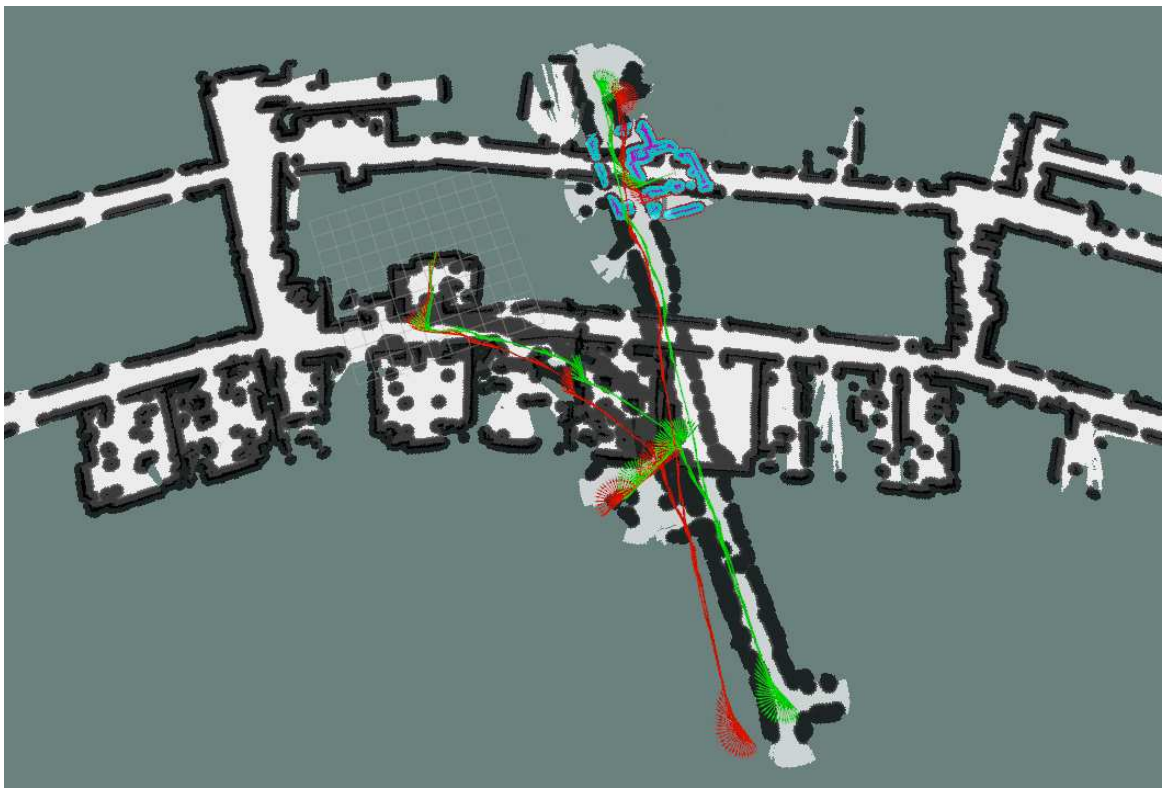


Figure 4.9: Estimation obtained with the *yaw* of the Jobot's odometry (green) with the estimation from the robot (red)

Having seen the results and tested that some changes to the process noise covariance did not affect the outcomes, it was decided to remove this data and evaluate the results using v_{yaw} from only the IMU (data 001):

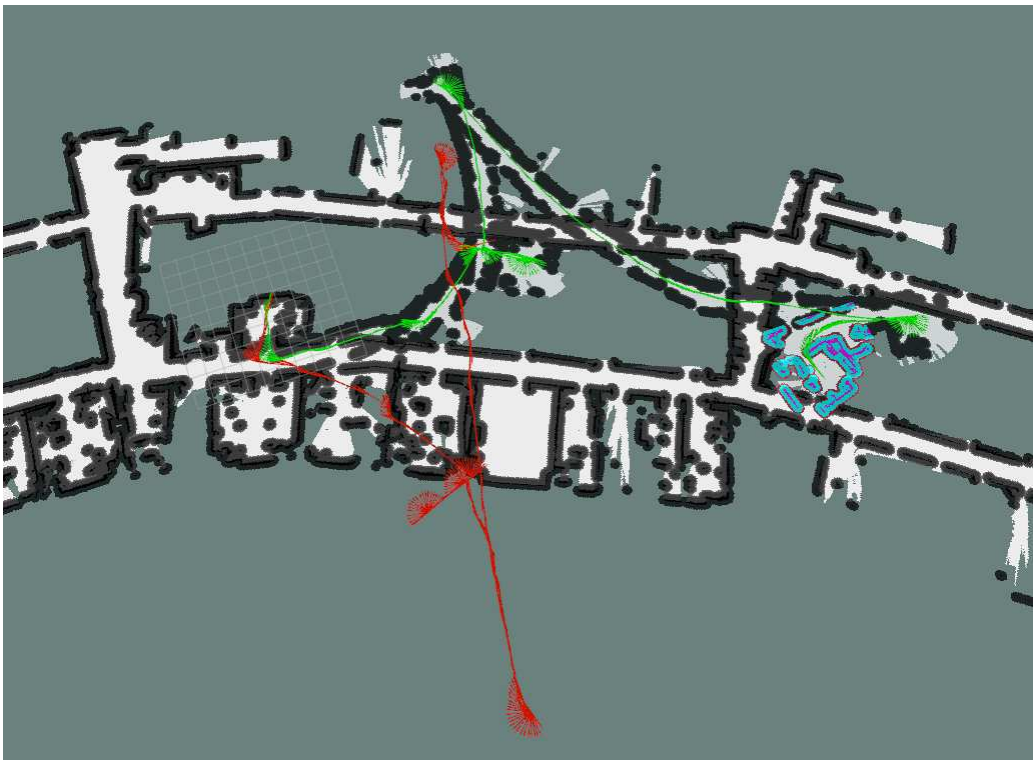


Figure 4.10: Estimation obtained with only the v_{yaw} of the Jobot's IMU (green) with the estimation from the robot (red)

It overestimates the rotation, so we tried summing v_{yaw} from both the IMU and the odometry provided by the Jobot (data 002):

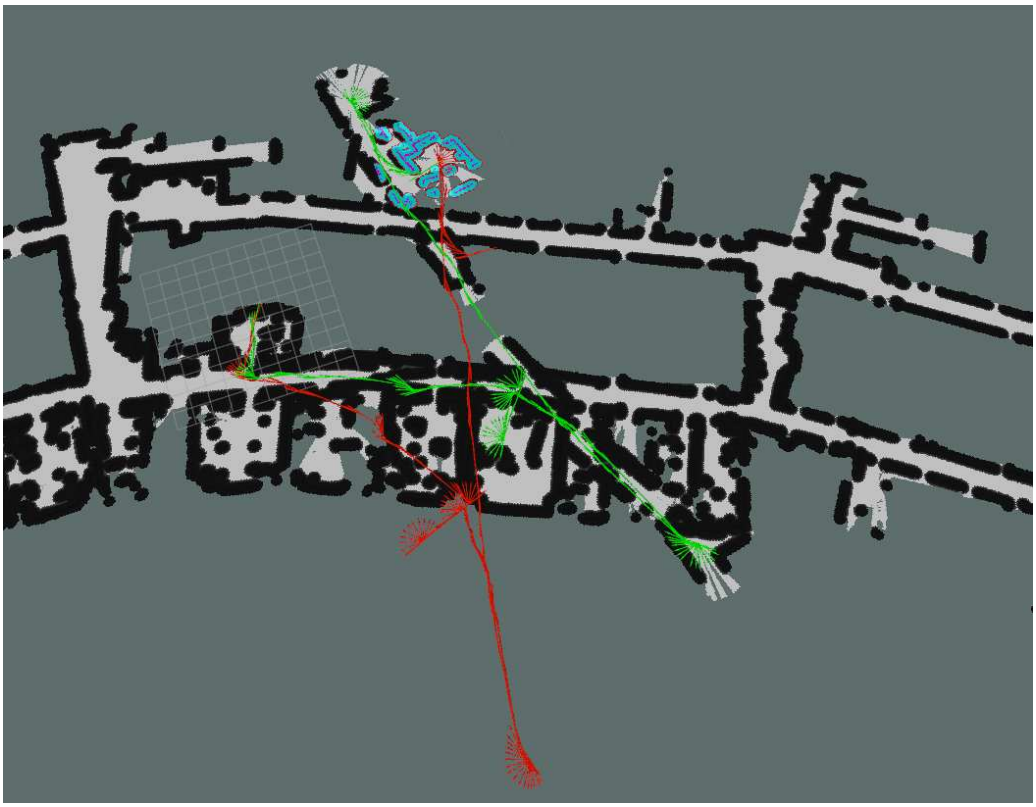


Figure 4.11: Estimation obtained with both the v_{yaw} (green) with the estimation from the robot (red)

Since this was the best outcome, we decided to use them as input and change the values of Q for the v_{yaw} because the error in translation is very low.
All the results are:

v_{yaw} (-)	record name	diff start end (m)
-	000	14.28
-	001	32.67
0.1	002	13.88
10.0	003	13.26
100.0	004	8.74
200.0	007	4.31
300.0	008	1.83
400.0	009	3.50
320.0	012	1.63

Table 4.2: Result tuning parameters process noise covariance for Jobot

Considering that, for the path followed, the distance between the initial and final positions is approximately 1 m. We obtained the best result with a value of 320 for the process noise covariance for the value of v_{yaw}.
The best outcome is visible in the following figure:

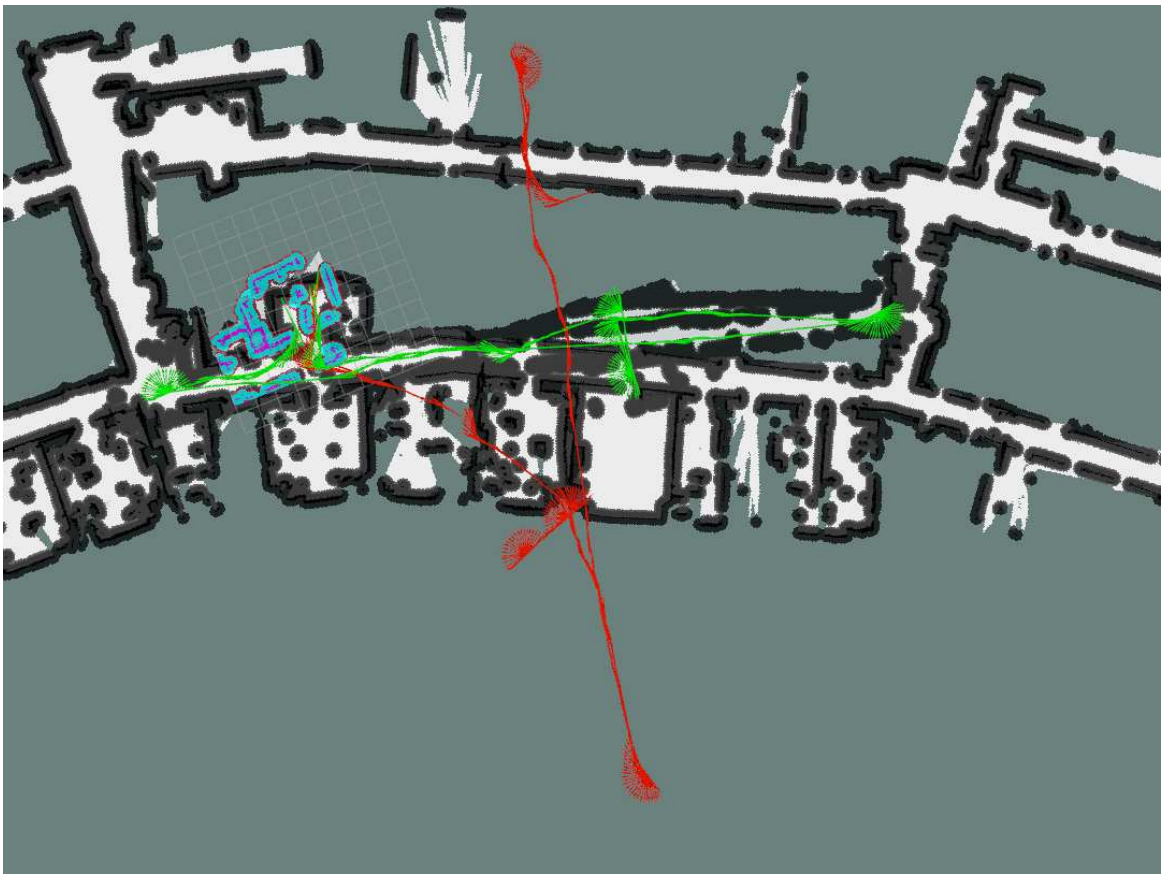


Figure 4.12: Best result obtained for Jobot (green) with the estimation from the robot (red)

The plotted data are:

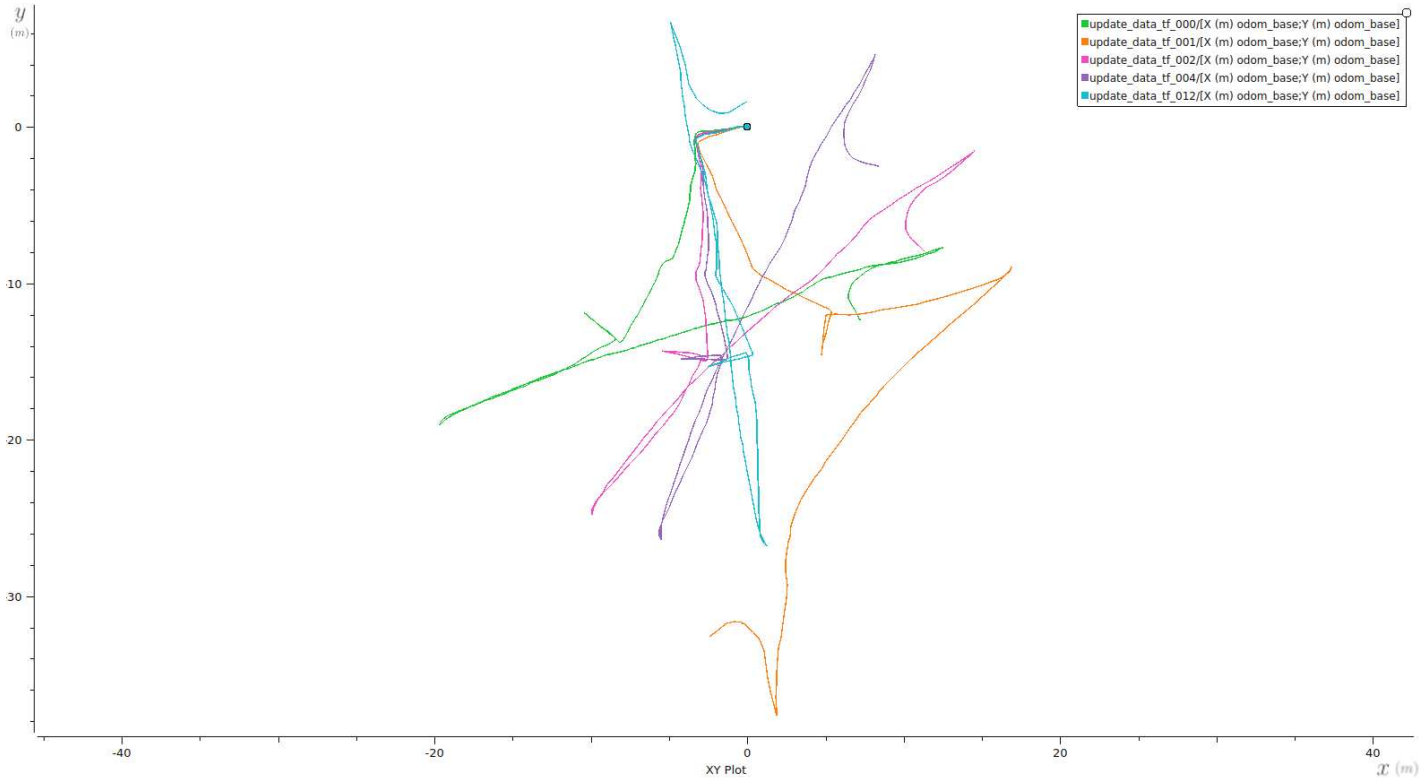


Figure 4.13: Results from the tuning of the process noise covariance for the Jobot plotted

4.3 Parameter tuning for the AMCL

Calibrated the process noise matrix Q for both robots. Now, we need to estimate the correct position relative to the map (global localization), which is achieved using the AMCL node as described earlier. This also compensates for all the errors caused by incorrect odometry estimation. In this section, we will adjust the α parameters and observe the results. The description of these parameters is as follows:

- $\alpha 1$: Expected process noise in odometry's rotation estimate from rotation.
- $\alpha 2$: Expected process noise in odometry's rotation estimate from translation.
- $\alpha 3$: Expected process noise in odometry's translation estimate from translation.
- $\alpha 4$: Expected process noise in odometry's translation estimate from rotation.

4.3.1 Setting up the AMCL of the Mini

These tests were performed on record 007 and the starting position is about 0.5m from the ending position. This choice was made because it was conducted in a challenging situation that demonstrates the difficulties an AMR may encounter. In the first half of the path, the robot encounters enough features that can be used by the AMCL node to retrieve the correct position relative to the map (the doors in that part of the corridor are open, providing sharp edges for the laser scanner). However, in the second half, there are not many environmental details (the doors of the other half of the corridor are closed), and the position estimation may

fail. In the following figure, the odometry estimation with the α parameters set to 0 for record 007 is shown.

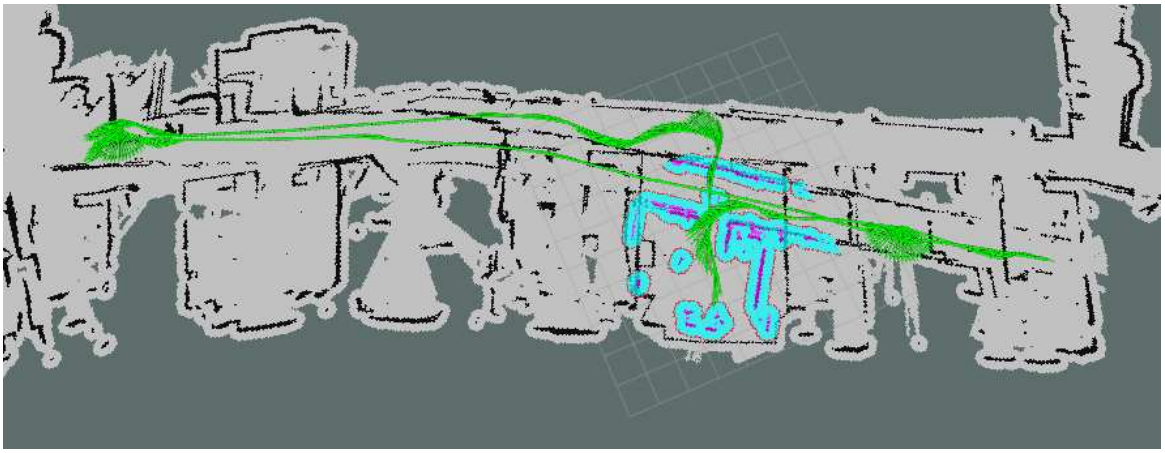


Figure 4.14: Best result with the Mini using the `robot_localization` for the record 007

Then, we tested changing only one parameter at a time, increasing its effect on the estimation while paying closer attention to the parameters that most effectively correct the estimation made in the `robot_localization` node. This is necessary to compensate for errors during rotation and translation. This can be compensated using the parameters: $\alpha 1$ and $\alpha 3$.
The results are shown below, with the first row displaying the data from odometry provided by the localization node without correction from the AMCL node.

$\alpha 1$ (-)	$\alpha 2$ (-)	$\alpha 3$ (-)	$\alpha 4$ (-)	distance (m)	data name
0.0	0.0	0.0	0.0	1.92	-
0.001	0.0	0.0	0.0	1.32	000
0.01	0.0	0.0	0.0	0.60	001
0.1	0.0	0.0	0.0	0.64	002
0.0	0.001	0.0	0.0	0.58	003
0.0	0.1	0.0	0.0	0.51	004
0.0	0.0	0.001	0.0	2.01	005
0.0	0.0	0.01	0.0	2.27	006
0.0	0.0	0.1	0.0	2.32	007
0.0	0.0	0.0	0.1	3.08	008
0.01	0.01	0.01	0.0001	0.53	009
0.01	0.01	0.1	0.0001	0.69	010

Table 4.3: Result AMCL Mini

Changing $\alpha 1$ and $\alpha 2$ fixes the rotation error, but we observe significant oscillation in the position correction if we increase them too much; this was seen in data 004. In the end, we decided to keep both parameters around 0.01. The resulting odometry plot (figure 4.15) shows that the rotation was corrected.

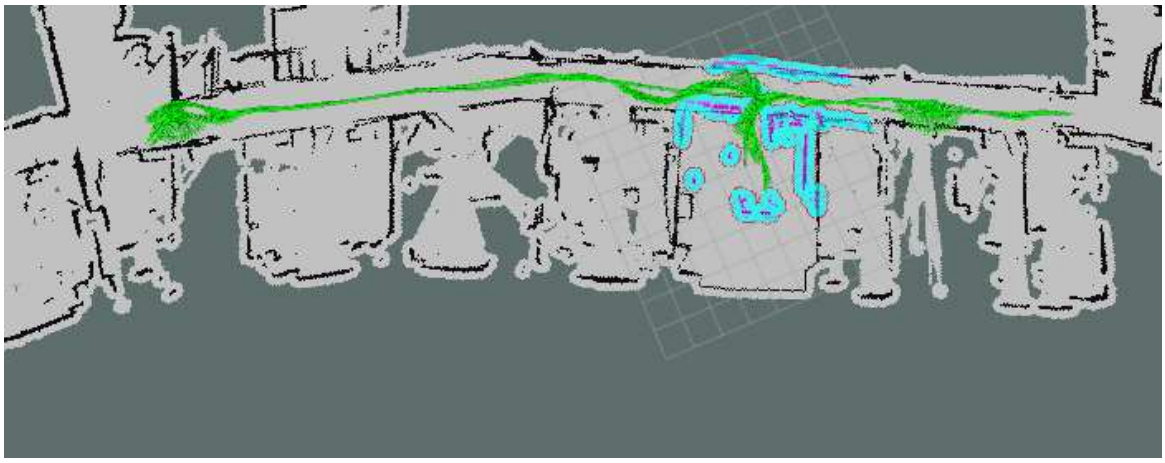


Figure 4.15: Result obtained by AMCL without translation correction

This allowed us to observe the corrections made for the heading. It is evident that the error increases as the robot moves, and we can see that we performed a good correction in rotation using the `robot_localization` package, but not in translation.

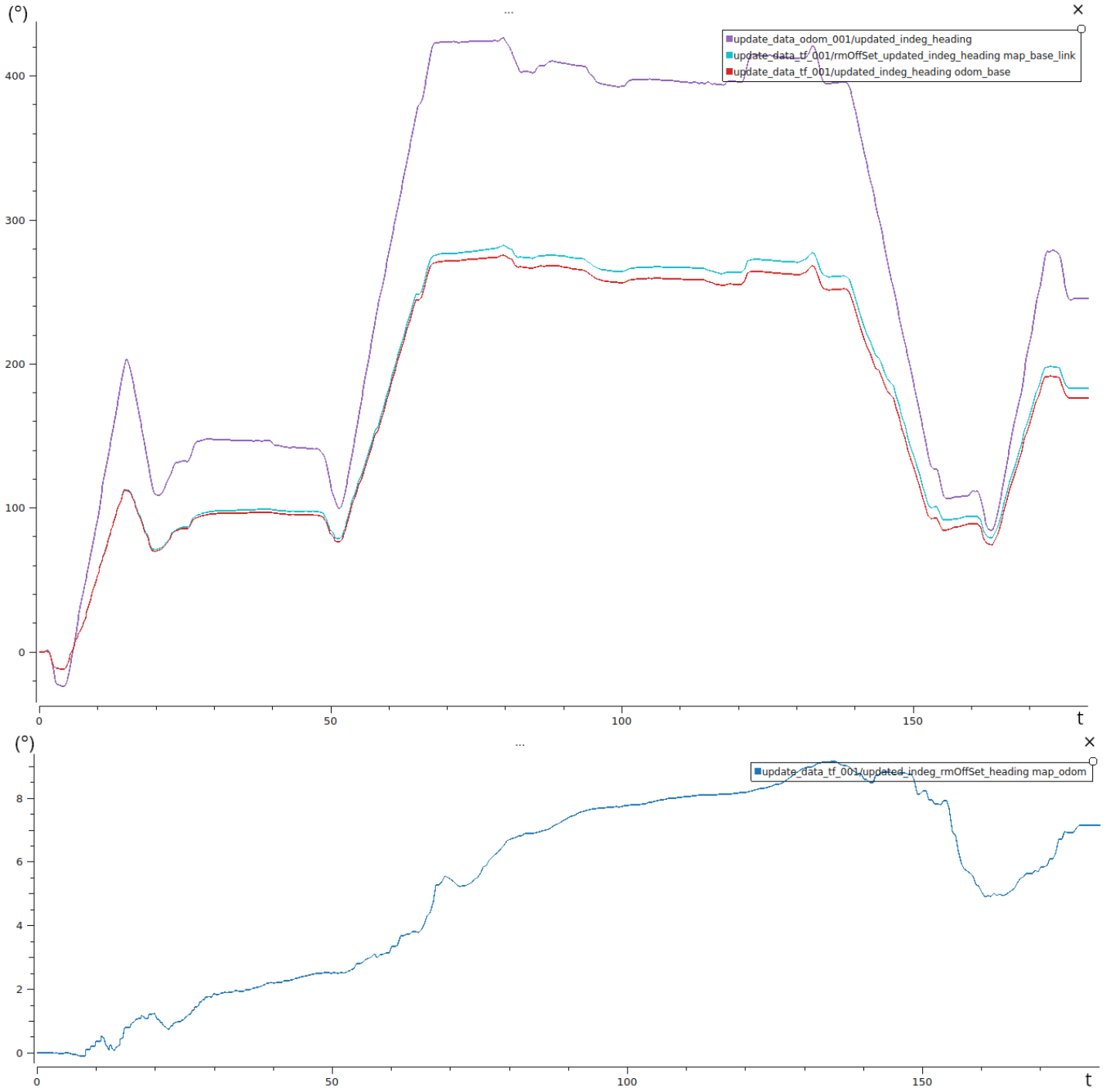


Figure 4.16: Top: Heading values returned by the system; Bottom: Heading correction for the Mini performed by AMCL

The table shows that without adding the correction for rotation, the translation correction parameter alone does not solve the problem. During testing (data 009), it was possible to observe that the combination of these parameters allowed for a correction of the position on the left side of the path (the one with more map detail). This did not completely resolve the issues on the right side (with less map detail), but did make some corrections. The final solution (data 010) is considered the best, effectively addressing the position error during movement. The parameter was kept high to accommodate worse scenarios, such as longer hallways.

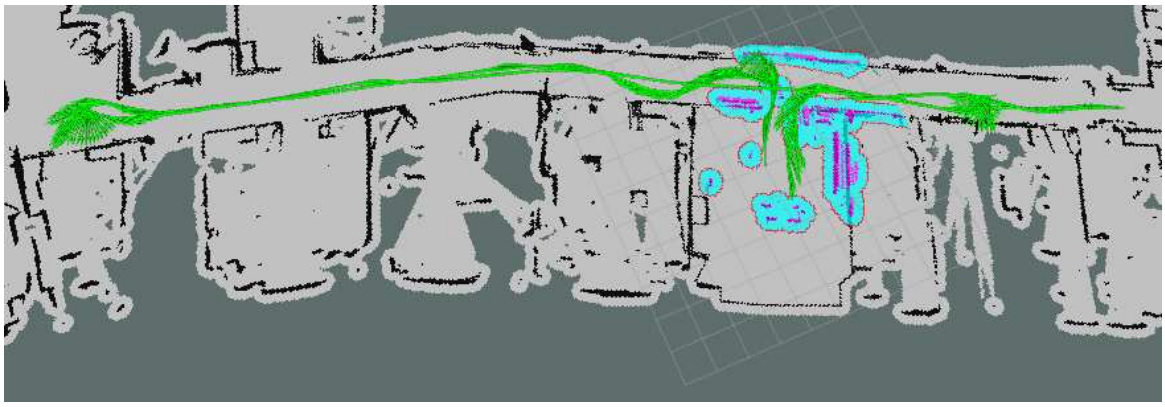


Figure 4.17: Best result obtained by AMCL on the Mini on record 007

The data is plotted in Figure 4.18 .

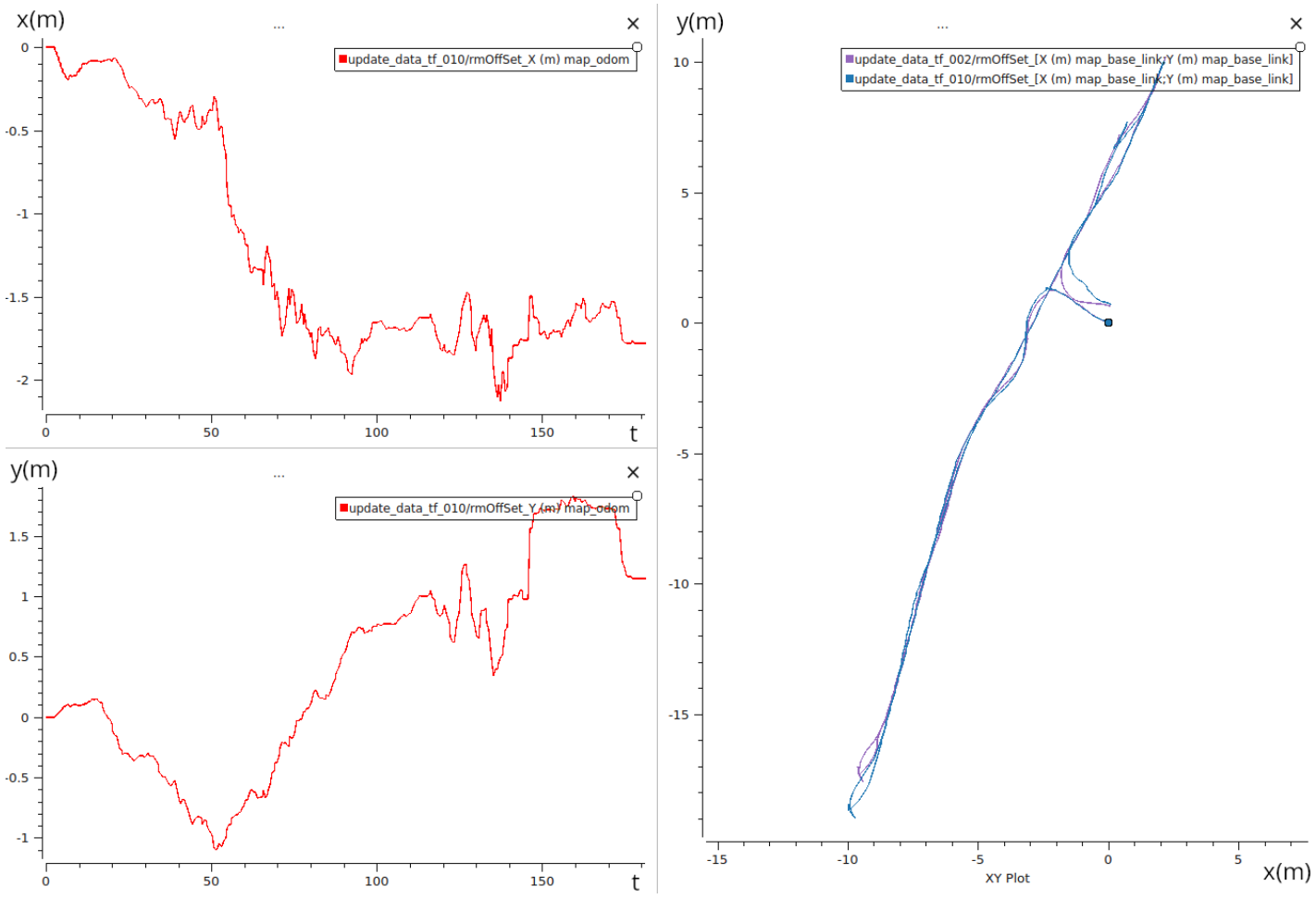


Figure 4.18: Left: Position correction performed by AMCL; Right: Path corrected by AMCL

It is evident that to the left of the starting point, the translation error is fully compensated. In contrast, to the right, the performance is poorer due to a lack of features, as many doors are closed along that path.

4.3.2 Setting up the AMCL for the Jobot

As mentioned before, in both records, when the robot passed the fire door, it "jumped," creating an error in the estimation. We decided to keep the data and observe how the AMCL package recovers from this jump. For record 001, which we will use in our test, the difference in distance from the starting position to the ending position is around 0.6 m. As shown in figure 4.12, the error that needs correcting is the angular position generated during rotation around the axis. Therefore, as previously described, we need to adjust the parameter $\alpha 1$, which relates to the expected process noise in the odometry's rotation estimate. For this test, we will increase that value and observe how the output changes.

$\alpha 1$ (-)	$\alpha 2$ (-)	$\alpha 3$ (-)	$\alpha 4$ (-)	distance (m)	data name
0.0	0.0	0.0	0.0	1.63	-
0.001	0.0001	0.0001	0.0001	1.58	000
0.01	0.0001	0.0001	0.0001	2.16	001
0.01	0.001	0.0001	0.0001	0.71	002
0.01	0.001	0.001	0.0001	0.87	003
0.01	0.001	0.001	0.0001	0.74	004
0.01	0.001	0.001	0.001	0.59	005

Table 4.4: Result AMCL Jobot

In the first row, the output of the estimation in odometry without correction is reported. With data 000 and 001, we observe that the robot did not recover from the jump, resulting in an increasing error.

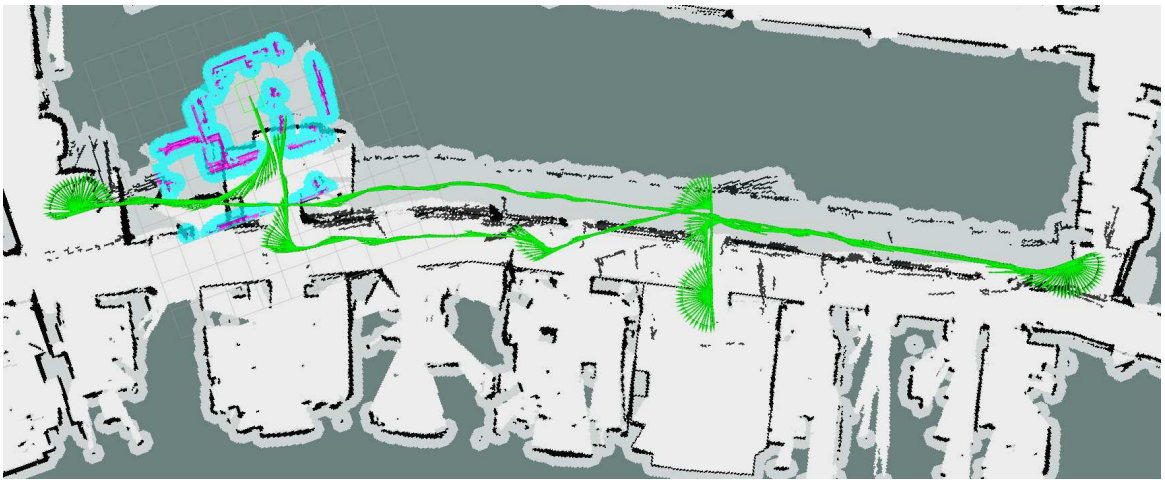


Figure 4.19: Figure of data 001 obtained by Jobot

Because we didn't increase the correction for rotation during translation $\alpha 2$, and since we moved straight after the jump, AMCL did not fix the rotation error. Therefore, we increased $\alpha 2$, achieving good results. To obtain even better outcomes, we added a bit more correction with $\alpha 3$ and $\alpha 4$. Considering that our test path was short and there could be additional errors in translation, we also observed an increase in estimation performance. The plot of the data for the best results is as follows:

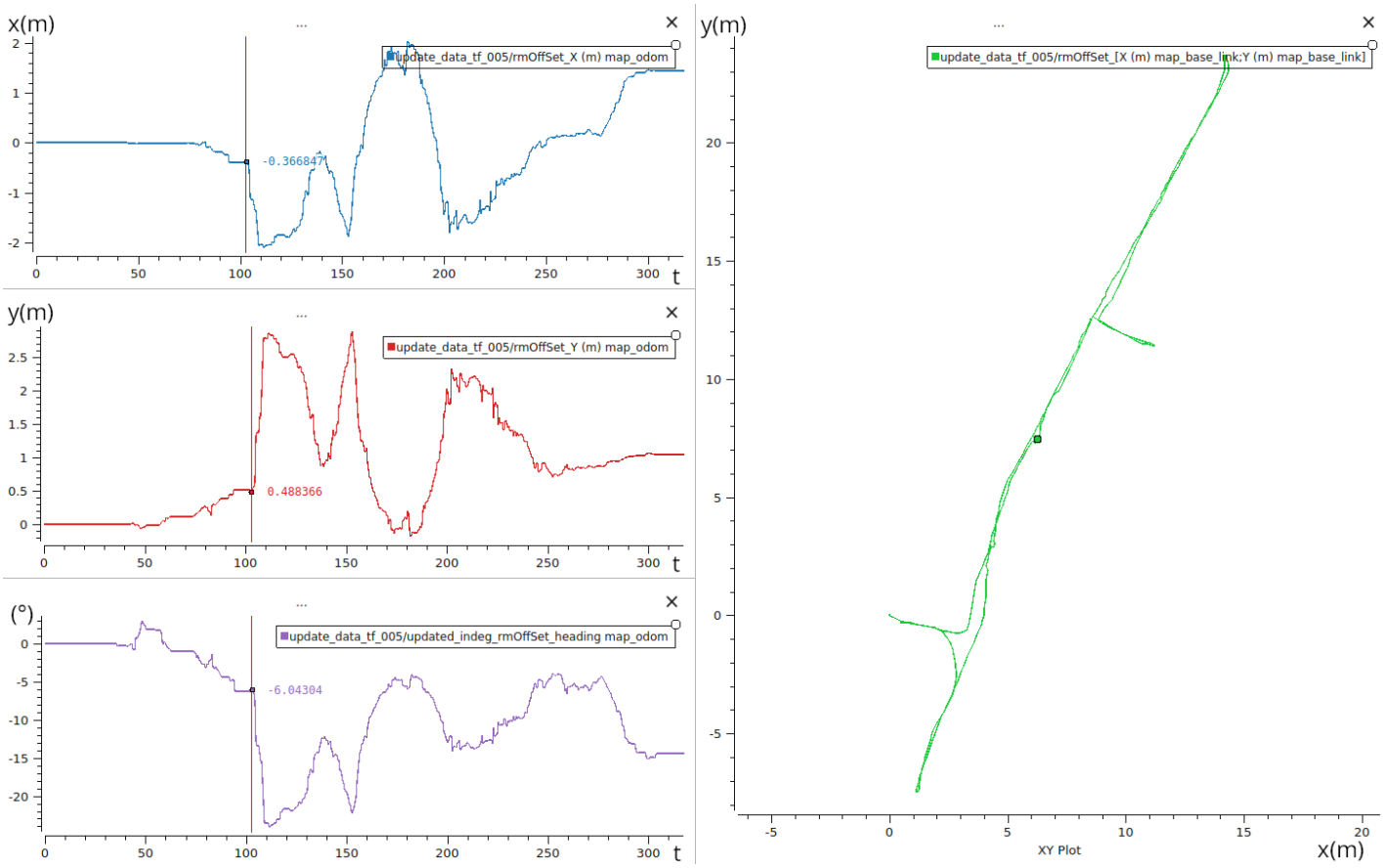


Figure 4.20: Left: Correction along x, y, and heading for the best result on Jobot with Record 001; Right: The path followed by the robot.

At the vertical red line, it is visible that the small step generated a very high error in all three states: x, y and yaw, but AMCL managed to correct it perfectly. It was also tested on record 002, which was created with the intention of generating errors in the estimation due to an environment lacking a sufficient number of features. Specifically, it was decided to move the robot along the corridor and, before coming back, to close the doors of some offices. Since the odometry estimated by the robot is good and the corrections in the AMCL are minimal, this did not create a problem. Figure 4.21 shows the result:

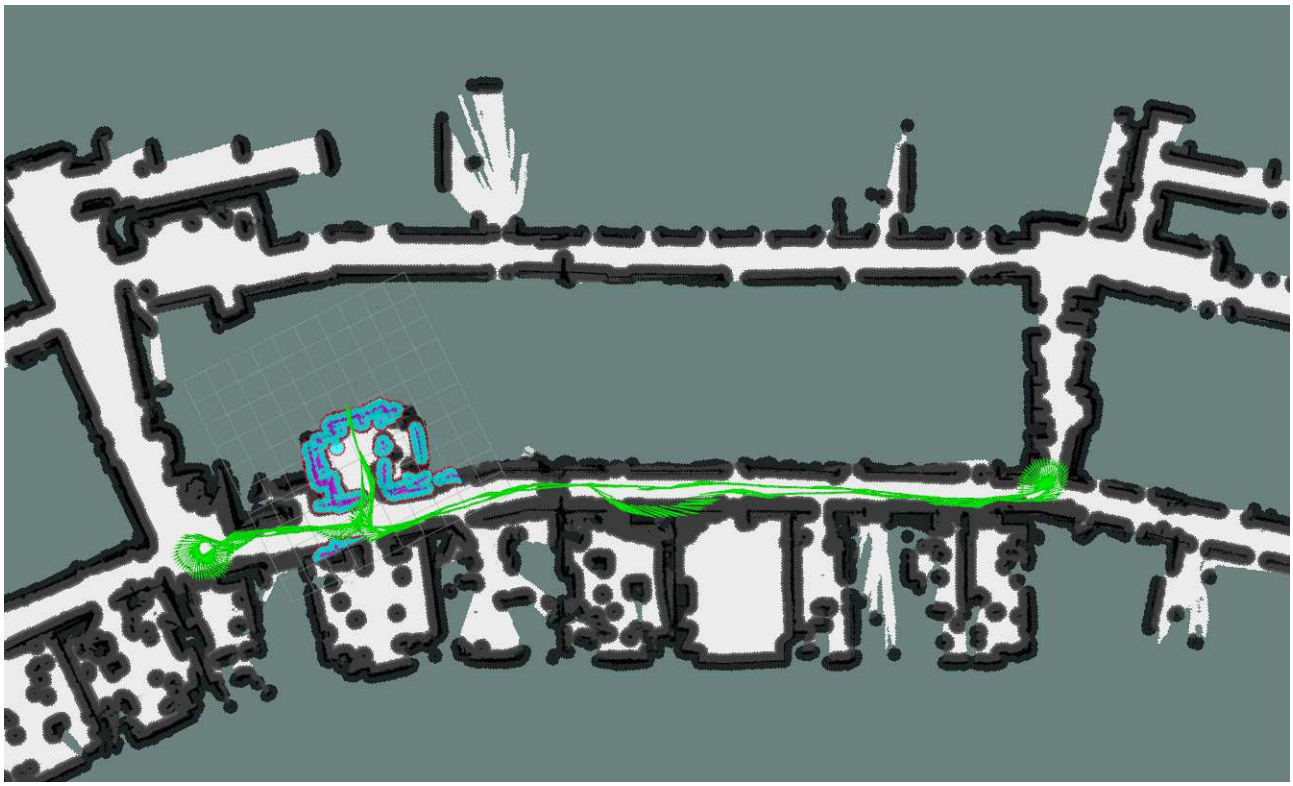


Figure 4.21: Result of the Jobot with record 002

4.4 Navigation startup procedure

Similar to the structure used in this paper[43], we create the services to start the communication with the sensor and, only after that they have been launched successfully, we launch the navigation stuck to control the UGVs. Since the systems are based on Ubuntu, the services use `systemd`[1], which provides a system and service manager with a logging daemon.

4.5 Navigation problems

4.5.1 In simulation

Now we tested the AMRs in simulation and discovered that for the Jobot, during sharp curves, as shown in figure 4.22 below, the AMR stopped right before the turn, rotated in place, and then proceeded to hit the wall, eventually stopping.
This issue is caused by the local planner failing to generate an effective command to move the AMR. It takes the global plan and the local costmap and produces command velocities to direct the AMR toward the goal. One solution was to change the local planner from the default Dynamic Window Approach DWB successor of the DWA, based on the sampling of possible velocities and selecting the one with the best score, to the Model Predictive Path Integral Controller MPPI[44], which uses a sampling-based approach to select optimal trajectories, optimizing across successive iterations. This approach has shown better performance[34] and it worked without any configuration.

[1]https://systemd.io/

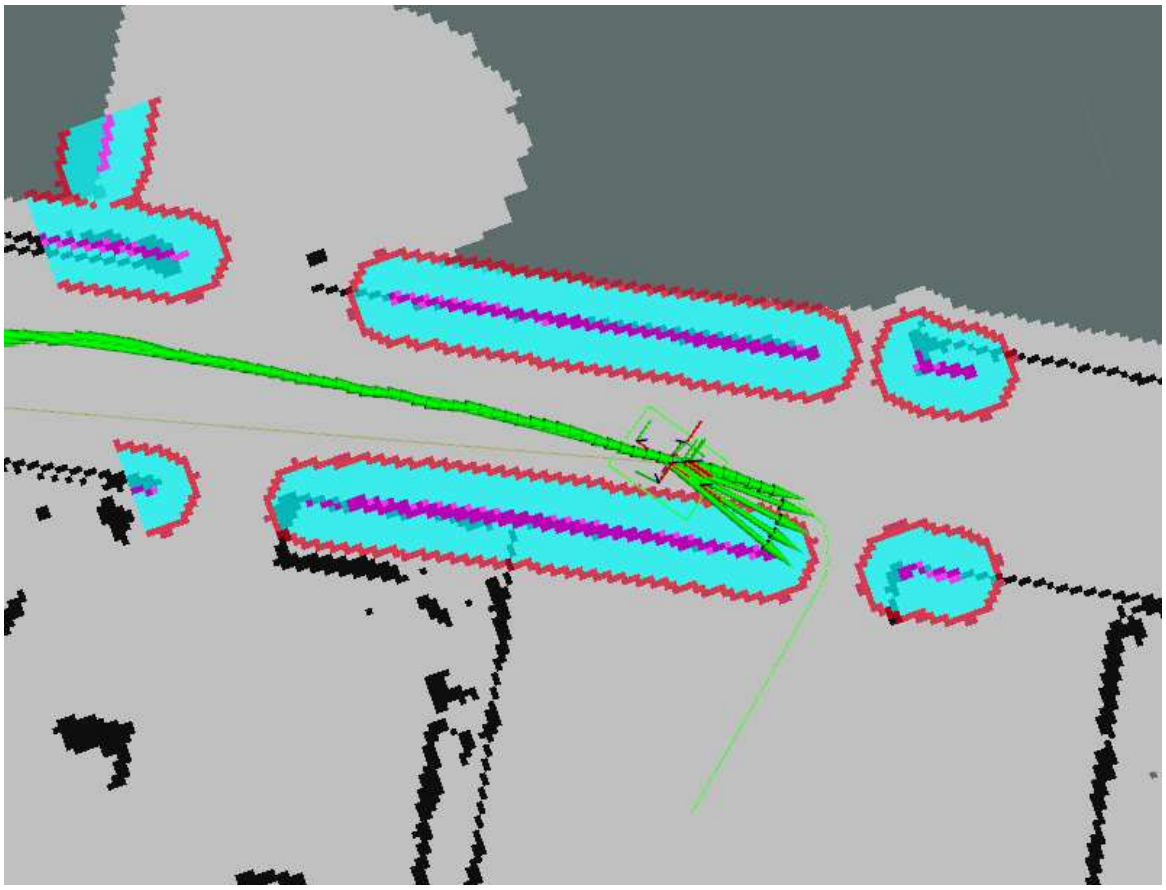


Figure 4.22: Problem with Local Planner for the Jobot

4.5.2 In reality

After the calibration of the parameters using the records, we tested how they perform in reality,
.

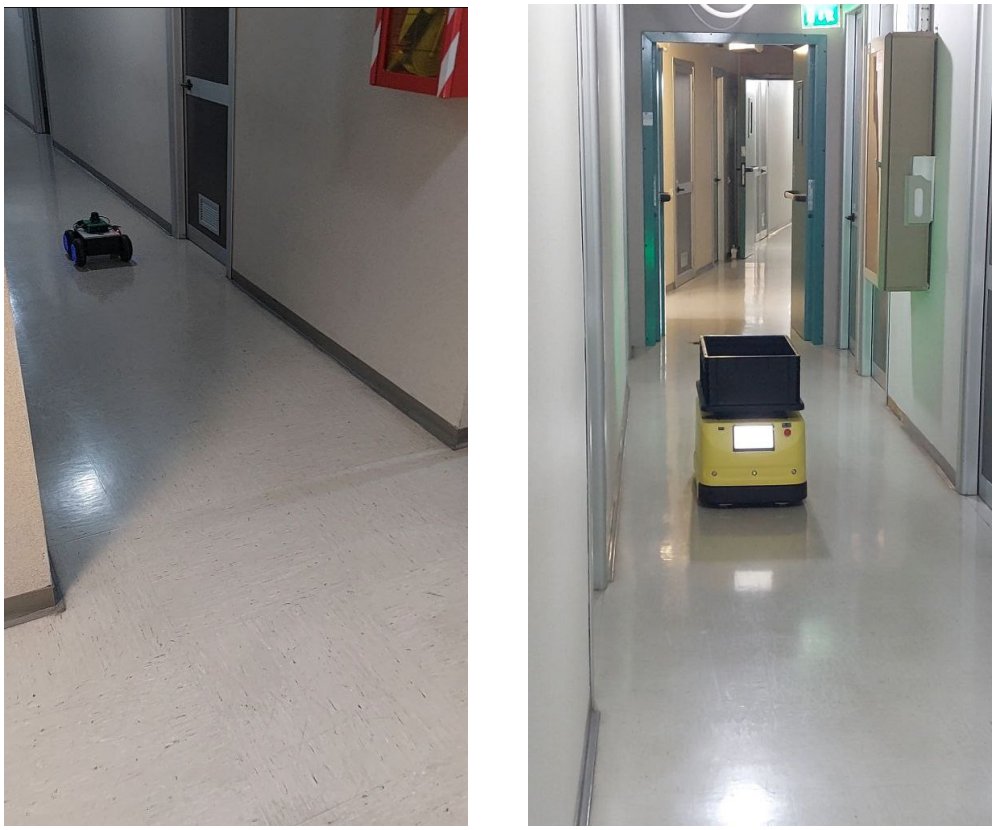


Figure 4.23: The Mini and the Jobot moving through the building

Mini test results

The results for the Mini improved a lot. While the doors were open the system manages to correct its position and move through the building without issue. But after the end of working time or if multiple people close their office doors, the robot could fail to correct its position. This happened sometimes when moving through a 22 m long hallway with the door closed and the system didn't manage to correct an error of about 1 m. After returning to the start position and since the robot was reporting that it was at the right position, we can deduce that there was no correction performed by the AMCL during that movement. While, in other situations, it managed to recover its position thanks also to the actions (spin and backup) of the Nav2 behavior tree, which the AMCL "used" to correct the robot's position with respect to the map. If we ask the robot to move to a location and the AMCL doesn't manage to correct the position in time, the robot will respond that no path was found. This happens because the position of the door in the global costmap is not aligned with the position of the door in the local costmap, resulting in the path being blocked for the global planner, as shown in Figure 4.24.
One solution could be to rotate beforehand, allowing the AMCL to correct the error between the odometry frame and the map frame by adjusting the parameter that regulates the error caused by rotation. This can be implemented in open-RMF by adding a vertex before the door to help correct the translation error, enabling the generation of a path to enter through the door. Another solution is to improve the odometry estimation of the Mini by using absolute reference markers visible to a camera[2][27], allowing the localization node to have periodic absolute corrections, similar to GPS. This enhancement can also facilitate corrections and movement outside the facility, allowing navigation between buildings to correct its absolute position.

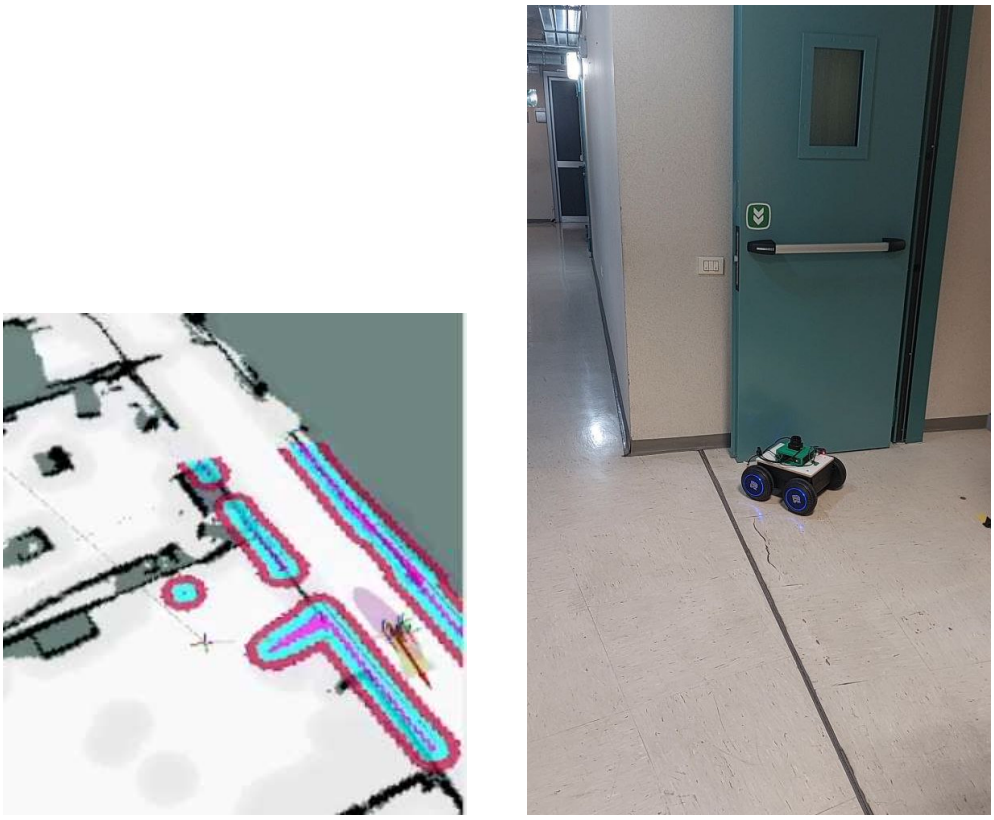


Figure 4.24: Examples when the estimation from AMCL fails

Jobot test results

Since the estimation of its odometry is better, the problem of incorrect position relative to the map was not present.

High oscillation movement was present when it was supposed to go in a straight line. This was solved by adjusting the parameters for the MPPI local planner, following the guide in the documentation.

Tests were performed using the DWB Planner to see the behavior seen in simulation was the same. During test the problem appeared and so we choose to use the MPPI controller.

4.6 Other problems discovered during testing

After some testing, these are the problems we found:

- The testing revealed some design issues with the Mini. When operating in an office environment, the robot's laser sensor cannot detect certain parts of obstacles, as shown in the image below.

Figure 4.25: Mini error in obstacle detection

One possible solution is to reduce the free space in the occupancy map increasing the cost of near obstacles using the `inflation_radius` or increasing the `footprint` of the robot. However, the dimensions of the chair legs are quite long, significantly reducing the available space to pass through doors which the planner then tries to avoid or fail to calculate a path. Another option is to relocate the LIDAR sensor underneath the robot. This would require a low-profile LIDAR and would create blind spots by the wheels if no additional sensors are added on top. A cheap solution is to add some ultrasonic sensors with a high field of view to catch these objects that the LIDAR doesn't see. Alternatively, adding a depth camera could provide a 3D occupancy grid, which can then be processed to generate a 2D costmap. This also applies to the Jobot, where its laser is positioned higher than the Mini (see 4.26 and 5.2), and so the laser cannot see the Mini, creating a potential collision risk. This problem is solved by the Open-RMF traffic manager because it knows the position of all the robots and can prevent the Jobot from moving too close to the Mini.

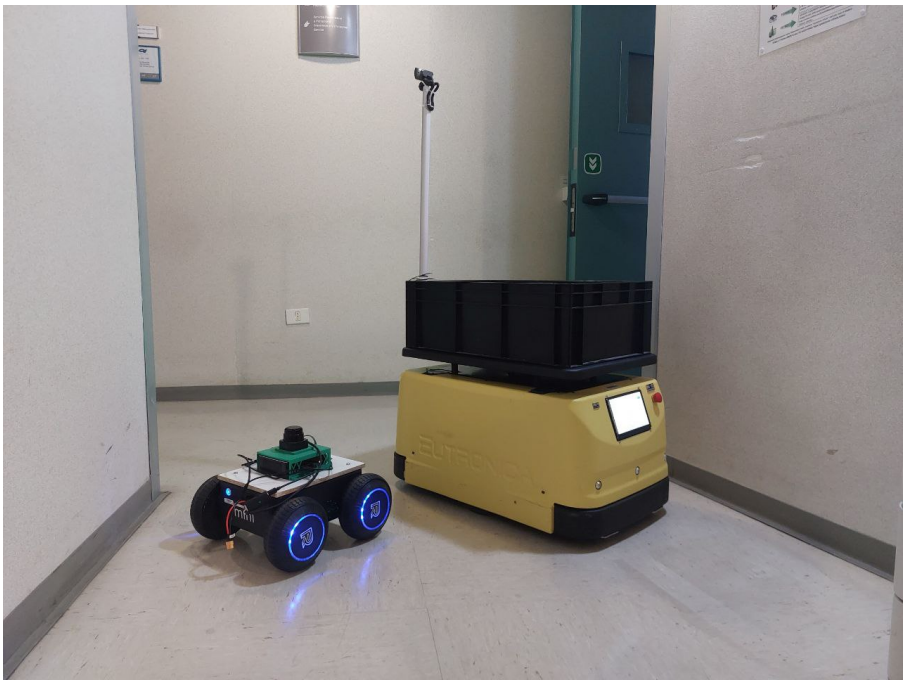


Figure 4.26: Jobot error in obstacle detection

- The ROS 2 infrastructure allows all devices within the same subnet to connect through multicasting. To avoid multiple nodes publishing to the same topic and creating errors in communication, several solutions are available:

  - some depending on the middleware used (cycloneDDS, FastRTS),
  - assigning each robot with a `namespace`,
  - associating each robot with a unique `ROS_DOMAIN_ID`.

  The solution that best applies to us and that is adopted in the Free Fleet package is the use of the `ROS_DOMAIN_ID`. This was also implemented in the simulation. Consequently, all important topics are shared through the Zenoh plugin, as described earlier.

- The D-star algorithm does not perform effectively within the ROS 2 planner ecosystem because the map update procedure does not allow the planner to detect obstacles, this implementation has been included in the roadmap.

- To improve the safety the Collision Monitor[2] was inserted. This node (see 4.27) uses the information from the sensors used to detect obstacles to reduce the velocity sent to the robot. If an object is inside the slowdown box, it reduces its velocity; it sends no velocity if an object is inside the stop box; or it reduces the velocity if it estimates that a collision using the approach box is possible in the future(see 4.28).

[2]https://github.com/ros-navigation/navigation2/tree/main/nav2_collision_monitor

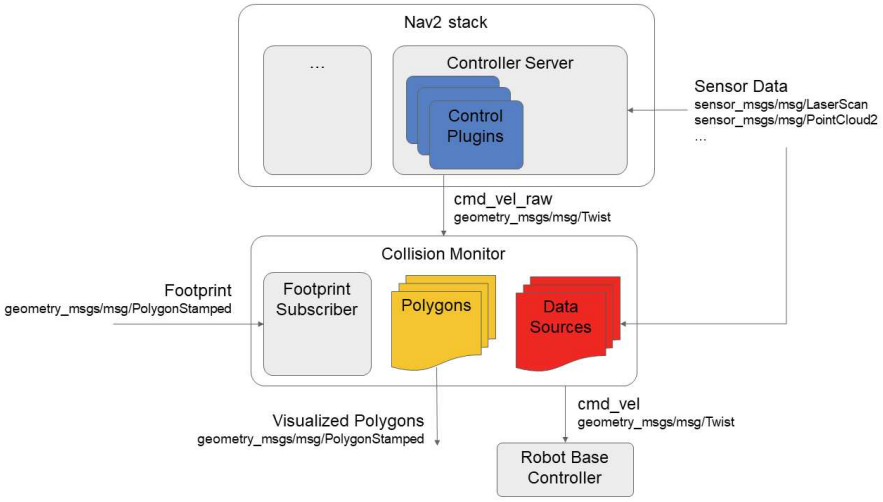


Figure 4.27: Collision monitor diagram

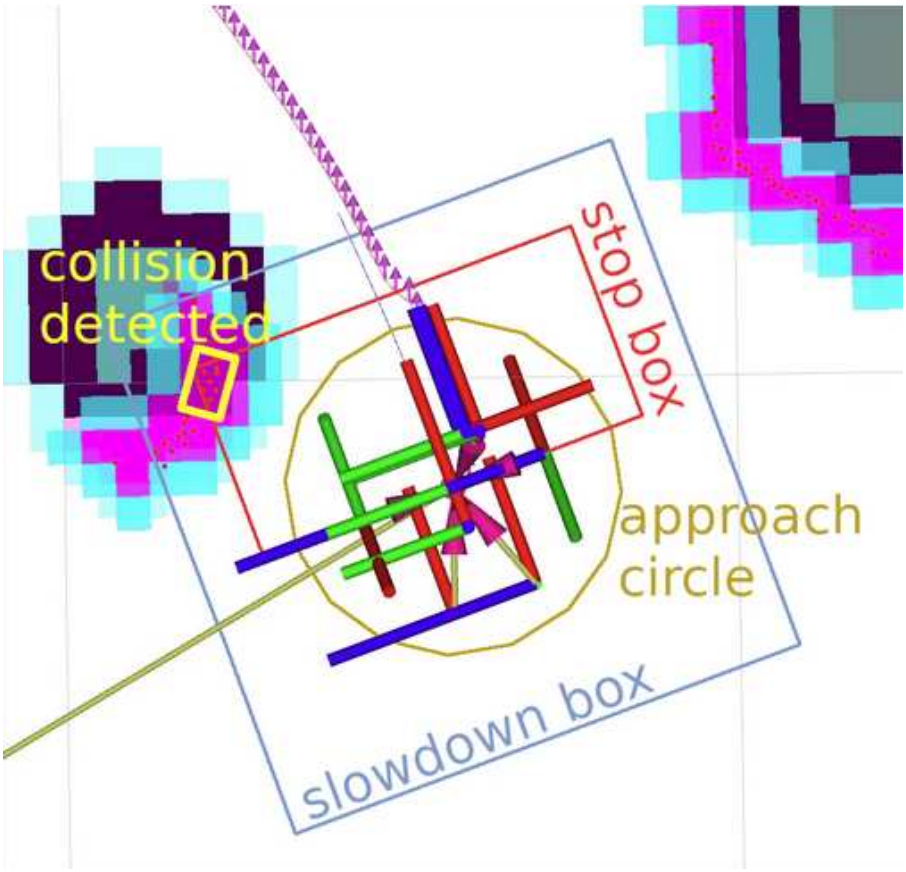


Figure 4.28: Collision monitor control boxes

- Additionally, we discovered misalignment between the planimetry image and the Cartographer map, as shown in the image below. It appears that at each major rotation, the system overestimates the robot's rotation angle during mapping.

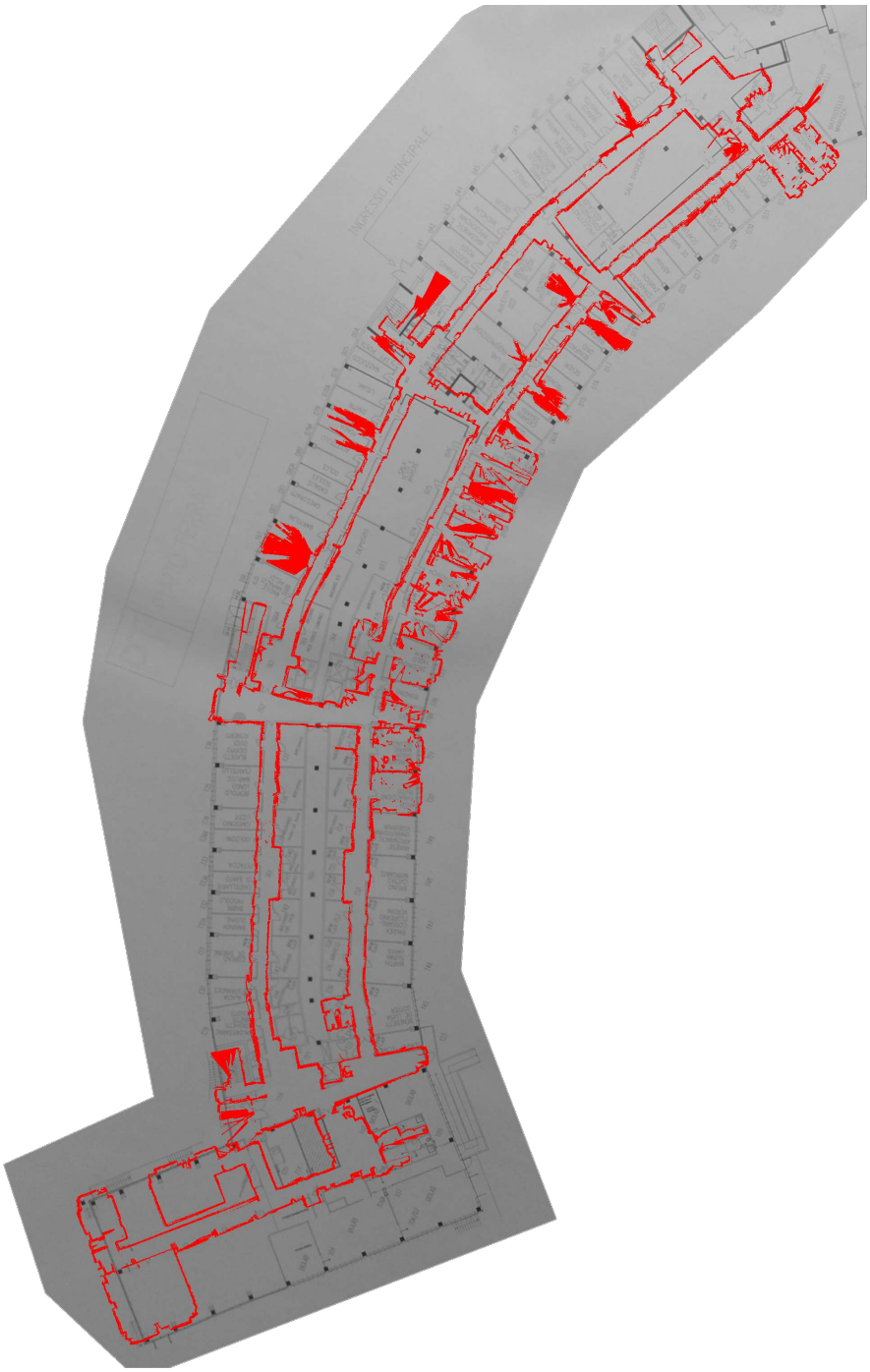


Figure 4.29: Overlap Cartographer Map and photo of floor plan

5. Presentation of results from testing and evaluation

This part is divided based on the things that have been completed:

- Porting the planner DStarGlobalPlanner from ROS 1 to ROS 2.
- "Building" two Autonomous Mobile Robots (AMR):
  - One from scratch and using the Rover Robotics Mini platform.
  - Proving the portability of the code structure used for the Jobot by updating it to ROS 2.
- Implementing the Open-RMF middleware using these robots.

5.1 Porting the planner

We have successfully ported the code to ROS 2 and used it for navigation. This is an output plan generated by the planner. More information about the functionality of the parameters is described in this paper [41].

Figure 5.1: Result obtained by the Dstar planner

In the end, the logical transformation from the `move_base` logic in ROS 1 to the Planner Server used in ROS 2 for the global planner is based on the application of more advanced functionalities of the programming languages used and the incorporation of the lifecycle structure. We also discovered that several improvements need to be made to enhance the performance of the planner Dstar within ROS 2.

5.2 Build 2 AMR in ROS 2

Both robots were implemented, both in simulation and in real. All the capabilities of Nav2 were utilized, along with a study of its parameters to ensure optimal control. The following pictures show the robots in simulation.

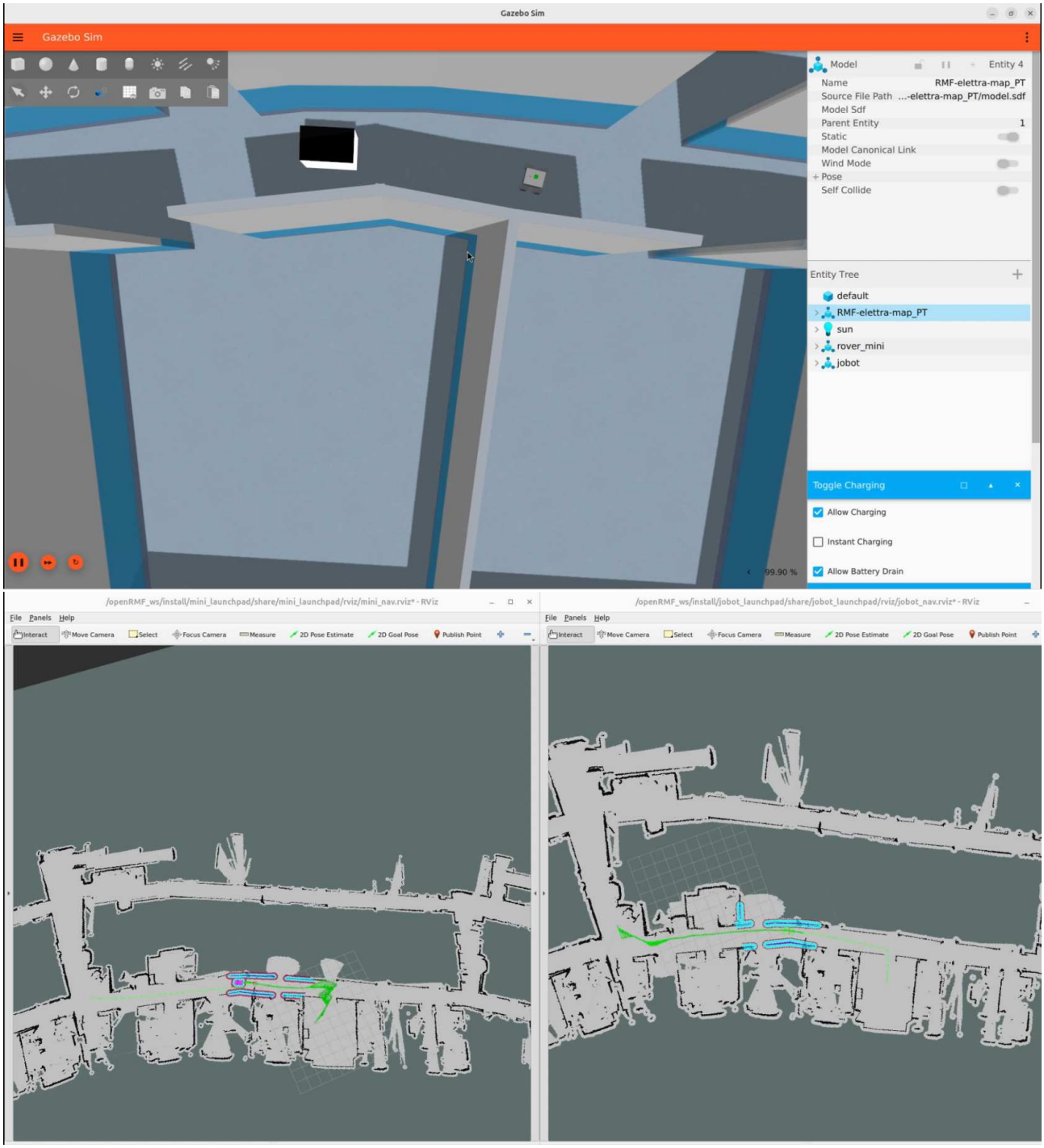


Figure 5.2: Both robots in simulation

5.2.1 In particular for the Mini

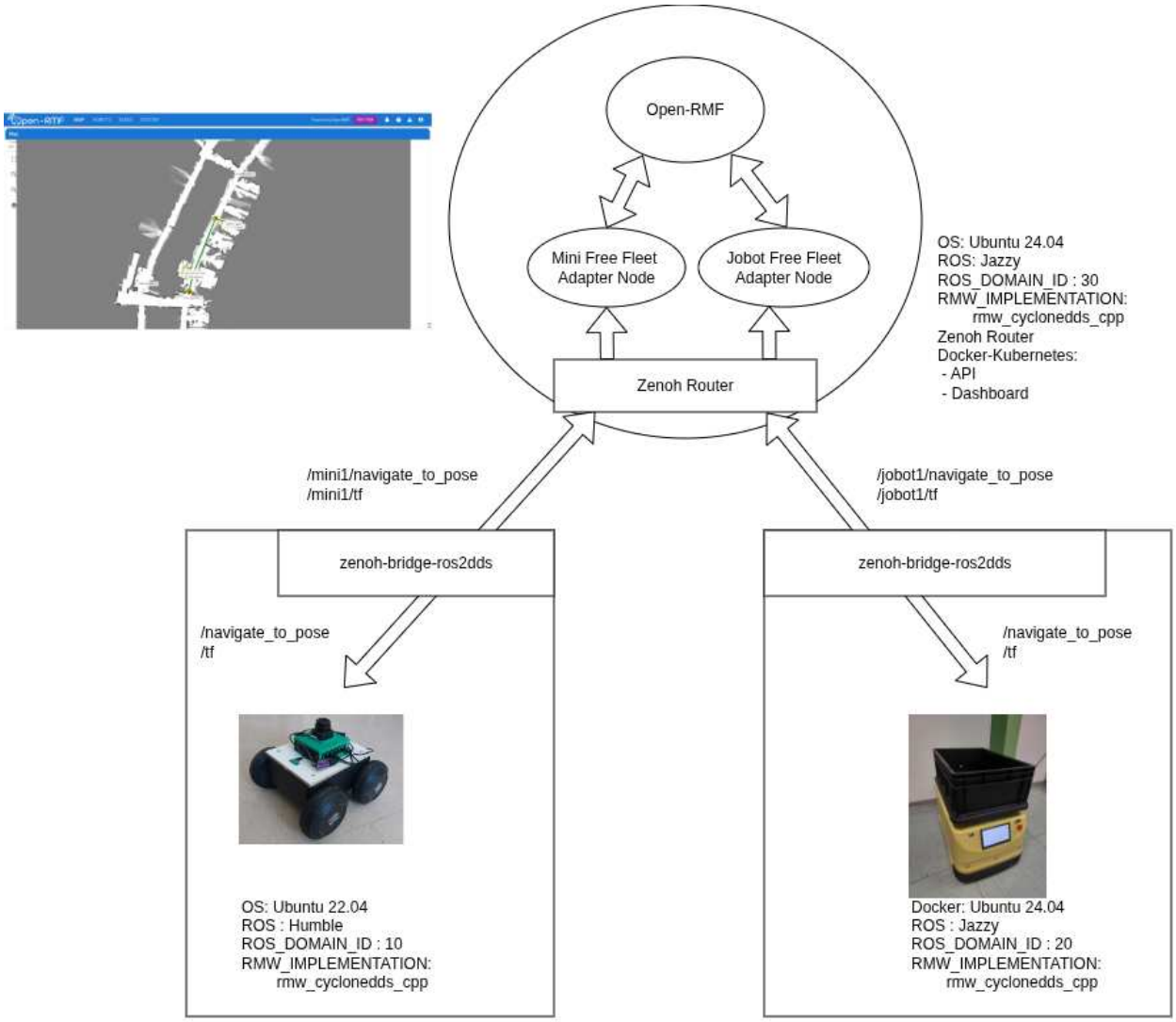
We found issues with its position estimation, which we resolved by tuning the parameters of the AMCL accordingly. We achieved the best performance with the following values for the process covariance matrix: v_{yaw} of 10.0 and for AMCL: $\alpha 1 = 0.01$, $\alpha 2 = 0.01$, $\alpha 3 = 0.1$ and $\alpha 4 = 0.0001$

5.3 In particolar for the Jobot

We converted all the code used to control the robot from ROS 1 to ROS 2, demonstrating the portability of that code structure. We added the Lifecycle implementation as part of the boot sequence of the robot with Nav2 and its sensors. We corrected the estimation in rotation by combining the velocity estimates from the robot and the gyroscope, and used the AMCL to correct the remaining error, obtaining these values: 320.0 for v_{yaw} of the process covariance matrix and for AMCL: $\alpha 1 = 0.01$, $\alpha 2 = 0.001$, $\alpha 3 = 0.001$ and $\alpha 4 = 0.001$

5.4 Open-RMF results

After following the steps described before (see 3.4), we obtained this structure:



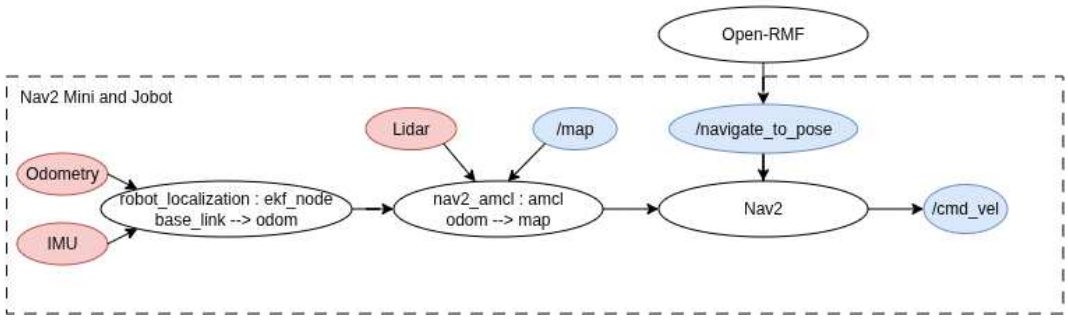


Figure 5.3: Ending Real Global Structure with a basic navigation diagram description

This structure allow us to start using the Open-RMF and send commands to the fleets. The following are some images of the navigation of the two robots in simulation and in reality:

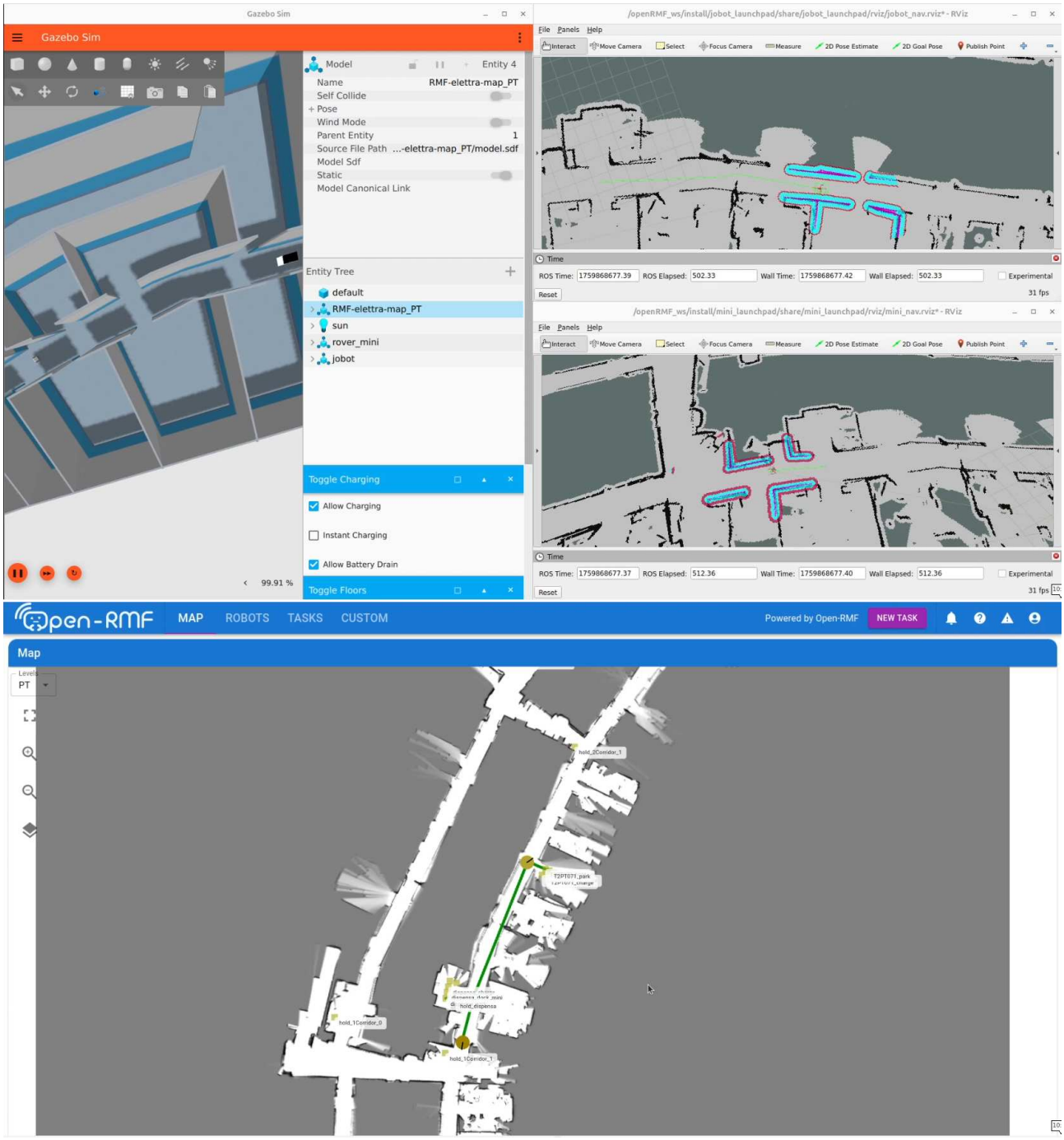


Figure 5.4: Images of the 2 robots with Open-RMF in the simulated environment

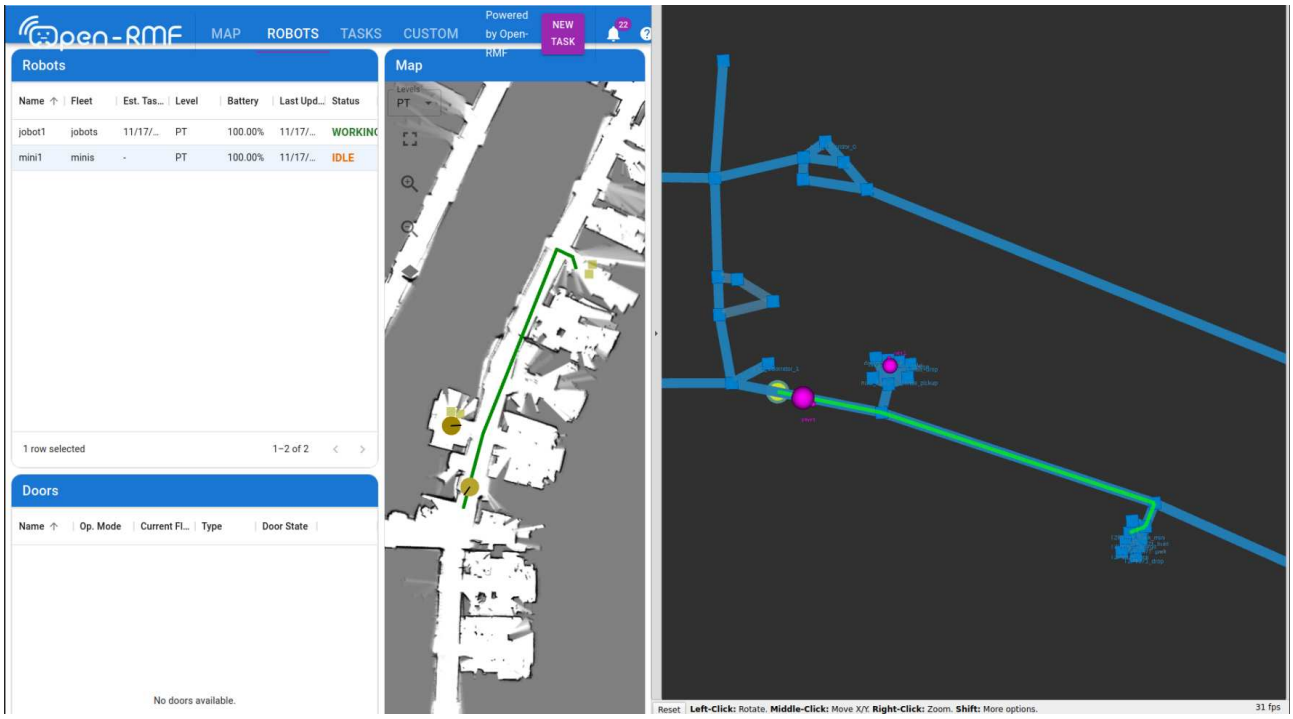


Figure 5.5: Picture of Open-RMF with the visualization on Rviz during real testing and without negotiation

The results are promising, and this solution can be implemented in our workspace. We tested the patrol task and obtained the following results:

- When asked individually, a robot moved to a location and reached it without issue.
- When asked to patrol a point without specifying the robot, Open-RMF performed a request and chose the robot that was nearest to perform the patrol task.
- It avoided the problem of congestion (in Figure 5.6b, it is possible to see the Mini waiting for the Jobot to pass the door).
- It solved the problem of lane negotiation (as visible in Figure 5.7, the Jobot, which was sent to perform the patrol task to the right, is letting the Mini, which was sent to perform the patrol task to the left, pass through the corridor, thereby avoiding that both robots occupy a single lane).

(a) Robots navigating under the control of Open-RMF

(b) Mini waiting the Jobot

Figure 5.6: Open-RMF results

Figure 5.7: Picture of the Open-RMF with the visualization on Rviz when negotiation is happening. The Jobot is allowing the Mini to pass.

6. Comparative analysis on the port system

6.1 Planner calculation time comparison

In this section, we show the time required for the global planner node to calculate different paths. we compared the performance between the DStar global planner and the NavfnPlanner over three path: from the office to the storage room (used as base in the setting up the parameters), from the office to the entrance of the building (fig: 6.1a,6.2a)and from the entrance to the our CAD designers location(fig:6.1b,6.2b). The results were obtained in a Jazzy Docker image with a Intel i5-1345U CPU and adding the flag:

```
-cmake-args -DCMAKE_BUILD_TYPE=Release
```

to the call of `colcon build`.

Start->End	ROS 2 D-start (s)	ROS 2 NavfnPlanner (s)
office -> storage room	1.00	0.020
office -> entrance	5.47	0.150
entrance -> CAD designers	5.36	0.130

Table 6.1: result planner first request

After the first calculated path:

Start->End	ROS 2 D-start (s)	ROS 2 NavfnPlanner (s)
office -> storage room	0.004	0.022
office -> entrance	0.009	0.156
entrance -> CAD designers	0.011	0.135

Table 6.2: result planner first request

There are some results generated using the planner Dstar:

(a) office -> entrance

(b) entrance -> CAD designers

Figure 6.1: Path plans generated by Dstar planner

and these are some generated by the NavfnPlanner:

(a) office -> entrance

(b) entrance -> CAD designers

Figure 6.2: Path plans generated by NavfnPlanner planner

The result showed us:

- The time to calculate the first plan using DStar is much higher than the NavfnPlanner.
- After the first calculated path, the algorithm returns the same path, which explains this fast response.

We see that optimization must be made to the ROS 2 global planner's code to unveil the potential of the D* algorithm, since it is not fully designed to work together with ROS.

7. Conclusion

With this work, we have demonstrated the potential of ROS 2 for creating industrial robotics solutions.
Through porting the code, we discovered the Lifecycle and Compose options for creating nodes. The new Lifecycle state machine available for nodes allows for better management of interactions between them. It provides control in the event that a node crashes or fails to change state, which could affect the behavior of other nodes. The Composite feature enables optimized data transmission.
Furthermore, when creating Autonomous Mobile Robots (AMRs), the modularity and the extensive range of solutions for each problem facilitate easy modifications to plugins and nodes, allowing for rapid implementation. The configurability of nodes leads to performance improvements with minimal code changes.
Nav2's modularity allowed for the quick import of code from our planner, enabling us to evaluate its advantages and disadvantages.
Finally, Open-RMF demonstrates considerable potential. Thanks to its tools, it enables the rapid creation of complex navigation graph paths. Additionally, it supports brand-independent robot infrastructures, facilitates the easy import of different robots with a dedicated package for ROS robots, controls structural elements such as doors and lifts, and allows for the performance of different tasks without conflicts with other robots.
In the office environment, we observe the advantages of combining AMRs with Open-RMF, as the robots are capable of moving while avoiding collisions with people, while Open-RMF manages to prevent conflicts with other robots.

8. Future work

Using OpenRMF, we can create nodes that, thanks to an extensive language model, can send tasks to the robot[1]. We aim to implement this functionality in our warehouse, allowing us to effectively control the robot. Additionally, with the Fleet Adapter, which forms the basis of the Free Fleet package, we can manage other robots in our fleet. We've also identified ways to improve the planner.

[1] https://github.com/robotmcp/ros-mcp-server

9. Appendices

9.1 Simulation

In the code, following the `README.md` files and inside the `.devcontainer` folders, Dockerfiles and instruction how to use them are available for simulation. All container obtained by these images can be used directly to simulate the code.
When the use of a program inside the container with a graphical interface is required, you have to share the access to the X server using the command `xhost local:root`.
At least 3 GB of VRAM are required for having a fluid simulation.

Bibliography

[1] N. Abu et al. "A Comprehensive Overview of Classical and Modern Route Planning Algorithms for Self-Driving Mobile Robots". In: *Journal of Robotics and Control (JRC)* 3 (Sept. 2022), pp. 666–678. DOI: 10.18196/jrc.v3i5.14683.

[2] Sy-Hung Bach, Phan-Bui Khoi, and Soo-Yeong Yi. "Application of QR Code for Localization and Navigation of Indoor Mobile Robot". In: *IEEE Access* 11 (2023), pp. 28384–28390. DOI: 10.1109/ACCESS.2023.3250253.

[3] Utkarsha Bhave et al. "Automating the Operation of a 3D-Printed Unmanned Ground Vehicle in Indoor Environments". In: *2019 Systems and Information Engineering Design Symposium (SIEDS)*. 2019, pp. 1–6. DOI: 10.1109/SIEDS.2019.8735597.

[4] L. Bruzzone and G. Quaglia. "Review article: locomotion systems for ground mobile robots in unstructured environments". In: *Mechanical Sciences* 3.2 (2012), pp. 49–62. DOI: 10.5194/ms-3-49-2012. URL: https://ms.copernicus.org/articles/3/49/2012/.

[5] Luca Carlone et al. "Part1 Prelude". In: *SLAM Handbook. From Localization and Mapping to Spatial Intelligence*. Ed. by Luca Carlone et al. Cambridge University Press.

[6] Chao Cheng et al. "Recent Advancements in Agriculture Robots: Benefits and Challenges". In: *Machines* 11.1 (2023). ISSN: 2075-1702. DOI: 10.3390/machines11010048. URL: https://www.mdpi.com/2075-1702/11/1/48.

[7] Michele Colledanchise and Petter Ögren. *Behavior Trees in Robotics and AI*. July 2018. DOI: 10.1201/9780429489105. URL: http://dx.doi.org/10.1201/9780429489105.

[8] Sagarnil Das. *Robot localization in a mapped environment using Adaptive Monte Carlo algorithm*. 2025. arXiv: 2501.01153 [cs.RO]. URL: https://arxiv.org/abs/2501.01153.

[9] Tully Foote. "tf: The transform library". In: *Technologies for Practical Robot Applications (TePRA), 2013 IEEE International Conference on*. Open-Source Software workshop. 2013, pp. 1–6. DOI: 10.1109/TePRA.2013.6556373.

[10] D. Fox, W. Burgard, and S. Thrun. "The dynamic window approach to collision avoidance". In: *IEEE Robotics & Automation Magazine* 4.1 (1997), pp. 23–33. DOI: 10.1109/100.580977.

[11] Giuseppe Fragapane et al. "Autonomous Mobile Robots in Hospital Logistics". In: *Advances in Production Management Systems. The Path to Digital Transformation and Innovation of Production Management Systems*. Ed. by Bojan Lalic et al. Cham: Springer International Publishing, 2020, pp. 672–679. ISBN: 978-3-030-57993-7.

[12] N.J. Gordon, D.J. Salmond, and A.F.M. Smith. "Novel approach to nonlinear/non-Gaussian Bayesian state estimation". In: *IEE Proceedings F (Radar and Signal Processing)* 140 (2 1993), pp. 107–113. DOI: 10.1049/ip-f-2.1993.0015. eprint: https://digital-library.theiet.org/doi/pdf/10.1049/ip-f-2.1993.0015. URL: https://digital-library.theiet.org/doi/abs/10.1049/ip-f-2.1993.0015.

[13] Steve Grehl et al. "Mining-rox–mobile robots in underground mining". In: *Proceedings of the Third International Future Mining Conference, Sydney, Australia*. 2015, pp. 4–6.

[14] Wolfgang Hess et al. "Real-Time Loop Closure in 2D LIDAR SLAM". In: *2016 IEEE International Conference on Robotics and Automation (ICRA)*. 2016, pp. 1271–1278.

[15] Universidad Rey Juan Carlos Intelligent Robotics Lab. *EasyNavigation (EasyNav)*. 2025. URL: https://easynavigation.github.io/ (visited on 11/10/2025).

[16] Russell Keith and Hung Manh La. *Review of Autonomous Mobile Robots for the Warehouse Environment*. 2024. arXiv: 2406.08333 [cs.RO]. URL: https://arxiv.org/abs/2406.08333.

[17] YoungEon Kim, Dong Yeop Kim, and Keunhwan Kim. "Traversability Assessment and Path Planning Using Obstacle Recognition Information for Multi-Robot System". In: *2024 24th International Conference on Control, Automation and Systems (ICCAS)*. 2024, pp. 1670–1671. DOI: 10.23919/ICCAS63016.2024.10773345.

[18] Maria Kyrarini et al. "A Survey of Robots in Healthcare". In: *Technologies* 9.1 (2021). ISSN: 2227-7080. DOI: 10.3390/technologies9010008. URL: https://www.mdpi.com/2227-7080/9/1/8.

[19] Renjie Liang et al. "UAV Flight Control System Design Based on Ardupilot". In: *2024 7th International Conference on Mechanical, Control and Computer Engineering (ICMCCE)*. 2024, pp. 321–328. DOI: 10.1109/ICMCCE63640.2024.11163478.

[20] Yuxiang Liu et al. "Autonomous Vehicle Based on ROS2 for Indoor Package Delivery". In: *2023 28th International Conference on Automation and Computing (ICAC)*. 2023, pp. 1–7. DOI: 10.1109/ICAC57885.2023.10275227.

[21] Steve Macenski and Ivona Jambrecic. "SLAM Toolbox: SLAM for the dynamic world". In: *Journal of Open Source Software* 6.61 (2021), p. 2783. DOI: 10.21105/joss.02783. URL: https://doi.org/10.21105/joss.02783.

[22] Steve Macenski et al. *Impact of ROS 2 Node Composition in Robotic Systems*. 2023. arXiv: 2305.09933 [cs.RO]. URL: https://arxiv.org/abs/2305.09933.

[23] Steven Macenski et al. "Robot Operating System 2: Design, architecture, and uses in the wild". In: *Science Robotics* 7.66 (2022), eabm6074. DOI: 10.1126/scirobotics.abm6074. URL: https://www.science.org/doi/abs/10.1126/scirobotics.abm6074.

[24] Steven Macenski et al. "The Marathon 2: A Navigation System". In: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2020.

[25] Alexey Merzlyakov and Steven Macenski. "A Comparison of Modern General-Purpose Visual SLAM Approaches". In: *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2021.

[26] T. Moore and D. Stouch. "A Generalized Extended Kalman Filter Implementation for the Robot Operating System". In: *Proceedings of the 13th International Conference on Intelligent Autonomous Systems (IAS-13)*. Springer, 2014.

[27] Edwin Olson. "AprilTag: A robust and flexible visual fiducial system". In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2011, pp. 3400–3407.

[28] Ali Gurcan Ozkil et al. "Service robots for hospitals: A case study of transportation tasks in a hospital". In: *2009 IEEE International Conference on Automation and Logistics*. 2009, pp. 289–294. DOI: 10.1109/ICAL.2009.5262912.

[29] Gerardo Pardo-Castellote, Bert Farabaugh, and Rick Warren. "An introduction to DDS and data-centric communications". In: *Real-Time Innovations* 61 (2005).

[30] Francisco J. Perez-Grau et al. "Introducing autonomous aerial robots in industrial manufacturing". In: *Journal of Manufacturing Systems* 60 (2021), pp. 312–324. ISSN: 0278-6125. DOI: https://doi.org/10.1016/j.jmsy.2021.06.008. URL: https://www.sciencedirect.com/science/article/pii/S0278612521001321.

[31] Dev Bahadur Poudel. "Coordinating hundreds of cooperative, autonomous robots in a warehouse". In: *Jan* 27.1-13 (2013), p. 26.

[32] Morgan Quigley et al. "ROS: an open-source Robot Operating System". In: *ICRA workshop on open source software*. Vol. 3. 3.2. Kobe. 2009, p. 5.

[33] Francisco Rubio, Francisco Valero, and Carlos Llopis-Albert. "A review of mobile robots: Concepts, methods, theoretical framework, and applications". In: *International Journal of Advanced Robotic Systems* 16.2 (2019), p. 1729881419839596. DOI: 10.1177/1729881419839596. eprint: https://doi.org/10.1177/1729881419839596. URL: https://doi.org/10.1177/1729881419839596.

[34] DV Lu A. Merzlyakov M. Ferguson S. Macenski T. Moore. "From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2". In: *Robotics and Autonomous Systems* (2023).

[35] Angelo Salzillo. "Robot fleet control for mining applications using OpenRMF and OpenTCS". PhD thesis. Politecnico di Torino, 2024.

[36] Kavan Singh Sikand et al. *Robofleet: Open Source Communication and Management for Fleets of Autonomous Robots*. 2021. arXiv: 2103.06993 [cs.RO]. URL: https://arxiv.org/abs/2103.06993.

[37] Rita Tomé Silva et al. "AGVs vs AMRs: A Comparative Study of Fleet Performance and Flexibility". In: *2024 7th Iberian Robotics Conference (ROBOT)*. 2024, pp. 1–6. DOI: 10.1109/ROBOT61475.2024.10797381.

[38] Luigi Tagliavini et al. "Wheeled mobile robots: state of the art overview and kinematic comparison among three omnidirectional locomotion strategies". In: *Journal of intelligent & robotic systems* 106.3 (2022), p. 57.

[39] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *Probabilistic Robotics (Intelligent Robotics and Autonomous Agents)*. The MIT Press, 2005. ISBN: 0262201623.

[40] Victor Mayoral Vilches et al. *SROS2: Usable Cyber Security Tools for ROS 2*. 2022. arXiv: 2208.02615 [cs.CR]. URL: https://arxiv.org/abs/2208.02615.

[41] Andrey Vukolov. "D-Star-Based Optimized Trajectory Planner for Mobile Robots Operating in Dense Environments". In: *New Trends in Mechanism and Machine Science*. Ed. by Giulio Rosati, Alessandro Gasparetto, and Marco Ceccarelli. Cham: Springer Nature Switzerland, 2024, pp. 294–301. ISBN: 978-3-031-67295-8.

[42] Andrey Vukolov. *ElettraSciComp/witmotion_IMU_QT: Version 0.9.17*. Version v0.9.17-alpha. 2022. DOI: 10.5281/zenodo.7017118. URL: https://doi.org/10.5281/zenodo.7017118.

[43] Andrey Vukolov et al. "Universal Configurable Navigation and Control System for Industrial Unmanned Ground Vehicles with Differential Chassis". In: *Advances in Mechanism and Machine Science*. Ed. by Masafumi Okada. Cham: Springer Nature Switzerland, 2023, pp. 287–301. ISBN: 978-3-031-45770-8.

[44] Grady Williams et al. "Aggressive driving with model predictive path integral control". In: *2016 IEEE International Conference on Robotics and Automation (ICRA)*. 2016, pp. 1433–1440. DOI: 10.1109/ICRA.2016.7487277.

[45] Wei Yu et al. "Analysis and Experimental Verification for Dynamic Modeling of A Skid-Steered Wheeled Vehicle". In: *IEEE Transactions on Robotics* 26.2 (2010), pp. 340–353. DOI: 10.1109/TRO.2010.2042540.